Programming Reference

What Is in This Part?

This part contains the following chapters:

Variables Declaration

What Is in This Chapter?

This chapter contains the following sections:

Declaration

What Is in This Section?

This section contains the following topics:

General Information

Overview

You can declare variables:
- in the Variables view of the Software Catalog
- in the Declaration Editor of a POU
- via the Auto Declare dialog box
- in a GVL editor

The kind (in the tabular declaration editor it is named **Scope**) of the variables to be declared is specified by the keywords embracing the declaration of one or several variables. In the textual declaration editor, the common variable declaration is embraced by `VAR` and `END_VAR`.

For further variable declaration scopes, refer to:
- `VAR_INPUT`
- `VAR_OUTPUT`
- `VAR_IN_OUT`
- `VAR_GLOBAL`

- VAR_TEMP
- VAR_STAT
- VAR_EXTERNAL
- VAR_CONFIG

The variable type keywords may be supplemented by attribute keywords.

Example: `RETAIN (VAR_INPUT RETAIN)`

Syntax

Syntax for variable declaration:

<Identifier> {`AT` <address>}:<data type> {:=<initialization>};

The parts in braces {} are optional.

Identifier

The identifier is the name of a variable.

Consider the following facts when defining an identifier.

- no spaces or special characters allowed
- no case-sensitivity: `VAR1`, `Var1` and `var1` are all the same variable
- recognizing the underscore character: `A_BCD` and `AB_CD` are considered 2 different identifiers. Do not use more than 1 underscore character in a row.
- unlimited length
- recommendations concerning multiple use (see next paragraph)

Also, consider the recommendations given in chapter Recommendations on the Naming of Identifiers.

Multiple Use of Identifiers (Namespaces)

The following outlines the regulations concerning the multiple use of identifiers:

- Do not create an identifier that is identical to a keyword.
- Duplicate use of identifiers is not allowed locally.
- Multiple use of an identifier is allowed globally: a local variable can have the same name as a global one. In this case, the local variable within the POU will have priority.
- A variable defined in a global variable list (GVL) can have the same name as a variable defined in another global variable list (GVL). In this context, consider the following IEC 61131-3 extending features:
  - Global scope operator: an instance path starting with a dot (`.`) opens a global scope. So, if there is a local variable, for example `ivar`, with the same name as a global variable, `.ivar` refers to the global variable.

- You can use the name of a global variable list (GVL) as a namespace for the included variables. You can declare variables with the same name in different global variable lists (GVL). They can be accessed specifically by preceding the variable name with the list name.
  Example
  ```
  globlist1.ivar := globlist2.ivar;
  (* ivar from globlist2 is copied to ivar in GVL globlist1 *)
  ```
- Variables defined in a global variable list of an included library can be accessed according to syntax <library namespace>.<name of GVL>.<variable>.
  Example:
  ```
  globlist1.ivar := lib1.globlist1.ivar
  (* ivar from globlist1 in library lib1 is copied to ivar in GVL globlist1 *)
  ```

- For a library also, a namespace is defined when it gets included via the **Library Manager**. So you can access a library module or variable by <library namespace>.<modulename|variablename>. Consider that, in case of nested libraries, the namespaces of all libraries concerned have to be stated successively.
  Example: If `Lib1` is referenced by `Lib0`, the module `fun` being part of `Lib1` is accessed by `Lib0.Lib1.fun`:
  `ivar := Lib0.Lib1.fun(4, 5);` (* return value of `fun` is copied to variable `ivar` in the project *)
  NOTE: Once the checkbox **Publish all IEC symbols to that project as if this reference would have been included there directly.** has been activated within the **Properties** dialog box of the referenced library `Lib`, the module `fun` may also be accessed directly via `Lib0.fun`.

- Variables declared in GVLs or POUs of the **Global** node of the **Applications tree** can be accessed by prefixing them with the operator `"__POOL."`.

AT <address>

You can link the variable directly to a definite address using the keyword `AT`.

In function blocks, you can also specify variables with incomplete address statements. In order that such a variable can be used in a local instance, an entry has to exist for it in the variable configuration.

Type

Valid data type, optionally extended by an :=< initialization>.

Pragma Instructions

Optionally, you can add pragma instructions in the declaration part of an object in order to affect the code generation for various purposes.

Hints

Automatic declaration of variables is also possible.

For faster input of the declarations, use the shortcut mode.

Recommendations on the Naming of Identifiers

Overview

Identifiers are defined:
- at the declaration of variables (variable name)
- at the declaration of user-defined data types
- at the creation of POUs (functions, function blocks, programs)

In addition to the general items to be considered when defining an identifier (refer to chapter General Information on variables declaration), consider the following recommendations in order to make the naming as unique as possible:
- Variable names
- Variable names in Libraries
- User-defined data types (DUTs) in Libraries
- Functions, Function blocks, Programs (POU), Actions
- POUs in Libraries

Variable Names

For naming variables in applications and libraries, follow the Hungarian notation as far as possible.

Find for each variable a meaningful, short description. This is used as the base name. Use a capital letter for each word of the base name. Use small letters for the rest (example: `FileSize`).

Data Type	Lower Limit	Upper Limit	Information Content	Prefix	Comment
BOOL	FALSE	TRUE	1 bit	`x*`	–
				`b`	reserved
BYTE	–	–	8 bit	`by`	bit string, not for arithmetic operations

WORD	–	–	16 bit	w	bit string, not for arithmetic operations
DWORD	–	–	32 bit	dw	bit string, not for arithmetic operations
LWORD	–	–	64 bit	lw	not for arithmetic operations
SINT	–128	127	8 bit	si	–
USINT	0	255	8 bit	usi	–
INT	–32,768	32,767	16 bit	i	–
UINT	0	65,535	16 bit	ui	–
DINT	–2,147,483,648	2,147,483,647	32 bit	di	–
UDINT	0	4,294,967,295	32 bit	udi	–
LINT	-2^{63}	$2^{63}-1$	64 bit	li	–
ULINT	0	$2^{64}-1$	64 bit	uli	–
REAL	–	–	32 bit	r	–
LREAL	–	–	64 bit	lr	–
STRING	–	–	–	s	–
WSTRING	–	–	–	ws	–
TIME	–	–	–	tim	–
TIME_OF_DAY	–	–	–	tod	–
DATE_AND_TIME	–	–	–	dt	–
DATE	–	–	–	date	–
ENUM	–	–	16 bit	e	–
POINTER	–	–	–	p	–

ARRAY	–	–	–	a	–

* intentionally for boolean variables x is chosen as a prefix in order to differentiate from BYTE and also in order to accommodate the perception of an IEC programmer (see addressing `%IX0.0`).

Simple declaration

Examples for simple declarations:

```
bySubIndex: BYTE;
sFileName: STRING;
udiCounter: UDINT;
```

Nested declaration

Example for a nested declaration where the prefixes are attached to each other in the order of the declarations:

```
pabyTelegramData: POINTER TO ARRAY [0..7] OF BYTE;
```

Function block instances and variables of user-defined data types

Function block instances and variables of user-defined data types get a shortcut for the function block or the data type name as a prefix (for example: `sdo`).

Example

```
cansdoReceivedTelegram: CAN_SDOTelegram;
```

```
TYPE CAN_SDOTelegram :       (* prefix: sdo *)
STRUCT
   wIndex:WORD;
   bySubIndex:BYTE;
   byLen:BYTE;
   aby: ARRAY [0..3] OF BYTE;
END_STRUCT
END_TYPE
```

Local constants

Local constants (c) start with prefix `c` and an attached underscore, followed by the type prefix and the variable name.

Example

```
VAR CONSTANT
   c_uiSyncID: UINT := 16#80;
END_VAR
```

Global variables and global constants

Global variables are prefixed by `g_` and global constants are prefixed by `gc_`.

Example

```
VAR_GLOBAL
   g_iTest:  INT;
END_VAR
VAR_GLOBAL  CONSTANT
   gc_dwExample:  DWORD;
END_VAR
```

Variable Names in Libraries

Structure

Basically, refer to the above description for variable names. Use the library namespace as prefix, when accessing a variable in your application code.

Example

```
g_iTest:  INT;  (declaration)
CAN.g_iTest  (implementation,  call  in  an  application  program
```

User-Defined Data Types (DUT) in Libraries

Structure

The name of each structure data type consists of a short expressive description (for example, `SDOTelegram`) of the structure.

Example (in library with namespace `CAL`):

```
TYPE  Day  :(
MONDAY,
TUESDAY,
WEDNESDAY,
THURSDAY,
FRIDAY,
SATURDAY,
SUNDAY);
```

Declaration:

```
eToday:  CAL.Day;
```

Use in application:

```
IF eToday = CAL.Day.MONDAY THEN
```

NOTE: Consider the usage of the namespace when using DUTs or enumerations declared in libraries.

Functions, Function Blocks, Programs (POU), Actions

The names of functions, function blocks, and programs are prefixed by an expressive short name of the POU (for example, `SendTelegram`). As with variables, the first letter of a word of the POU name should always be a capital letter whereas the others should be small letters. It is recommended to compose the name of the POU of a verb and a substantive.

Example

```
FUNCTION_BLOCK SendTelegram (* prefix: canst *)
```

In the declaration part, provide a short description of the POU as a comment. Further on, the inputs and outputs should be provided with comments. In case of function blocks, insert the associated prefix for set-up instances directly after the name.

Actions

Actions do not get a prefix. Only those actions that are to be called only internally that is by the POU itself, start with `prv_`.

POUs in Libraries

Structure

For creating method names, the same rules apply as for actions. Enter English comments for possible inputs of a method. Add a short description of a method to its declaration. Start interface names with letter `I`; for example, `ICANDevice`.

NOTE: Consider the usage of the namespace when using POUs declared in libraries.

Variables Initialization

Default Initialization Value

The default initialization value is 0 for all declarations, but you can add user-defined initialization values in the declaration of each variable and data type.

User-Defined Initialization Values

The user-defined initialization is brought about by the assignment operator `:=` and can be any valid ST expression. Thus, constant values as well as other variables or functions can be used to define the initialization value. Verify that a variable used for the

initialization of another variable is already initialized itself.

Example of valid variable initializations:

```
VAR
var1:INT := 12;                    * Integer variable with initial value of 12. *
x : INT := 13 + 8;                  * Integer value defined an expression with literal values.*
y : INT := x + fun(4);             * Integer value defined by an expression
containing a function call. NOTE: Be sure that any variables used in
the variable initialization have already been defined. *
z : POINTER TO INT := ADR(y); * POINTER is not described by the IEC61131-3:
Integer value defined by an address function; NOTE: The pointer will
not be initialized if the declaration is modified online. *
END_VAR
```

Further Information

For further information, refer to the following descriptions:
- initializing arrays
- initialization of structures
- initialization of a variable with a subrange type

NOTE: Variables of global variables lists (GVL) are initialized before local variables of a POU.

NOTE: As from SoMachine version 4.0, variables in a function block are initialized in the following order: First, the constants in accordance with the order of their declarations, then the other variables in accordance with the order of their declarations.

For further information regarding initialization order, refer to the Attribute global_init_slot.

Declaration

Declaration Types

You can declare variables manually by using the textual or tabular declaration editor or automatically like explained in this chapter.

Automatic Autodeclaration

You can define in the **Options** dialog box, category **Text editor > Editing**, that the **Auto Declare** dialog box should open as soon as a not yet declared string is entered in the implementation part of an editor and the ENTER key is pressed. This dialog box supports the declaration of the variable.

Manual Autodeclaration

To open the **Auto Declare** dialog box manually:
- execute the command **Auto Declare**, which by default is available in the **Edit** menu or
- press the keys SHIFT+F2

If you select an already declared variable before opening the **Auto Declare** *dialog box*, you can edit the declaration of this variable.

Shortcut Mode

Overview

The declaration editor and the other text editors where declarations are performed support the shortcut mode.

Activate this mode by pressing CTRL+ENTER when you end a line of declaration.

It allows you to use shortcuts instead of completely typing the declaration.

Supported Shortcuts

The following shortcuts are supported:
- All identifiers up to the last identifier of a line will become declaration variable identifiers.
- The type of declaration is determined by the last identifier of the line.

In this context, the following replacements are performed:

`B` or `BOOL`	is replaced by	BOOL
`I` or `INT`		INT
`R` or `REAL`		REAL
`S` or `string`		STRING

- If no type has been established through these rules, automatically BOOL is the type and the last identifier will not be used as a type (see example 1).
- Every constant, depending on the type of declaration, will turn into an initialization or a string (see examples 2 and 3).
- An address (as in `%MD12`) is extended by the `AT` keyword (see example 4).
- A text after a semicolon (;) becomes a comment (see example 4).
- All other characters in the line are ignored (see, for example, the exclamation point in example 5).

Examples

Example No.	Shortcut	Resulting Declaration
1	`A`	`A: BOOL;`
2	`A B I 2`	`A, B: INT := 2;`
3	`sX S 2; A string`	`sX:STRING(2); // A string`
4	`X %MD12 R 5; Real Number`	`X %MD12 R 5 Real Number`
5	`B !`	`B: BOOL;`

AT Declaration

Overview

In order to link a project variable with a definite address, you can assign variables to an address in the **I/O Mapping** view of a device in the controller configuration (device editor). Alternatively you can enter this address directly in the declaration of the variable.

Syntax

<identifier> `AT` <address> : <data type>;

A valid address has to follow the keyword `AT`. For further information, refer to the Address description. Consider possible overlaps in case of byte addressing mode.

This declaration allows assigning a meaningful name to an address. Any changes concerning an incoming or outgoing signal may only be done in a single place (for example, in the declaration).

Consider the following when choosing a variable to be assigned to an address:

- Variables requiring an input cannot be accessed by writing. The compiler intercepts this detecting an error.
- `AT` declarations only can be used with local or global variables. They cannot be used with input and output variables of POUs.
- `AT` declarations are not allowed in persistent variable lists.
- If `AT` declarations are used with structure or function block members, all instances will access the same memory location of this structure / function block. This corresponds to static variables in classic programming languages such as C.
- The memory layout of structures is determined by the target as well.

Examples

```
xCounterHeat7 AT %QX0.0: BOOL;
xLightCabinetImpulse AT %IX7.2: BOOL;
xDownload AT %MX2.2: BOOL;
```

Note

If boolean variables are assigned to a BYTE, WORD or DWORD address, they occupy 1 byte with TRUE or FALSE, not just the first bit after the offset.

The memory size for input, output, and memory data (declarations with `AT %I`, `%Q` and `%M`) is predefined by the target device and can be overwritten in the *properties of an application object* for PacDrive controllers (PacDrive LMC Eco, PacDrive LMC Pro/Pro2).

Keywords

Overview

Write keywords in uppercase letters in the editors.

The following strings are reserved as keywords. They cannot be used as identifiers for variables or POUs:

- `-`
- `&`
- `(`
- `)`
- `*`
- `,`
- `.`
- `..`
- `/`
- `:`
- `:=`
- `;`
- `[`
- `]`
- `^`
- `__CATCH`
- `__CHECKLICENSE`
- `__CHECKLICENSEBIT`

- __DELETE
- __ENDTRY
- __FINALLY
- __ISVALIDREF
- __MEMORYBARRIER
- __NEW
- __QUERYINTERFACE
- __QUERYPOINTER
- __THROW
- __TRY
- __XADO
- I
- +
- <
- <=
- <>
- =
- >=
- >
- =>
- ABS
- ACOS
- ADR
- AND
- AND_THEN
- ASIN
- ATAN
- BITADR
- BY
- CASE
- CONTINUE
- COS
- DO
- ELSE
- ELSIF
- END_CASE
- END_FOR
- END_IF

- END_REPEAT
- END_WHILE
- EXIT
- EXP
- EXPT
- FALSE
- FOR
- IF
- INI
- LIMIT
- LN
- LOG
- LOWER_BOUND
- MAX
- MIN
- MOD
- MOVE
- MUX
- NOT
- OR
- OR_ELSE
- R=
- REF=
- REPEAT
- RETURN
- ROL
- ROR
- S=
- SEL
- SHL
- SHR
- SIN
- SIZEOF
- SQRT
- SUPER
- TAN
- THEN
- THIS

- TO
- TRUE
- TRUNC
- TRUNC_INT
- UNTIL
- UPPER_BOUND
- WHILE
- XOR

Additionally, the conversion operators as listed in the **Input Assistant** are handled as keywords.

Variable Types

What Is in This Section?

This section contains the following topics:

Variable Types

Overview

This chapter provides further information on the following variable types:

- VAR local variables
- VAR_INPUT input variables
- VAR_OUTPUT output variables
- VAR_IN_OUT input and output variables
- VAR_GLOBAL global variables
- VAR_TEMP temporary variables
- VAR_STAT static variables
- VAR_EXTERNAL external variables

- VAR_INST instance variables

Local Variables - VAR

Between the keywords VAR and END_VAR, all local variables of a POU are declared. These have no external connection; in other words, they cannot be written from the outside.

Consider the possibility of adding an attribute to VAR.

Example

```
VAR
  iLoc1:INT;  (*  1.  Local  Variable*)
END_VAR
```

Input Variables - VAR_INPUT

Between the keywords VAR_INPUT and END_VAR, all variables are declared that serve as input variables for a POU. This means that at the call position, the value of the variables can be provided along with a call.

Consider the possibility of adding an attribute.

Example

```
VAR_INPUT
  iIn1:INT  (*  1.  Inputvariable*)
END_VAR
```

Output Variables - VAR_OUTPUT

Between the keywords VAR_OUTPUT and END_VAR, all variables are declared that serve as output variables of a POU. This means that these values are carried back to the POU that makes the call.

Consider the possibility of adding an attribute to VAR_OUTPUT.

Example

```
VAR_OUTPUT
  iOut1:INT;  (*  1.  Outputvariable*)
END_VAR
```

Output variables in functions and methods:

According to IEC 61131-3 draft 2, functions (and methods) can have additional outputs.You can assign them in the call of the function as shown in the following example.

Example

```
fun(iIn1 := 1, iIn2 := 2, iOut1 => iLoc1, iOut2 => iLoc2);
```

Input and Output Variables - VAR_IN_OUT

Between the keywords VAR_IN_OUT and END_VAR, all variables are declared that serve as input and output variables for a POU.

NOTE: With variables of IN_OUT type, the value of the transferred variable is changed (transferred as a pointer, Call-by-Reference). This means that the input value for such variables cannot be a constant. For this reason, even the VAR_IN_OUT variables of a function block cannot be read or written directly from outside via <FBinstance>.<InOutVariable>.

NOTE: Do not assign bit-type symbols (such as %MXaa.b or BOOL variables that are located on such a bit-type address) to BOOL-type VAR_IN_OUT parameters of function blocks. If any such assignments are detected, they are reported as a detected **Build** error in the **Messages** *view*.

Example

```
VAR_IN_OUT
  iInOut1:INT;  (*  1.  Inputoutputvariable  *)
END_VAR
```

Global Variables - VAR_GLOBAL

You can declare normal variables, constants, external, or remanent variables that are known throughout the project as global variables. To declare global variables, use the global variable lists (GVL). You can add a GVL by executing the **Add Object** command (by default in the **Project** menu).

Declare the variables locally between the keywords VAR_GLOBAL and END_VAR.

Consider the possibility of adding an attribute to VAR_GLOBAL.

A variable is recognized as a global variable by a preceding dot, for example, .iGlobVar1.

For detailed information on multiple use of variable names, the global scope operator dot (.) and name spaces refer to the chapter Global Scope Operator.

Global variables can only be declared in global variable lists (GVLs). They serve to manage global variables within a project. You can add a GVL by executing the **Add Object** command (by default in the **Project** menu).

NOTE: A variable defined locally in a POU with the same name as a global variable will have priority within the POU.

NOTE: Global variables are initialized before local variables of POUs.

Temporary Variables - VAR_TEMP

This feature is an extension to the IEC 61131-3 standard.

Temporary variables get (re)initialized at each call of the POU. VAR_TEMP declarations are only possible within programs and function blocks. These variables are also only accessible within the body of the program POU or function block.

Declare the variables locally between the keywords VAR_TEMP and END_VAR.

NOTE: You can use VAR_TEMP instead of VAR to reduce the memory space needed by a POU (for example inside a function block if the variable is only used temporarily).

Static Variables - VAR_STAT

This feature is an extension to the IEC 61131-3 standard.

Static variables can be used in function blocks, methods, and functions. Declare them locally between the keywords VAR_STAT and END_VAR. They are initialized at the first call of the respective POU.

Such as global variables, static variables do not lose their value after the POU in which they are declared is left. They are shared between the POUs they are declared in (for example, several function block instances, functions or methods share the same static variable). They can be used, for example, in a function as a counter for the number of function calls.

Consider the possibility of adding an attribute to VAR_STAT.

External Variables - VAR_EXTERNAL

These are global variables which are imported into the POU.

Declare them locally between the keywords VAR_EXTERNAL and END_VAR and in the global variable list (GVL). The declaration and the global declaration have to be identical. If the global variable does not exist, a message will display.

NOTE: It is not necessary to define variables as external. These keywords are provided in order to maintain compatibility to IEC 61131-3.

Example

```
VAR_EXTERNAL
  iVarExt1:INT;  (*  1st  external  variable  *)
END_VAR
```

Instance Variables - VAR_INST

If you declare a variable of a method as an instance variable by using the VAR_INST attribute, then this variable is not stored on the method stack but on the stack of the function block instance. It therefore behaves like other variables of the function block instance and is not reinitialized when the method is called.

VAR_INST variables are only allowed in methods. You can access such variables within the method only. Attributes such as CONST, RETAIN are not allowed in the declaration. The values of the variables can be monitored in the declaration part of the method.

Example

```
METHOD meth_last : INT
VAR_INPUT
   iVar : INT;
END_VAR
VAR_INST
   iLast : INT := 0;
END_VAR
meth_last := iLast;
iLast := iVar;
```

Attribute Keywords for Variable Types

Overview

You can add the following attribute keywords to the declaration of the variable type in order to specify the scope:

- RETAIN: refer to Retain Variables
- PERSISTENT: refer to Persistent Variables
- CONSTANT: refer to Constants - CONSTANT, Typed Literals

Remanent Variables - RETAIN, PERSISTENT

Remanent variables can retain their value throughout the usual program run period. Declare them as retain variables or even more stringent as persistent variables.

The declaration determines the degree of resistance of a remanent variable in the case of resets, downloads, or a reboot of the controller. In applications mainly the combination of both remanent flags is used (refer to Persistent Variables).

NOTE: A VAR PERSISTENT declaration is interpreted in the same way as a VAR PERSISTENT RETAIN or VAR RETAIN PERSISTENT.

NOTE: Use the *command* **Add all instance paths** to take variables declared as persistent into the **Persistent list** object.

Retain Variables

Variables declared as retain variables are stored in a nonvolatile memory area. To declare this kind of variable, use the keyword `RETAIN` in the declaration part of a POU or in a global variable list.

Example

```
VAR RETAIN
  iRem1 : INT; (* 1. Retain variable*)
END_VAR
```

Retain variables maintain their value even after an unanticipated shutdown of the controller as well as after a normal power cycle of the controller (or when executing the **Online** command **Reset Warm**). At restart of the program, the retained values will be processed further on. The other (non-retain) variables are newly initialized, either with their initialization values or with their default initialization values (in case no initialization value was declared).

For example, you may want to use a retained value when an operation, such as piece counting in a production machine, should continue after a power outage.

Retain variables, however, are reinitialized when executing the **Online** command **Reset origin** and, in contrast to persistent variables, when executing the **Online** command **Reset cold** or in the course of an application download.

NOTE: Only the specific variables defined as `VAR RETAIN` are stored in nonvolatile memory. However, local variables defined as `VAR RETAIN` in functions are NOT stored in nonvolatile memory. Defining `VAR RETAIN` locally in functions is of no effect.

Using interfaces or function blocks out of System Configuration libraries in the retain program section (`VAR_RETAIN`) will cause system exceptions, which may make the controller inoperable, requiring a re-start.

WARNING

UNINTENDED EQUIPMENT OPERATION

- Do not use interfaces out of the SystemConfigurationItf library in the retain program section (`VAR_RETAIN`).
- Do not use function blocks out of the SystemConfiguration library in the retain program section (`VAR_RETAIN`).

Failure to follow these instructions can result in death, serious injury, or equipment damage.

NOTE: The libraries SystemConfigurationItf and SystemConfiguration are only available for PacDrive controllers (PacDrive LMC Eco, PacDrive LMC Pro/Pro2).

Persistent Variables

Persistent variables are identified by keyword `PERSISTENT` (`VAR_GLOBAL PERSISTENT`). They are only reinitialized when executing the **Online** command **Reset origin**. In contrast to retain variables, they maintain their values after a download.

NOTE: Do not use the `AT` declaration in combination with `VAR PERSISTENT`.

Application example:

A counter for operating hours, which should continue counting even after a power outage or a download. Refer to the synoptic table on the behavior of remanent variables.

You can only declare persistent variables in a special global variable list of object type persistent variables, which is assigned to an application. You can add only one such list to an application.

NOTE: A declaration with `VAR_GLOBAL PERSISTENT` has the same effect as a declaration with `VAR_GLOBAL PERSISTENT RETAIN` or `VAR_GLOBAL RETAIN PERSISTENT`.

Like retain variables, the persistent variables are stored in a separate memory area.

Example

```
VAR_GLOBAL PERSISTENT RETAIN
   iVarPers1 : DINT; (* 1. Persistent+Retain Variable App1 *)
   bVarPers : BOOL;  (* 2. Persistent+Retain Variable App1 *)
END_VAR
```

NOTE: Persistent variables can only be declared inside the **Persistent list** object. If they are declared elsewhere, they will behave like retain variables and they will be reported as a detected **Build** error in the **Messages** view. (Retain variables can be declared in the global variable lists or in POUs.)

At each reload of the application, the persistent variable list on the controller will be checked against that of the project. The list on the controller is identified by the application. In case of inconsistencies, you will be prompted to reinitialize all *persistent variables* of the application. Inconsistency can result from renaming or removing or other modifications of the existing declarations in the list.

NOTE: Carefully consider any modifications in the declaration part of the persistent variable list and the effect of the results regarding reinitialization.

You can add new declarations only at the end of the list. During a download, these are detected as new and will not demand a reinitialization of the complete list. If you modify the name or data type of a variable, this is handled as a new declaration and provokes a reinitialization of the variable at the next online change or download.

Behavior of Remanent Variables

Consult the *Programming Guide* specific to your controller for further information on the behavior of remanent variables.

Constants - CONSTANT

Constants are identified by the keyword `CONSTANT`. You can declare them locally or globally.

Syntax

```
VAR CONSTANT<identifier>:<type> := <initialization>;END_VAR
```

Example

```
VAR CONSTANT
  c_iCon1:INT:=12;  (*  1.  Constant*)
END_VAR
```

Refer to the Operands chapter for a list of possible constants.

Typed Literals

Basically, in using IEC constants, the smallest possible data type will be used. If another data type has to be used, this can be achieved with the help of typed literals without the necessity of explicitly declaring the constants. For this, the constant will be provided with a prefix which determines the type.

Syntax

<type>#<literal>;

<type>	specifies the desired data type possible entries: BOOL, SINT, USINT, BYTE, INT, UINT, WORD, DINT, UDINT, DWORD, REAL, LREAL Write the type in uppercase letters.
<literal>	specifies the constant Enter data that fits within the data type specified under <type>.

Example

```
iVar1:=DINT#34;
```

If the constant cannot be converted to the target type without data loss, a message is issued.

You can use typed literals wherever normal constants can be used.

Constants in Online Mode

As long as the default setting **Replace constants** (**File > Project Settings > Compile options**) is activated, constants in online mode have a symbol preceding the value in the **Value** column in the declaration or watch view. In this case, they cannot be accessed by, for example, forcing or writing.

Variables Configuration - VAR_CONFIG

Overview

You can use the variable configuration to map function block variables on the process image that is on the device I/Os. This avoids the need of specifying the definite address already in the declaration of the function block variable. The assignment of the definite address in this case is done centrally for all function block instances in a global VAR_CONFIG list.

For this purpose, you can assign incomplete addresses to the function block variables declared between the keywords VAR and END_VAR. Use an asterisk to identify these addresses.

Identifier Syntax

<identifier> AT %<I|Q>* : <data type>

Example of the use of incompletely defined addresses:

```
FUNCTION_BLOCK locio
VAR
  xLocIn AT %I*: BOOL := TRUE;
  xLocOut AT %Q*: BOOL;
END_VAR
```

In this example, 2 local I/O variables are defined: a local input (%I*) and a local output variable (%Q*).

Define the addresses in the variable configuration in a global variable list (GVL) as follows:

Step	Action
1	Execute the **Add Object** command.
2	Add a **Global Variable List** (GVL) object to the **Devices Tree**.
3	Enter the declarations of the instance variables with the definite addresses between the keywords VAR_CONFIG and END_VAR.

When defining the addresses, note the following:

- Specify the instance variables by the complete instance path and separate the individual POUs and instance names from one another by periods.
- In the declaration, enter an address whose class (input/output) corresponds to that of the incomplete specified address (%I*, %Q*) in the function block.
- Verify that the data type agrees with the declaration in the function block.

Instance Variable Path Syntax

<instance variable path> `AT` %<I|Q><location> : <data type>;

Configuration variables whose instance path is invalid because the instance does not exist are denoted as detected errors. An error is also detected if no definite address configuration exists for an instance variable assigned to an incomplete address.

Example for a variable configuration

Assume that the following definition for function block `locio` - see the previous example - is given in a program:

```
PROGRAM PLC_PRG
VAR
locioVar1: locio;
locioVar2: locio;
END_VAR
```

Then a corrected variable configuration (in a global variable list) will be:

```
VAR_CONFIG
PLC_PRG.locioVar1.xLocIn AT %IX1.0 : BOOL;
PLC_PRG.locioVar1.xLocOut AT %QX0.0 : BOOL;
PLC_PRG.locioVar2.xLocIn AT %IX1.0 : BOOL;
PLC_PRG.locioVar2.xLocOut AT %QX0.3 : BOOL;
END_VAR
```

NOTE: Changes on directly mapped I/Os are immediately shown in the process image, whereas changes on variables mapped via `VAR_CONFIG` are not shown before the end of the responsible task.

NOTE: In a global variable list, the keywords `VAR_GLOBAL` and `VAR_CONFIG` can only be used exclusively.

Method Types

`FB_Init`, `FB_Reinit`, and `FB_Exit` Methods

General Purpose of the Methods

You can explicitly use the methods `FB_Init` and `FB_Reinit` to influence the initialization of function block variables as well as the behavior when exiting function blocks.

This chapter describes the methods, and the applications and effects of the methods in different conditions that require variable initialization.

FB_Init

By default, the `FB_Init` method is available implicitly. It is used by EcoStruxure Machine Expert to initialize a function block or a structure.

In order to influence the initialization, you can explicitly declare the `FB_Init` method by extending the given default initialization code. This allows you to evaluate the return value.

FB_Reinit

The `FB_Reinit` method must be declared explicitly.

If the `FB_Reinit` method is available, it is called after the instance of the corresponding function block has been copied (during an online change after modifications in the function block declaration). It reinitializes the new instance module. The return value is not evaluated. In order to achieve that the base function block is reinitialized, call `FB_Reinit` explicitly for that function block. This allows you to evaluate the return value.

FB_Exit

The `FB_Exit` method must be declared explicitly.

If there is an implementation, then the method is called before the controller removes the code of the function block instance (implicit call). The return value is not evaluated.

The following paragraphs provide use cases of these methods for different operating conditions.

First Download

When you download an application to a controller that is in default state, the memory locations of the variables are set to the desired initial state. The data areas of function blocks are set to the desired values. You can influence this process by explicitly implementing `FB_Init` for function blocks in the program code of the application.

The method parameters `bInCopyCode` set to FALSE and `bInitRetains` set to TRUE indicate that a first download is being executed.

Online Change

When an **Online Change** *command* is executed, the methods `FB_Exit`, `FB_Init`, and `FB_Reinit` can be used to influence the initialization of function blocks.

During online change, the modifications made to the application in offline mode are downloaded to the controller. The instances of function blocks are updated by the new instances as follows:

If you have only made changes in the implementation part of the function block and not in the declaration part, the data areas are not replaced. The methods `FB_Init`, `FB_Reinit`, and `FB_Exit` are not called.

If you have made changes in the declaration part of a function block, the copy process described in the `FB_Reinit` paragraph is performed when the **Online Change** *command* is executed. A list of the objects that have been changed since the last download is provided in the **Application Information** dialog box. Open this dialog box by clicking the **Details...** button in the dialog box where you select the option **Login with online change**.

The `FB_Init` and `FB_Reinit` method parameters `bInCopyCode` set to TRUE and `bInitRetains` set to FALSE indicate that an online change is being executed.

Calls Executed During an Online Change

Executing the **Online Change** command can change the contents of addresses.

CAUTION

INVALID POINTER

Verify the validity of the pointers when using pointers on addresses and executing the Online Change command.

Failure to follow these instructions can result in injury or equipment damage.

Step	Action	Comment
1	`FB_Exit`	`old_inst.FB_Exit(bInCopyCode := TRUE);` `FB_Exit` is called to initiate a cleanup process before the copy process is started. It prepares the data for the next copy process and influences the state of the new instance. Other parts of the application are informed about the position changes that are performed in the memory. Keep in mind that variables of type POINTER or REFERENCE keep their values during an online change and may no longer refer to the desired memory locations after the process has been completed. Variables of type INTERFACE are adapted during online change. External resources, such as socket, file, or other handles possibly can be adopted by the new instance, and often need no separate treatment during online change when the system resource is not affected by the online change copy process. If needed, this must be treated in `Init` or `Reinit` implementations.

2	`FB_Init`	`new_inst.FB_Init(bInitRetains := FALSE, bInCopyCode := TRUE);` `FB_Init` can be used to perform specific operations during the online change process. These are, for example, appropriately initializing variables at the new memory locations, or providing information on the new position of certain variables to other parts of the application.
3	Copy operation `copy`	`copy(&old_inst, &new_inst);` Existing values remain unchanged. For this purpose, they are copied from the old instance into the new instance.
4	`FB_Reinit`	`new_inst.FB_Reinit();` The `FB_Reinit` method is called after the copy operation. It sets the variables of the function block instance to defined values. For example, you can appropriately initialize variables at the new memory locations, or provide information on the new position of certain variables to other parts of the application. Implement the `FB_Reinit` method independent of online change because the method can be called by the application anytime, whenever a function block is to be reset to the original state.

NOTE: If you add the pragma `{attribute no_copy}` to a variable of a function block, this variable will not be copied during online change; it will only be initialized.

Downloading an Updated Application

When you download an application to a controller that is already running an application, the existing application will be replaced. You can use the `FB_Exit` method, for example, to assign a defined state to external resources (such as socket or file handles).

The method parameters `bInCopyCode` set to FALSE and `bInitRetains` set to FALSE indicate that a download of an updated application is being executed.

Starting an Application

Before the first cycle of the tasks of an application is executed, the initial assignments are processed.

Example:

```
T1 : TON := (PT:=t#500ms);
```

The assignments are executed after `FB_Init` has been called. To be able to verify the impacts of these assignments, attach the `{attribute call_after_init}` pragma to a function block and a method of a function block (for example, called `MyInit`). Insert this attribute above the declaration part of the function block and above the declaration part of the corresponding method. Attach this pragma also to function blocks that extend other function blocks which are using the

`{attribute call_after_init}` pragma. It is a good practice to assign the same name, the same signature, and the same attribute to the corresponding method. To achieve this, call `SUPER^.MyInit`. Select a method name of your choice (except `FB_Init`, `FB_Reinit`, and `FB_Exit`). The method is called after the initial assignments have been processed and before the tasks of an application are started.

NOTE: When the explicitly defined initialization code is executed, then the function block has already been initialized completely via the implicit initialization code. Due to this, calling `SUPER^.FB_Init` is not allowed.

Interface of the `FB_Init` Method

```
METHOD FB_Init : BOOL
VAR_INPUT
   bInitRetains : BOOL; // TRUE: the retain variables are initialized (reset warm /reset cold)
   bInCopyCode : BOOL; // TRUE the instance will be copied to the copy-
code afterward (online change)
END_VAR
```

The return value is not used.

In an `FB_Init` method, you can declare additional function block inputs. Assign the inputs in the declaration of a function block instance.

Example: Method `FB_Init` for a function block `serialdevice`:

```
METHOD PUBLIC FB_Init : BOOL
VAR_INPUT
      bInitRetains : BOOL; // Initialization of the retain variables
      bInCopyCode : BOOL; // Instance is copied to copy-code
      iCOMnum : INT; // additional input: number of the COM interface that is to be observed
END_VAR
```

Instantiation of function block `serialdevice`:

```
      com1: serialdevice (iCOMnum:=1);
      com0: serialdevice (iCOMnum:=0);
```

Interface of the `FB_Reinit` Method

```
METHOD FB_Reinit : BOOL
```

Interface of the `FB_Exit` Method

The parameter `bInCopyCode.` is mandatory.

```
METHOD FB_Exit : BOOL
VAR_INPUT
bInCopyCode : BOOL; // TRUE: the exit method is called in order to leave the instance which will be copied afterwards (online change).
END_VAR
```

Derived Function Blocks

If a function block is derived from another function block, then the `FB_Init` method of the derived function block must define the same parameters as the `FB_Init` method of the base function block. However, you can add further parameters in order to implement a special initialization for the instance.

Example for the Call Order of Derived Function Blocks for `FB_Exit` and `FB_Init`

Step	Action
1	`fbSubSubFb.FB_Exit(...);`
2	`fbSubFb.FB_Exit(...);`
3	`fbMainFb.FB_Exit(...);`
4	`fbMainFb.FB_Init(...);`
5	`fbSubFb.FB_Init(...);`
6	`fbSubSubFb.FB_Init(...);`

Pragma Instructions

What Is in This Section?

This section contains the following topics:

- Pragma Instructions
- Message Pragmas

- Conditional Pragmas
- Region Pragmas

Pragma Instructions

Overview

A pragma instruction is used to affect the properties of 1 or several variables concerning the compilation or precompilation (preprocessor) process. This means that a pragma influences the code generation.

NOTE: Consider that the available pragmas are not 1:1 implementations of C preprocessor directives. They are handled as normal statements and therefore can only be used at statement positions. They must not be used within an expression and not in the declaration part of editors.

A pragma can determine whether a variable will be initialized, monitored, added to a parameter list, added to the symbol list, or made invisible in the **Library Manager**. It can force message outputs during the build process. You can use conditional pragmas to define how the variable should be treated depending on certain conditions. You can also enter these pragmas as definitions in the compile properties of a particular object.

You can use a pragma in a separate line, or with supplementary text in an implementation or declaration editor line. Within the FBD/LD/IL editor, execute the command **Insert Label** and replace the default text `Label:` in the arising text field by the pragma. In case you want to set a label as well as a pragma, insert the pragma first and the label afterwards.

The pragma instruction is enclosed in curly brackets.

Syntax

{ <instruction text> }

The opening bracket can immediately come after a variable name. Opening and closing brackets have to be in the same line.

Correct Positions for a Conditional Pragma

```
{IF defined(abc)}
IF x =abc THEN
   {IF defined(cde)}
      y := 12;
   {ELSE}
      y :=13;
```

```
   {END_IF}
END_IF
{ELSE}
IF x = 12 THEN
   {IF defined(cde)}
      y := 12;
   {ELSE}
      y :=13;
   {END_IF}
END_IF
```

Incorrect Positions for a Conditional Pragma

NOTE: Do **not** use conditional pragmas at positions indicated in this negative example.

Further Information

Depending on the type and contents of a pragma, the pragma operates on the subsequent statement, respectively all subsequent statements, until 1 of the following conditions is met:

- It is ended by an appropriate pragma.
- The same pragma is executed with different parameters.
- The end of the code is reached.

The term code in this context refers to a declaration part, implementation part, global variable list, or type declaration.

NOTE: Pragma instructions are case-sensitive.

If the compiler cannot meaningfully interpret the instruction text, the entire pragma is handled as a comment and is skipped.

Refer to the following pragma types:

- Message_Pragmas
- Attribute_Obsolete
- Attribute_Pragmas
- Conditional_Pragmas
- Region_Pragmas
- Attribute_Symbol

Message Pragmas

Overview

You can use message pragmas to force the output of messages in the **Messages** view (by default in the **Edit** menu) during the compilation (build) of the project.

You can insert the pragma instruction in an existing line or in a separate line in the text editor of a POU. Message pragmas positioned within currently not defined sections of the implementation code will not be considered when the project is compiled. For further information, refer to the example provided with the description of the defined (identifier) in the chapter Conditional Pragmas.

Types of Message Pragmas

There are 4 types of message pragmas:

Pragma	Icon	Message Type
{text 'text string'}	–	text type The specified text string will be displayed.
{info 'text string'}		information The specified text string will be displayed.
{warning digit 'text string'}		alert type The specified text string will be displayed. In contrast to the global obsolete pragma, this alert is explicitly defined for the local position.
{error 'text string'}		error type The specified text string will be displayed.

NOTE: For messages of types information, alert, and detected error, you can reach the source position of the message - that is where the pragma is placed in a POU - by executing the command **Next Message**. This is not possible for the text type.

Example of Declaration and Implementation in ST Editor

```
VAR
ivar : INT; {info 'TODO: should get another name'}
bvar : BOOL;
arrTest : ARRAY [0..10] OF INT;
i:INT;
END_VAR
arrTest[i] := arrTest[i]+1;
ivar:=ivar+1;
{warning 'This is an alert'}
{text 'Part xy has been compiled completely'}
```

Output in **Messages** view:

Conditional Pragmas

Overview

The ExST (Extended ST) language supports several conditional Pragma Instructions, which affect the code generation in the precompile or compile process.

NOTE: Do not use any conditional pragmas in the declaration part. They are not regarded.

The implementation code which will be regarded for compilation can depend on the following conditions:
- Is a certain data type or variable declared?
- Does a type or variable have a certain attribute?
- Does a variable have a certain data type?
- Is a certain POU or task available or is it part of the call tree, etc...

NOTE: It is not possible for a POU or GVL declared in the **POUs Tree** to use a `{define...}` declared in an application. Definitions in applications will only affect interfaces inserted below the respective application.

`{define identifier string}`	During preprocessing, all subsequent instances of the identifier will be replaced with the given sequence of tokens if the token string is not empty (which is allowed and well-defined). The identifier remains defined and in scope until the end of the object or

	until it is undefined in an {undefine} directive. Used for conditional compilation.
{undefine identifier}	The preprocessor definition of the identifier (by {define}, see first row of this table) will be removed and the identifier hence is undefined. If the specified identifier is not currently defined, this pragma will be ignored.
{IF expr} ... {ELSIF expr} ... {ELSE} ... {END_IF}	These are pragmas for conditional compilation. The specified expressions exprs are required to be constant at compile time; they are evaluated in the order in which they appear until one of the expressions evaluates to a non-zero value. The text associated with the successful directive is preprocessed and compiled normally; the others are ignored. The order of the sections is determinate; however, the elsif and else sections are optional, and elsif sections may appear arbitrarily more often. Within the constant expr, you can use several conditional compilation operators.
<expr>	Within the constant expression expr of a conditional compilation pragma ({if} or {elsif}) (see previous table), you can use several operators. These operators may not be undefined or redefined via {undefine} or {define}, respectively.

You can also use these expressions as well as the definition completed by {define} in the **Compiler defines:** text field in the **Properties** dialog box of an object (**View > Properties > Build**).

Conditional Compilation Operators

The following operators are supported:

- defined (identifier)

defined (identifier)

This operator affects that the expression gets value TRUE, as soon as the `identifier` has been defined with a `{define}` instruction and has not been undefined later by an `{undefine}` instruction. Otherwise its value is FALSE.

Example on `defined (identifier)`:

Precondition: There are 2 applications `App1` and `App2`. Identifier `pdef1` is defined in `App2`, but not in `App1`.

```
{IF defined (pdef1)}
     (* this code is processed in App1 *)
     {info 'pdef1 defined'}
     hugo := hugo + SINT#1;
{ELSE}
     (* the following code is only processed in application App2 *)
     {info 'pdef1 not defined'}
     hugo := hugo - SINT#1;
{END_IF}
```

Additionally, an example for a message pragma is included:

Only information `pdef1 defined` will be displayed in the **Messages** view when the application is compiled because `pdef1` is defined. The message **pdef1 not defined** will be displayed when `pdef1` is not defined.

defined (variable:variable)

When applied to a variable, its value is TRUE if this particular variable is declared within the current scope. Otherwise it is FALSE.

Example on `defined (variable:variable)`:

Precondition: There are 2 applications `App1` and `App2`. Variable `g_bTest` is declared in `App2`, but not in `App1`.

```
{IF defined (variable:g_bTest)}
(* the following code is only processed in application App2 *)
g_bTest := x > 300;
{END_IF}
```

defined (type:identifier)

When applied to a type identifier, its value is TRUE if a type with that particular name is declared. Otherwise it is FALSE.

Example on `defined (type:identifier)` :

Precondition: There are 2 applications `App1` and `App2`. Data type DUT is defined in `App2`, but not in `App1`.

```
{IF defined (type:DUT)}
(* the following code is only processed in application App1 *)
bDutDefined := TRUE;
{END_IF}
```

defined (pou:pou-name)

When applied to a POU name, its value is TRUE if a POU or an action with that particular POU name is defined. Otherwise it is FALSE.

Example on `defined (pou: pou-name)`:

Precondition: There are 2 applications `App1` and `App2`. POU `CheckBounds` is available in `App2`, but not in `App1`.

```
{IF defined (pou:CheckBounds)}
(* the following code is only processed in application App1 *)
arrTest[CheckBounds(0,i,10)] := arrTest[CheckBounds(0,i,10)] + 1;
{ELSE}
(* the following code is only processed in application App2 *)
arrTest[i] := arrTest[i]+1;
{END_IF}
```

hasattribute (pou: pou-name, attribute)

When applied to a POU, its value is TRUE if this particular `attribute` is specified in the first line of the POUs declaration part.

Example on `hasattribute (pou: pou-name, attribute)`:

Precondition: There are 2 applications `App1` and `App2`. Function `fun1` is defined in `App1` and `App2`, but in `App1` has an attribute `vision`:

Definition of `fun1` in `App1`:

```
{attribute 'vision'}
FUNCTION fun1 : INT
VAR_INPUT
i : INT;
END_VAR
VAR
END_VAR
```

Definition of `fun1` in `App2`:

```
FUNCTION fun1 : INT
VAR_INPUT
i : INT;
END_VAR
VAR
END_VAR
```

Pragma instruction

```
{IF hasattribute (pou: fun1, 'vision')}
(* the following code is only processed in application App1 *)
ergvar := fun1 ivar);
{END_IF}
```

hasattribute (variable: variable, attribute)

When applied to a `variable`, its value is TRUE if this particular attribute is specified via the `{attribute}` instruction in a line before the declaration of the variable.

Example on `hasattribute (variable: variable, attribute)`:

Precondition: There are 2 applications `App1` and `App2`. Variable `g_globalInt` is used in `App1` and `App2`, but in `App1` has an attribute `DoCount` :

Declaration of `g_globalInt` in `App1`

```
VAR_GLOBAL
{attribute 'DoCount'}
g_globalInt : INT;
```

```
g_multiType  :  STRING;
END_VAR
```

Declaration of `g_globalInt` in `App2`

```
VAR_GLOBAL
g_globalInt  :  INT;
g_multiType  :  STRING;
END_VAR
```

Pragma instruction

```
{IF hasattribute (variable: g_globalInt, 'DoCount')}
(* the following code line will only be processed in App1, because there variable g_globalInt has go
g_globalInt := g_globalInt + 1;
{END_IF}
```

hastype (variable:variable, type-spec)

When applied to a `variable`, its value is TRUE if this particular variable has the specified `type-spec`. Otherwise it is FALSE.

Available data types of `type-spec`

- LREAL
- REAL
- LINT
- DINT
- INT
- SINT
- ULINT
- UDINT
- UINT
- USINT
- TIME
- LWORD
- DWORD
- WORD
- BYTE
- BOOL
- STRING
- WSTRING

- DATE_AND_TIME
- DATE
- TIME_OF_DAY

Example on operator `hastype (variable: variable, type-spec)`:

Precondition: There are 2 applications `App1` and `App2`. Variable `g_multitype` is declared in `App1` with type LREAL and in application `App2` with type STRING:

```
{IF (hastype (variable: g_multitype, LREAL))}
(* the following code line will be processed only in App1 *)
g_multitype := (0.9 + g_multitype) * 1.1;
{ELSIF (hastype (variable: g_multitype, STRING))}
(* the following code line will be processed only in App2 *)
g_multitype := 'this is a multitalent';
{END_IF}
```

hasvalue (define-ident, char-string)

If the define (`define-ident`) is defined and it has the specified value (`char-string`), then its value is TRUE. Otherwise it is FALSE.

Example on `hasvalue (define-ident, char-string)`:

Precondition: Variable `test` is used in applications `App1` and `App2`. It gets value 1 in `App1` and value 2 in `App2`:

```
{IF hasvalue(test,'1')}
(* the following code line will be processed in App1, because there variable test has value 1 *)
x := x + 1;
{ELSIF hasvalue(test,'2')}
(* the following code line will be processed in App1, because there variable test has value 2 *)
x := x + 2;
{END_IF}
```

NOT operator

The expression gets value TRUE when the inverted value of `operator` is TRUE. `operator` can be one of the operators described in this chapter.

Example on `NOT operator`:

Precondition: There are 2 applications `App1` and `App2`. POU `PLC_PRG1` is used in `App1` and `App2`. POU `CheckBounds` is only available in `App1`:

```
{IF defined (pou: PLC_PRG1) AND NOT (defined (pou: CheckBounds))}
(* the following code line is only executed in App2 *)
bANDNotTest := TRUE;
{END_IF}
```

AND operator

The expression gets value TRUE if both operators are TRUE. `operator` can be one of the operators listed in this table.

Example on `AND operator`:

Precondition: There are 2 applications `App1` and `App2`. POU `PLC_PRG1` is used in applications `App1` and `App2`. POU `CheckBounds` is only available in `App1`:

```
{IF defined (pou: PLC_PRG1) AND (defined (pou: CheckBounds))}
(* the following code line will be processed only in applications App1, because only there "PLC_PRG1'
bORTest := TRUE;
{END_IF}
```

OR operator

The expression is TRUE if one of the operators is TRUE. `operator` can be one of the operators described in this chapter.

Example on `OR operator`:

Precondition: POU `PLC_PRG1` is used in applications `App1` and `App2`. POU `CheckBounds` is only available in `App1`:

```
{IF defined (pou: PLC_PRG1) OR (defined (pou: CheckBounds))}
(* the following code line will be processed in applications App1 and App2, because both contain at
bORTest := TRUE;
{END_IF}
```

(operator)

`(operator)` braces the operator.

Region Pragmas

Overview

Use region pragmas to group several lines into one block in a text editor. You can assign a name to the block. Region pragmas can be nested.

The figure shows a program code that contains a region pragma in the extended and in the collapsed view.

Region pragmas can be used in the ST editor and in the declaration editors. You can adapt syntax highlighting to your individual requirements in the **Tool > Options > Syntax Highlighting** *dialog box*.

Attribute Pragmas

What Is in This Section?

This section contains the following topics:

- Attribute_initialize_on_call
- Attribute_init_namespace
- Attribute_init_On_Onlchange
- Attribute_instance-path
- Attribute_linkalways
- Attribute_monitoring
- Attribute_namespace
- Attribute_no_assign
- Attribute_no_check
- Attribute_no_copy
- Attribute_no-exit
- Attribute_no_init
- Attribute_no_instance_in_retain
- Attribute_no_virtual_actions
- Attribute_pingroup
- Attribute_pin_presentation_order_inputs/outputs
- Attribute_obsolete
- Attribute_pack_mode
- Attribute_qualified_only
- Attribute_reflection
- Attribute_subsequent
- Attribute_symbol
- Attribute_warning_disable
- Attribute_enable_dynamic_creation

Attribute Pragmas

Overview

You can assign attribute pragmas to a signature in order to influence the compilation or pre-compilation that is the code generation.

There are user-defined attributes, which you can use in combination with conditional pragmas.

Attributes are defined within the declaration part. An exception is made for the action and transition objects which do not have a declaration part. You can define the attributes at the beginning of the implementation part.

There are also the following predefined standard attribute pragmas:

- attribute displaymode
- attribute ExpandFully
- attribute global_init_slot
- attribute hide
- attribute hide_all_locals
- attribute initialize_on_call
- attribute init_namespace
- attribute init_On_Onlchange
- attribute instance-path
- attribute linkalways
- attribute monitoring
- attribute no_check
- attribute no_copy
- attribute no-exit
- attribute noinit
- attribute no_virtual_actions
- attribute obsolete
- attribute pack_mode
- attribute qualified_only
- attribute reflection
- attribute subsequent
- attribute symbol
- attribute warning_disable

User-Defined Attributes

Overview

You can assign arbitrary user-defined or application-defined attribute pragmas to POUs, type declarations, or variables. This attribute can be queried before compilation by conditional pragmas.

Syntax

{attribute 'attribute'}

This pragma instruction is valid for the subsequent POU declaration or variable declaration.

You can assign a user-defined attribute to:
- a POU or action
- a variable
- a data type

Example on POUs and Actions

Attribute `vision` for function `fun1`:

```
{attribute 'vision'}
FUNCTION fun1 : INT
VAR_INPUT
i : INT;
END_VAR
VAR
END_VAR
```

Example on Variables

Attribute `DoCount` for variable `ivar` :

```
PROGRAM PLC_PRG
VAR
{attribute 'DoCount'};
ivar:INT;
bvar:BOOL;
END_VAR
```

Example on Types

Attribute `aType` for data type DUT_1:

```
{attribute 'aType'}
TYPE DUT_1 :
STRUCT
a:INT;
b:BOOL;
END_STRUCT
END_TYPE
```

For the usage of conditional pragmas, refer to the chapter Conditional Pragmas.

Attribute call_after_global_init_slot

Overview

All functions and programs containing this attribute in an own line above their declaration part are called after the global initialization (`GlobalInit`). The calling sequence is determined by the attribute value.

NOTE: Compile errors will be detected (during code generation) if `VAR_INPUT` declarations are used in functions or methods that contain this attribute. The reason is that the input variables are unknown when the function is called implicitly during online change.

Syntax

{attribute 'call_after_global_init_slot' := '<slot>'}

Replace `<slot>` by an integer value defining the priority within the calling sequence: The lower the value, the earlier the call. In case of several signatures carrying the same value for the attribute, the sequence of their initialization remains undefined.

If a method is provided with the attribute, then it is called for all instances of the concerned function block. All instances are called within the specified slot; however, you cannot control the order among the instances themselves.

Attribute call_after_init

Overview

Use the pragma `{attribute call_after_init}` to define a method that is called implicitly after the initialization of a function block instance. For performance reasons, attach the attribute both to the function block itself and to the instance method to be called. The method has to be called after FB_Init and after having applied the variable values of an initialization expression in the instance declaration.

NOTE: Compile errors will be detected if `VAR_INPUT` declarations are used in methods that contain this attribute. The reason is that the input variables are unknown when the method is called implicitly during online change.

Syntax

{attribute 'call_after_init'}

Example

With the following definition:

```
{attribute 'call_after_init'}
FUNCTION_BLOCK FB
... <functionblock definition>
{attribute 'call_after_init'}
METHOD FB_AfterInit
... <method definition>
```

... declaration like the following:

```
inst : FB := (in1 := 99);
```

... will result in the following order of code processing:

```
inst.FB_Init();
inst.in1 := 99;
inst.FB_AfterInit();
```

So, in `FB_Afterinit`, you can react on the user-defined initialization.

Attribute call_after_online_change_slot

Overview

All functions and programs containing this attribute in an own line above their declaration part are called after an online change. The calling sequence is determined by the attribute value.

NOTE: Compile errors will be detected (during code generation) if `VAR_INPUT` declarations are used in functions or methods that contain this attribute. The reason is that the input variables are unknown when the function is called implicitly during online change.

Syntax

{attribute 'call_after_online_change_slot' := '<slot>'}

Replace `<slot>` by an integer value defining the priority within the calling sequence: The lower the value, the earlier the call. If several modules have the same priority value for the attribute, then the order in which they are called remains undefined.

If a method is provided with the attribute, then it is called for all instances of the concerned function block. All instances are called within the specified slot; however, you cannot control the order among the instances themselves.

NOTE: As the application cannot run during an online change, each code executed in this situation can effect jitter. For this reason, minimize the code to be executed.

Attribute call_before_global_exit_slot

Overview

All functions and programs containing this attribute in an own line above their declaration part are called after the `GlobalExit`. The `GlobalExit` is executed before a new download, at a reset, or during an online change and affects modules which are provided with an `FB_exit` method. The calling sequence is determined by the attribute value.

NOTE: Compile errors will be detected (during code generation) if `VAR_INPUT` declarations are used in functions or methods that contain this attribute. The reason is that the input variables are unknown when the function is called implicitly during online change.

Syntax

{attribute 'call_before_global_exit_slot' := '<slot>'}

Replace `<slot>` by an integer value defining the priority within the calling sequence: The lower the value, the earlier the call. If several modules have the same priority value for the attribute, then the order in which they are called remains undefined.

If a method is provided with the attribute, then it is called for all instances of the concerned function block. All instances are called within the specified slot; however, you cannot control the order among the instances themselves.

Attribute call_on_type_change

Overview

Attach the `Attribute call_on_type_change` pragma to methods of a function block `A` in order to achieve that this method is called when the data type is changed for one or more function blocks `B`, `C`, etc. that are referenced by `A`. The function blocks can be referenced by pointers or references.

Syntax

{attribute 'call_on_type_change':= '<name of the first referenced function block>|<name of the second referenced function block>|<name of the nth referenced function block>'}

Insert the `Attribute call_on_type_change` above the first line in the method declaration.

Examples

Example of a function block with references:

```
FUNCTION_BLOCK FB_A
...
VAR
     var_pt: POINTER TO FB_B;
     var_ref: REFERENCE TO FB_C;
END_VAR
...
```

Example of a method that is called when data types are changed in the referenced function blocks `FB_B` and `FB_C`:

```
{attribute 'call_on_type_change' := 'FB_B,
FB_C'}
METHOD METH_react_on_type_change : INT
VAR_INPUT
...
```

Attribute const_replaced, Attribute const_non_replaced

Overview

Insert the pragma `{attribute 'const_replaced'}` in the declaration of a global constant if you explicitly want to activate the compiler option **Replace constants** for this constant. This has the effect that the constant becomes available in the **Symbol Configuration**.

Correspondingly, you can insert the pragma `{attribute 'const_non_replaced'}` in order to deactivate the compiler option **Replace constants**.

The option **Replace constants** is pre-defined for the whole project in the **Project Settings > Compile options** *dialog box*.

Syntax

```
{attribute 'const_replaced'}
```

```
{attribute 'const_non_replaced'}
```

Example

The constants `iTestCon` and `bTestCon` are available in the **Symbol Configuration** because **Replace constants** is deactivated by pragmas.

```
VAR_GLOBAL CONSTANT
     {attribute 'const_non_replaced'}
     iTestCon      :      INT   := 12;
     {attribute 'const_non_replaced'}
     bTestCon      :      BOOL  := TRUE;
     rTestCon      :      REAL  := 1.5;
END_VAR
```

```
VAR_GLOBAL
     iTestVar      :      INT   := 12;
     bTestVar      :      BOOL  := TRUE;
END_VAR
```

Attribute 'dataflow'

Overview

The pragma `{attribute 'dataflow'}` enables you to control the dataflow when processing function blocks in the FBD/LD/IL editor. The attribute defines the input or output of a function block that is used as the connection to the next or previous function block.

You can assign this attribute to only one input and one output in the declaration of the function block.

For function blocks without the `{attribute 'dataflow'}`, the dataflow is determined automatically as follows:

The connection is established between an output and an input of the same type. The uppermost input and output variables of the function block are used first. If there are no variables with the same data type, then the uppermost output is connected to the uppermost input of the next function block.

You can also change the control flow in the editor by using the pointer to connect the connector pins of the function block to other positions. For further information, refer to the description of Inserting, Arranging, and Replacing Elements.

Syntax

```
{attribute 'dataflow'}
```

Example

The `FB` and the previous function block are connected using the input variable i1. The `FB` and the next function block are connected using the output variable `outRes1`.

```
FUNCTION_BLOCK FB
VAR_INPUT
                r1 : REAL;
                {attribute 'dataflow'}
                i1 : INT;
                i2 : INT;
                r2 : REAL;
END_VAR
```

```
VAR_OUTPUT
                {attribute 'dataflow'}
                outRes1 : REAL;
                out1 : INT;
                g1 : INT;
                g2 : REAL;
END_VAR
```

Attribute displaymode

Overview

Use the pragma `{attribute displaymode}` to define the display mode of a single variable. This setting will overwrite the global setting for the display mode of all monitoring variables done via the commands of the submenu **Display Mode** (by default in the

Online menu).

Position the pragma in the line above the line containing the variable declaration.

Syntax

{attribute 'displaymode':=<displaymode>}

The following definitions are possible:

- to display in binary format

```
{attribute 'displaymode':='bin'}
{attribute 'displaymode':='binary'}
```

- to display in decimal format

```
{attribute 'displaymode':='dec'}
{attribute 'displaymode':='decimal'}
```

- to display in hexadecimal format

```
{attribute 'displaymode':='hex'}
{attribute 'displaymode':='hexadecimal'}
```

Example

```
VAR
    {attribute 'displaymode':='hex'}
    dwVar1: DWORD;
END_VAR
```

Attribute estimated-stack-usage

Overview

The pragma `{attribute 'estimated-stack-usage' := '<allowed stack size in bytes>'}` helps to prevent runtime systems with an active stack check from issuing a message indicating that there is insufficient space in the stack. This check is performed during the code generation. For recursive methods, you may want to reduce the number of messages.

Syntax

```
{attribute 'estimated-stack-usage' := '<allowed stack size in bytes>'}
```

Insert Location

Insert this pragma in the line above the `METHOD` declaration in the declaration section of the relevant method.

Example

```
{attribute 'estimated-stack-usage' := '99'}
METHOD xxMETH : INT
VAR_INPUT
END_VAR
     VAR next: AFB;
     big: ARRAY[0..100] OF DINT;
END_VAR
next.xxMETH();
```

Attribute ExpandFully

Overview

Use the pragma `{attribute 'ExpandFully'}` to make all members of arrays used as input variables for referenced visualizations accessible within the **Visualization Properties** dialog box.

Syntax

{attribute 'ExpandFully'}

Example

Visualization `visu` is intended to be inserted in a frame within visualization `visu_main`.

As input variable `arr` is defined in the interface editor of `visu` and will later be available for assignments in the **Properties** dialog box of the frame in `visu_main`.

In order to get the available particular components of the array in the **Properties** dialog box, insert the attribute `ExpandFully` in the interface editor of `visu` directly before `arr`.

Declaration in the interface editor of `visu`:

```
VAR_INPUT
{attribute 'ExpandFully'}
arr : ARRAY[0..5] OF INT;
END_VAR
```

Resulting **Properties** dialog box of frame in `visu_main`:

Attribute global_init_slot

Overview

The pragma `{attribute 'global_init_slot'}` defines the sequence of initialization of POUs or global variable lists.

Variables in a list (GVL or POU) are initialized from top to bottom.

If there are several global variable lists available, then the sequence of initialization is not defined.

The sequence of initialization is irrelevant for literal values, such as `1`, `'hello'`, `3.6`, or for constants of base data types. However, if there are dependencies between the different lists, you must define the sequence of initialization by yourself. To achieve this, you can assign a defined initialization slot to a GVL or a POU using the `attribute global_init_slot`.

Syntax

{attribute 'global_init_slot' := '<slot>'}

Replace <slot> by an integer value that defines the position in the initialization order. The default value for a POU (program, function block) is 50,000. The default value for a GVL is 49,990. A lower value provokes an earlier initialization. In case of several POUs or GVLs carrying the same value for the attribute `global_init_slot`, the sequence of their initialization remains undefined. This will be indicated as a detected programming error in the **Build** category of the **Messages** *view*.

The pragma `{attribute 'global_init_slot'}` is valid for the entire GVL or POU and must therefore be located above the `VAR_GLOBAL` or POU declaration.

Example

The example includes 2 global variable lists `GVL_1` and `GVL_2` and a program `PLC_PRG` that uses variables from both lists.

`GVL_1` uses the variable `B` for initializing a variable `A` which is initialized in the `GVL_2` with a value of `1000`.

GVL_1:

```
VAR_GLOBAL //49990
   A : INT := GVL_2.B*100;
END_VAR
```

GVL_2:

```
VAR_GLOBAL //49990
      B : INT := 1000;
   C : INT := 10;
END_VAR
```

PLC_PRG:

```
PROGRAM PLC_PRG //50000
VAR
      ivar: INT := GVL_1.A;
      ivar2: INT;
END_VAR
ivar:=ivar+1;
ivar2:=GVL_2.C;
```

When building this example, a programming error is issued in the **Build** category of the **Messages** *view* because `GVL_2.B` is used for initializing `GVL_1.A` before `GVL_2` has been initialized. To avoid this, use the `attribute global_init_slot` in order to position `GVL_2` before `GVL_1` in the sequence of initialization.

`GVL_2` must have a slot value of `49989` or lower to achieve the earliest initialization within the program.

GVL_2:

```
{attribute 'global_init_slot' := '100'}
VAR_GLOBAL
   B : INT := 1000;
   C : INT := 10;
END_VAR
```

You can even use `GVL_2.C` in the implementation part of `PLC_PRG` without a pragma because both GVLs are initialized before the program in either case.

Attribute hide

Overview

The pragma `{attribute hide}` helps you to prevent variables or even whole signatures from being displayed within the functionality of listing components or the input assistant or the declaration part in online mode. Only the variable subsequent to the pragma will be hidden.

Syntax

{attribute 'hide'}

To hide all local variables of a signature, use the attribute hide_all_locals.

Example

The function block `myPOU` is implemented using the attribute:

```
FUNCTION_BLOCK myPOU
VAR_INPUT
a:INT;
{attribute 'hide'}
a_invisible: BOOL;
a_visible: BOOL;
END_VAR
VAR_OUTPUT
b:INT;
END_VAR
```

In the main program 2 instances of function block `myPOU` are defined:

```
PROGRAM PLC_PRG
VAR
POU1, POU2: myPOU;
END_VAR
```

When assigning an input value to `POU1`, the functionality of listing components that works on typing `POU1` in the implementation part of `PLC_PRG` will display the input variables `a` and `a_visible` (and the output variable `b`). The hidden input variable `a_invisible` will not be displayed.

Attribute hide_all_locals

Overview

The pragma `{attribute 'hide_all_locals'}` helps you to prevent all local variables of a signature from being displayed within the functionality of listing components or the input assistant. This attribute is identical to assigning the attribute hide to each particular of the local variables.

Syntax

{attribute 'hide_all_locals'}

Example

The function block `myPOU` is implemented using the attribute:

```
{attribute 'hide_all_locals'}
FUNCTION_BLOCK myPOU
VAR_INPUT
a:INT;
END_VAR
VAR_OUTPUT
b:BOOL;
END_VAR
VAR
c,d:INT;
END_VAR
```

In the main program 2 instances of function block `myPOU` are defined:

```
PROGRAM PLC_PRG
VAR
POU1, POU2: myPOU;
END_VAR
```

When assigning an input value to `POU1`, the functionality of listing components that works on typing `POU1` in the implementation part of `PLC_PRG` will display the variables `a` and `b`. The hidden local variables `c` or `d` will not be displayed.

Attribute initialize_on_call

Overview

You can add the pragma `{attribute initialize_on_call}` to input variables. An input of a function block with this attribute will be initialized at any call of the function block. If an input expects a pointer and if this pointer has been removed due to an online change, then the input will be set to NULL.

Syntax

{attribute 'initialize_on_call'}

Attribute init_namespace

Overview

A variable of type STRING or WSTRING, which is declared with the pragma `{attribute init_namespace}` in a library, will be initialized with the current namespace of that library. For further information, refer to the description of the *library management*.

Syntax

{attribute 'init_namespace'}

Example

The function block `POU` is provided with all necessary attributes:

```
FUNCTION_BLOCK POU
VAR_OUTPUT
{attribute 'init_namespace'}
myStr: STRING;
END_VAR
```

Within the main program `PLC_PRG` an instance `fb` of the function block `POU` is defined:

```
PROGRAM PLC_PRG
VAR
fb:POU;
newString: STRING;
END_VAR
newString:=fb.myStr;
```

The variable `myStr` will be initialized with the current namespace, for example `MyLib.XY`. This value will be assigned to `newString` within the main program.

Attribute init_On_Onlchange

Overview

Step	Action
1	Attach the pragma `{attribute 'init_on_onlchange'}` to a variable to initialize this variable with each online change.
2	Open the **Build** tab of the **Properties** *dialog box of the application* and enter the string `no_fast_online_change` in the **Compiler defines** box.

Syntax

{attribute 'init_on_onlchange' }

Insert Location

Insert this pragma in the line above the declaration of the variables.

Attribute instance-path

Overview

You can add the pragma `{attribute instance-path}` to a local string variable. This local string variable will be initialized with the **Applications tree** path of the POU to which this string variable belongs. Applying this pragma presumes the use of the attribute reflection for the corresponding POU and the additional attribute noinit for the string variable.

Syntax

{attribute 'instance-path'}

Example

Assume the following function block `POU` being equipped with the attribute `'reflection'`:

```
{attribute 'reflection'}
FUNCTION_BLOCK POU
VAR
{attribute 'instance-path'}
{attribute 'noinit'}
str: STRING;
END_VAR
```

In the main program `PLC_PRG` an instance `myPOU` of function block `POU` is called:

```
PROGRAM PLC_PRG
VAR
myPOU:POU;
myString: STRING;
END_VAR
myPOU();
myString:=myPOU.str;
```

After initialization of instance `myPOU`, the string variable `str` gets assigned the path of instance `myPOU`, for example: `PLC.Application.PLC_PRG.myPOU`. This path will be assigned to variable `myString` within the main program.

NOTE: The length of a string variable may be arbitrarily defined (even >255). However, the string will be cut (from its back end) if it gets assigned to a string variable of a shorter length.

Attribute linkalways

Overview

Use the pragma `{attribute 'linkalways'}` to mark POUs or global variable lists for the compiler in a way so that they are always included into the compile information. As a result, objects with this option will always be compiled and downloaded to the controller. The compiler option **Link always** affects the same.

Syntax

{attribute 'linkalways'}

When you use the symbol configuration editor, the marked POUs are used as a basis for the selectable variables for the symbol configuration.

Example

The global variable list `GVLMoreSymbols` is implemented making use of the attribute `'linkalways'`:

```
{attribute 'linkalways'}
VAR_GLOBAS
g_iVar1: INT;
g_iVar2: INT;
END_VAR
```

With this code, the symbols of `GVLMoreSymbols` become selectable in the **Symbol configuration**.

Attribute monitoring

Overview

This attribute pragma allows you to get properties and function call results monitored in the online view of the IEC editor or in a watch list.

Monitoring of Properties

Add the pragma in the line above the property definition. Then the name, type, and value of the variables of the property will be displayed in the online view of the POU using the property or in a watch list. Therein, you can also enter prepared values to force variables belonging to the property.

Example of property prepared for variable monitoring

Example of monitoring view

Monitoring the Current Value of the Property Variables

There are two different ways to monitor the current value of the property variables. For the particular use case, consider carefully which attribute is suitable to actually get the desired value. This will depend on whether operations on the variables are implemented within the property:

1. **Pragma** {attribute 'monitoring':='variable'}

An implicit variable is created for the property, which will get the current property value whenever the application calls the set or get method. The latest value stored in this implicit variable will be monitored.

Syntax

{attribute 'monitoring':='variable'}

2. **Pragma** {attribute 'monitoring':='call'}

You can only use this attribute for properties returning simple data types or pointers, not for structured types.

The value to be monitored is read or written by a direct call of property: the monitoring service of the runtime system executes the `Get` or `Set` method of the property function including the implementation part of the property.

NOTE: When choosing this monitoring type instead of using an intermediate variable (see *1. Pragma*), consider possible side effects due to any operations implemented within the property.

NOTE: The `monitoring` pragma is also evaluated by the symbol configuration. If the value `variable` was specified, only a read-access on the property is available in the symbol configuration.

Syntax

{attribute 'monitoring':='call'}

Monitoring of Function Call Results

You can use function call monitoring for any constant value that can be interpreted as 4 byte numerical value (for example, INT, SHORT, LONG). For the other input parameters (for example, BOOL), use a variable instead of a constant parameter. Add the pragma `{attribute 'monitoring':='call'}` in the line above the function declaration. You can then monitor this variable in the text editor view in online view of the POU in which a variable gets assigned the result of a function call. You can also add the variable to a watch list for the same purpose. To get the variable immediately provided within a watch view, execute the command **Add watchlist**.

Example 1: Functions `FUN2` and `FUN_BOOL2` with attribute `'monitoring'`

Example 2: Call of functions `FUN2` and `FUN_BOOL2` in a program POU

Example 3: Function calls in online mode:

Monitoring of Variables with an Implicit Call of an External Function

For monitoring variables with an implicit call of an external function, the following conditions have to be fulfilled:

- The function is marked with `{attribute 'monitoring' := 'call'}`.
- The function is marked as **Link Always**.
- The variable is marked with `{attribute 'monitoring_instead' := 'MyExternalFunction(a,b,c)'}`.
- The values `a,b,c` are integer values and match the input parameters of the function to call.

NOTE: Forcing or writing of functions is not supported. You can implicitly implement forcing by adding an additional input parameter for the particular function that serves as an internal force flag.

NOTE: Function monitoring is not possible on the compact runtime system.

Attribute namespace

Overview

In combination with the attribute symbol, the pragma `{attribute namespace}` allows you to redefine the namespace of project variables. You can apply it on complete POUs, like GVLs or programs, but not on particular variables. The concerned variables will be exported with the new namespace definition to a symbol file and after a download of this file be available on the controller.

This also allows you to access variables from POUs or visualizations which originally have got different namespaces. For example, it allows you to run a previous EcoStruxure Machine Expert visualization also in a later EcoStruxure Machine Expert environment.

For further information, refer to the description of the symbol configuration. A new symbol file will be created at a download or online change of the project. It is downloaded to the controller together with the application.

Syntax

{attribute 'namespace' := '<namespace>'}

Example of a Namespace Replacement for the Variables of a Program

```
{attribute 'namespace':='prog'}
PROGRAM PLC_PRG
VAR
   {attribute 'symbol' := 'readwrite'}
   iVar:INT;
   bVar:BOOL;
END_VAR
```

If `iVar`, for example, was accessed by `App1.PLC_PRG.ivar` before, now it is accessible via `prog.ivar`.

Further Replacement Examples

Original Namespace	Variable	Namespace Replacement	Access on the Variable Within the Current Project
App1.Lib2.GVL2	Var07	{attribute 'namespace':=''}	.Var07
App1.GVL2	Var02	{attribute 'namespace':='Ext''}	Ext.Var02
App1.GVL2.FB1	Var02	{attribute 'namespace':='App1.GVL2''}	App1.GVL2.Var02

The replacements shown in the table result in the following entries in the symbol file:

```
<NodeList>
 <Node name="">
      <Node name="Var07" type="T_INT" access="ReadWrite">
 </Node>
</NodeList>
<NodeList>
 <Node name="Ext">
  <Node name="Var02 " type="T_INT" access="ReadWrite"></Node>
 </Node>
</NodeList>
<NodeList>
 <Node name="App1">
  <Node name="GVL2">
   <Node name="Var02 " type="T_INT" access="ReadWrite"></Node>
     </Node>
 </Node>
</NodeList>
```

Attribute no_assign

Overview

Insert the pragma `{attribute 'no assign'}` as first line of the declaration part of a function block. This has the effect that compile errors are generated if an instance of the function block is assigned to another instance of the same function block. For example, you might want to avoid such assignments if the function block contains pointers. This could cause issues because they are copied when values are assigned.

NOTE: Use the `{attribute 'no assign'}` in function blocks with internal pointers to help to avoid assigning an instance of the function block to another instance of the same function block.

Assignment of Function Block Instances Containing Pointers

In this example, the value assignment of function block instances causes issues when `fb_exit` is executed:

```
VAR_GLOBAL
inst1 : TestFB;
     awsBufferLogFile : ARRAY [0..9] OF WSTRING(66);(* Area: 0, Offset: 0x1304 (4868)*)
     LogFile : SEDL.LogRecord := (sFileName := 'LogFile.log', pBuffer := ADR(awsBufferLogFile), udiMax
END_VAR
```

```
PROGRAM PLC_PRG
VAR
     inst2 : TestFB := inst1;
     LogFileNew : LogRecord := LogFile;
END_VAR
```

In this case, `LogRecord` manages a list of pointers. Different actions are executed for them if `fb_exit` applies. When assigning function block instances, `fb_exit` will be executed twice which causes an issue. Prevent this by adding the `no_assign` attribute to the declaration of function block `TestFB`:

```
{attribute 'no_assign'}
FUNCTION_BLOCK TestFB
VAR_INPUT
...
```

The following compile errors are reported:

```
C0328: Assignment not allowed for type TestFB
```

```
C0328: Assignment not allowed for type LogRecord
```

Attribute no_check

Overview

You can add the pragma `{attribute 'no_check'}` to the declaration of a POU in order to suppress the call of any POUs for implicit checks. As checking functions may influence the performance, apply this attribute to POUs that are frequently called or already approved.

NOTE: This attribute has an automatic effect also on the child objects of a POU.

Example: In programs with this attribute, no check functions are executed. Not even for actions that are assigned to this program.

Syntax

{attribute 'no_check'}

Attribute no_copy

Overview

Generally, an online change will require a reallocation of instances, for example of POUs. The value of the variables within this instance will get copied.

If, however, the pragma `{attribute no_copy}` is added to a variable, an online change copy of this variable will not be performed; this variable will be initialized instead. This can be reasonable in case of local pointer variable, pointing on a variable actually shifted by the online change (and thus having a modified address).

Syntax

{attribute 'no_copy'}

Attribute no-exit

Overview

If a function block provides an `FB_exit` method , you can suppress its call for a special instance with the help of assigning the pragma `{attribute no-exit}` to the function block instance.

Syntax

{attribute 'no-exit'}

Example

Assume the method `FB_Exit` being added to a function block named POU:

In the main program `PLC_PRG`, 2 variables of type POU are instantiated:

```
PROGRAM PLC_PRG
VAR
POU1 : POU;
{attribute 'no-exit'}
POU2 : POU;
END_VAR
```

When variable `bInCopyCode` becomes TRUE within `POU1`, the method `FB_Exit` is called exiting an instance that will get copied afterwards (online change). The method `FB_Exit` is not called in context of the function block instance `POU2`.

Attribute no_init

Overview

Variables provided with the pragma `{attribute no_init}` will not be initialized implicitly. The pragma belongs to the variable declared subsequently.

Syntax

```
{attribute 'no_init'}
```

also possible

```
{attribute 'no-init'}
{attribute 'noinit'}
```

Example

```
PROGRAM PLC_PRG
VAR
```

```
A  :  INT;
{attribute  'no_init'}
B  :  INT;
END_VAR
```

If a reset is performed on the associated application, the integer variable `A` will be again initialized implicitly with 0, whereas variable `B` maintains the value it is currently assigned to.

Attribute no_instance_in_retain

Overview

The pragma `{attribute 'no_instance_in_retain'}` helps to avoid that the instance of a certain function block instance is stored in the retain memory area.

Insert it as first line of the declaration of the function block. This has the effect that a message is created in case any instance of the POU is declared as a `RETAIN` variable.

Syntax

```
{attribute  'no_instance_in_retain'}
```

Attribute no_virtual_actions

Overview

This attribute is valid for function blocks, which are derived from a base function block implemented in SFC, and which are using the main SFC workflow of the base class. The actions called therein show the same virtual behavior as methods. This means that the base class actions may be overridden by specific implementations related to the derived classes.

In order to help to prevent the action of the base class from being overridden, you can assign the pragma `{attribute 'no_virtual_actions'}` to the base class.

Syntax

{attribute 'no_virtual_actions'}

Example

In the following example, the function block `POU_SFC` provides the base class to be extended by the function block `POU_child`.

By use of the keyword `SUPER`, the derived class `POU_child` calls the workflow of the base class that is implemented in SFC.

The exemplary implementation of this workflow is restricted to the initial step. This is followed by 1 single step with associated step action `ActiveAction` concerned with the assignment of the output variables:

```
an_int:=an_int+1;              // counting the action calls
test_act:='father_action'; // writing string variable test_act
METH();                           // Calling method METH for writing string variable test_meth
```

In case of the derived class `POU_child`, the step action will be overwritten by a specific implementation of `ActiveAction`. It differs from the original one by assigning the string `'child_action'` instead of `'father_action'` to variable `test_act`.

Likewise, the method `METH`, assigning the string `'father_method'` to variable `test_meth` within the base class, will be overwritten such that `test_meth` will be assigned to `'child_method'` instead.

The main program `PLC_PRG` will execute repeated calls to `Child` (an instance of `POU_child`). As expected, the actual value of the output string report the call to action and method of the derived class:

You can observe a different behavior if the base class is preceded by the attribute `'no_virtual_actions'`

```
{attribute 'no_virtual_actions'}
FUNCTION_BLOCK POU_SFC...
```

Whereas method `METH` will still be overwritten by its implementation within the derived class, a call of the step action will now result in a call of action `ActiveAction` of the base class. Therefore, `test_act` will be assigned to string `'father_action'`.

Attribute pingroup

Overview

Insert the pragma `{attribute 'pingroup' := '<groupname>'}` in the declaration of a function block, for grouping the input pins or output pins (parameters). Then, in the respective box of the FBD and LD editor, each pin group can be displayed, folded,

or unfolded. Multiple groups are possible and are differentiated by their names. The particular state (folded/unfolded) per box is saved in the project options.

Inputs and outputs without attribute `pingroup` are always displayed above any possible group or groups.

Syntax

```
{attribute 'pingroup' := '<groupname>'}
```

Example

Two groups are defined:
- general `(i1, out1)`
- group1 `(i2, g1)`

`r1, r2, outRes1` and `g2` are always displayed.

```
FUNCTION_BLOCK FB
VAR_INPUT
    r1 : REAL;
    {attribute 'pingroup' := 'general'}
    i1 : INT;
    {attribute 'pingroup' := 'group1'}
    i2 : INT;
    r2 : REAL;
END_VAR
```

```
VAR_OUTPUT
    outRes1 : REAL;
    {attribute 'pingroup' := 'general'}
    out1 : INT;
    {attribute 'pingroup' := 'group1'}
    g1 : INT;
    g2 : REAL;
END_VAR
```

Pingroups in FBD editor

Attribute pin_presentation_order_inputs/outputs

Overview

The pragmas define the order in which the inputs and outputs of a function block are displayed in graphical language editors.

Syntax

```
{attribute 'pin_presentation_order_inputs' :=
'<input_k>,<input_l>,*,<input_m>'}
```

```
{attribute 'pin_presentation_order_outputs' :=
'<output_k>,<output_l>,*,<output_m>'}
```

The `*` character is the separator between the beginning and the end of the sorted list of input or output parameters. The separator is replaced by undefined input or output parameters. If the separator is not available, then the input or output parameters that are not defined explicitly in the pragma will be added to the end of the sorted list.

The pragmas are inserted in the first line in the declaration part of a function block.

NOTE: The pragmas `attribute 'pin_presentation_order_inputs` and `attribute 'pin_presentation_order_outputs` are not evaluated when the pragma `pingroup` is used.

Example

```
{attribute 'pin_presentation_order_inputs' :=
'input_2,*,input_1'}
{attribute 'pin_presentation_order_outputs' :=
'output_2, output_1}
FUNCTION_BLOCK POU_BASE
VAR_INPUT
     input_1 : BOOL;
     input_2 : INT;
     input_3 : INT;
     input_4 : INT;
END_VAR
VAR_OUTPUT
     output_1 : BOOL;
     output_2 : INT;
```

```
      output_3  :  INT;
      output_4  :  BOOL;
END_VAR
```

This sample pragma definition leads to the following order of the input and output pins of the `POU_Base` function block:

Attribute obsolete

Overview

You can add an obsolete pragma to a data type definition in order to cause a user-defined alert during a build, if the respective data type (structure, function block, and so on) is used within the project. Thus, you can announce that the data type is not used any longer.

Unlike a locally used message pragma, this alert is defined within the definition and thus global for all instances of the data type.

This pragma instruction is valid for the current line or - if placed in a separate line - for the subsequent line.

Syntax

{attribute 'obsolete' := 'user-defined text'}

Example

The obsolete pragma is inserted in the definition of function block `fb1`:

```
{attribute 'obsolete' := 'datatype fb1 not valid!'}
FUNCTION_BLOCK fb1
VAR_INPUT
i:INT;
END_VAR
...
```

If `fb1` is used as a data type in a declaration, for example, `fbinst: fb1;` the following alert will be dumped when the project is built:

```
'datatype fb1 not valid'
```

Attribute pack_mode

Overview

The pragma `{attribute 'pack_mode'}` defines the mode a data structure is packed while being allocated. Set the attribute on top of a data structure. It influences the packing of the whole structure.

Syntax

{attribute 'pack_mode' := '<value>'}

The placeholder <value> can have the following values:

pack_mode	Associated packing method
0	Aligned
1	1-byte-aligned
2	2-byte-aligned
4	4-byte-aligned
8	8-byte-aligned

Depending on the structure, there may be no difference in the memory mapping of the individual modes. For example, the memory allocation of a structure with `pack_mode = 4` can correspond to that of `pack_mode = 8`.

If structures are combined to arrays, bytes are added at the end of each structure to achieve that the next structure is aligned.

Example

```
{attribute 'pack_mode' := '1'}
TYPE myStruct:
STRUCT
 Enable: BOOL;
 Counter: INT;
 MaxSize: BOOL;
 MaxSizeReached: BOOL;
```

```
END_STRUCT
END_TYPE
```

A variable of data type `myStruct` is instantiated aligned.

If the address of its component `Enable` is `0x0100`, then the component `Counter` will follow on address `0x0101`, `MaxSize` on `0x0103` and `MaxSizeReached` on `0x0104`.

With `pack_mode=2`, `Counter` would be found on `0x0102`, `MaxSize` on `0x0104` and `MaxSizeReached` on `0x0106`.

Attribute qualified_only

Overview

When the pragma `{attribute 'qualified_only'}` is assigned on top of a global variable list, the variables of this list can only be accessed by using the global variable name, for example `gvl.g_var`. This works even for variables of enumeration type. It can be useful to avoid name mismatch with local variables.

Syntax

{attribute 'qualified_only'}

Example

Assume the following global variable list (GVL) is provided with attribute `'qualified_only'`:

```
{attribute 'qualified_only'}
VAR_GLOBAL
iVar:INT;
END_VAR
```

Within POU `PLC_PRG`, the global variable has to be called with the prefix `GVL`, as shown in this example:

```
GVL.iVar:=5;
```

The following incomplete call of the variable will be detected as an error:

```
iVar:=5;
```

Attribute reflection

Overview

The pragma `{attribute 'reflection'}` is attached to signatures. Due to performance reasons, it is an obligatory attribute for POUs carrying the `instance-path` attribute.

Syntax

{attribute 'reflection'}

Example

Refer to the `attribute instance-path` example.

Attribute subsequent

Overview

The pragma `{attribute 'subsequent'}` forces variables to be allocated in a row at one location in memory. If the list changes, the whole list will be allocated at a new location. This pragma is used in programs and global variable lists (GVL).

Syntax

{attribute 'subsequent'}

NOTE: If one variable in the list is `RETAIN`, the whole list will be located in retain memory.

NOTE: `VAR_TEMP` in a program with attribute `subsequent` will be detected as a compiler error.

Attribute symbol

Overview

The pragma `{attribute 'symbol'}` defines which variables are to be handled in the symbol configuration.

The following export operations are performed on the variables:
- Variables are exposed as symbols in the symbol configuration.
- Variables are exported to an XML file in the project directory.
- Variables are exported to a file not visible and available on the target system for external access, for example, by an OPC server.

Variables provided with that attribute will be downloaded to the controller even if they have not been configured or are not visible within the symbol configuration editor.

NOTE: The symbol configuration has to be available as an object below the respective application in the **Tools Tree**.

Syntax

{attribute 'symbol' := 'none' | 'read' | 'write' | 'readwrite'}

Access is only allowed on symbols coming from programs or global variable lists. For accessing a symbol, specify the symbol name completely.

You can assign the pragma definition to particular variables or collectively to all variables declared in a program.
- To be valid for a single variable, place the pragma in the line before the variable declaration.
- To be valid for all variables contained in the declaration part of a program, place the pragma in the first line of the declaration editor. In this case, you can also modify the settings for particular variables by explicitly adding a pragma.

The possible access on a symbol is defined by the following pragma parameters:
- `'none'`
- `'read'`
- `'write'`
- `'readwrite'`

If no parameter is defined, the default `'readwrite'` will be valid.

Example

With the following configuration, the variables `A` and `B` will be exported with read and write access. Variable `D` will be exported with read access.

```
{attribute 'symbol' := 'readwrite'}
PROGRAM PLC_PRG
VAR
A : INT;
B : INT;
```

```
{attribute 'symbol' := 'none'}
C : INT;
{attribute 'symbol' := 'read'}
D : INT;
END_VAR
```

Attribute warning disable

Overview

You can use the pragma `warning disable` to suppress alerts. To enable the display of the alert, use the pragma `warning restore`.

Syntax

{warning disable <compiler ID>}

Compiler ID: Every alert and every error detected by the compiler has a unique ID, which is displayed at the beginning of the description.

Example Compiler Messages

```
------ Build started: Application: Device.Application ------
typify code ...
C0196: Implicit conversion from unsigned Type 'UINT' to signed Type 'INT' : possible change of sign
Compile complete -- 0 errors
```

Example

```
VAR
     {warning disable C0195}
     test1 : UINT := -1;
     {warning restore C0195}
     test2 : UINT := -1;
END_VAR
```

In this example, an alert will be detected for `test2`. But no alert will be detected for `test1`.

Attribute enable_dynamic_creation

Overview

The pragma `enable_dynamic_creation` is required for using the `__NEW` operator for function blocks.

Syntax

{attribute 'enable_dynamic_creation'}

Insert the pragma in the first line in the declaration of the function block.

The Smart Coding Functionality

Smart Coding

Overview

Wherever identifiers (like variables or function block instances) can be entered (this can be inside of the IEC 61131-3 language editors or inside **Watch, Trace, Visualization** windows), the smart coding functionality is available. You can customize this feature in the **SmartCoding** section of the **Tools > Options** dialog box.

Support in Identifier Insertion

The smart coding functionality helps to insert a correct identifier:

- If you - at any place, where a global identifier can be inserted - insert a dot (.) instead of the identifier, a selection box will display. It lists the currently available global variables. You can choose one of these elements and press the RETURN key to insert it behind the dot. You can also insert the element by double-clicking the list entry.
- If you enter a function block instance or a structure variable followed by a dot (.), then a selection box will appear. It lists the input and output variables of the corresponding function block or the structure components. You can choose the desired element by pressing the RETURN key or by double-clicking the list entry to insert it.
- In the ST editor, if you enter any string and press CTRL+SPACE, a selection box will display. It lists the POUs and global variables available in the project. The first list entry, which is starting with the given string, will be selected. Press the RETURN key to insert it into the program.

Examples

The smart coding functionality offers components of structure:

The smart coding functionality offers components of a function block:

Data Types

What Is in This Chapter?

This chapter contains the following sections:

- General Information
- Standard Data Types
- Extensions to IEC Standard
- User-Defined Data Types

General Information

Data Types

Overview

You can use standard data types, user-defined data types, or instances of function blocks when programming in EcoStruxure Machine Expert. Each identifier is assigned to a data type. This data type dictates how much memory space will be reserved and what type of values it stores.

Standard Data Types

Standard Data Types

Overview

EcoStruxure Machine Expert supports all data types described by standard IEC61131-3.

The following data types are described in this chapter:

- BOOL
- Integer
- REAL / LREAL
- STRING
- WSTRING
- Time Data Types (LTIME)
- ANY and ANY_<type>

Additionally, some standard-extending data types are supported and you can define your own user-defined data types.

BOOL

BOOL type variables can have the values TRUE (1) and FALSE (0). 8 bits of memory space are reserved.

For further information, refer to the chapter BOOL constants.

NOTE: You can use implicit checks to validate the conversion of variable types (refer to the chapter POUs for Implicit Checks).

Integer

The table lists the available integer data types. Each of the types covers a different range of values. The following range limitations apply.

Data Type	Lower Limit	Upper Limit	Memory Space
BYTE	0	255	8 bit
WORD	0	65,535	16 bit
DWORD	0	4,294,967,295	32 bit
LWORD	0	$2^{64}-1$	64 bit
SINT	–128	127	8 bit
USINT	0	255	8 bit

INT	–32,768	32,767	16 bit
UINT	0	65,535	16 bit
DINT	–2,147,483,648	2,147,483,647	32 bit
UDINT	0	4,294,967,295	32 bit
LINT	-2^{63}	$2^{63}-1$	64 bit
ULINT	0	$2^{64}-1$	64 bit

NOTE: Conversions from larger types to smaller types may result in loss of information.

For further information, refer to the description of number constants.

NOTE: You can use implicit checks to validate the conversion of variable types (refer to the chapter POUs for Implicit Checks).

REAL / LREAL

The data types REAL and LREAL are floating-point types. They represent rational numbers.

Characteristics of REAL and LREAL data types:

Data type	Lower limit	Upper limit	Memory Space
REAL	-3.402823e+38	3.402823e+38	32 bit
LREAL	-1.7976931348623158e+308	1.7976931348623158e+308	64 bit

NOTE: The support of data type LREAL depends on the target device. See in the corresponding documentation whether the 64-bit type LREAL gets converted to REAL during compilation (possibly with a loss of information) or persists.

NOTE: If a REAL or LREAL is converted to SINT, USINT, INT, UINT, DINT, UDINT, LINT, or ULINT and the value of the real number is out of the value range of that integer, the result will be undefined and will depend on the target system. Even an exception is possible in this case. In order to get target-independant code, handle any range exceedance by the application. If the REAL/LREAL number is within the integer value range, the conversion will work on all systems in the same way.

When assigning `i1 := r1;` an error is detected. Therefore, the previous note applies when using conversion operators such as the following:

```
i1 := REAL_TO_INT(r1);
```

For further information, refer to REAL/LREAL constants (operands).

NOTE: You can use implicit checks to validate the conversion of variable types (refer to the chapter POUs for Implicit Checks).

STRING

A STRING data type variable can contain any string of characters. The size entry in the declaration determines the memory space to be reserved for the variable. It refers to the number of characters in the string and can be placed in parentheses or square brackets. If no size specification is given, the default size of 80 characters will be used.

In general, the length of a string is not limited. But string functions can only process strings with a length of 1...255 characters. If a variable is initialized with a string too long for the variable data type, the string will be correspondingly cut from right to left.

NOTE: The memory space needed for a variable of type STRING is 1 byte per character + 1 additional byte. This means, the `"STRING[80]"` declaration needs 81 bytes.

Example of a string declaration with 35 characters:

```
str:STRING(35):='This is a String';
```

For further information, refer to WSTRING and STRING Constants (Operands).

NOTE: You can use implicit checks to validate the conversion of variable types (refer to the chapter POUs for Implicit Checks).

WSTRING

The WSTRING data type differs from the STRING type (ASCII) by interpretation in Unicode format, and needing two bytes for each character and two bytes extra memory space (each only one in case of a STRING).

The library standard64.lib provides functions for WSTRING strings.

The number of characters for WSTRING depends on the contained characters. A size of 10 for WSTRING means that the length of the WSTRING can take a maximum of 10 WORDS. For some characters in Unicode, multiple WORDS are required for coding a character so that the number of characters does not have to correspond to the length of the WSTRING (10 in this case). The data type requires one WORD of extra memory as it is terminated with a `0`.

If a size is not defined, then 80 WORDS plus one for the terminating character `0` are allocated.

Examples:

```
wstr:WSTRING:="This is a WString";
```

```
wstr10 : WSTRING(10) := "1234567890";
```

For further information, refer to the following descriptions:

- STRING
- STRING constants (operands)

Time Data Types

The data types TIME, TIME_OF_DAY (shortened TOD), DATE, and DATE_AND_TIME (shortened DT) are handled internally like DWORD. Time is given in milliseconds in TIME and TOD. Time in TOD begins at 12:00 A.M. Time is given in seconds in DATE and DT beginning with January 1, 1970 at 12:00 A.M.

LTIME

LTIME is supported as time base for high resolution timers. LTIME is of size 64 bit and resolution nanoseconds.

Syntax of LTIME:

LTIME#<time declaration>

The time declaration can include the time units as used with the TIME constant and as:
- us : microseconds
- ns : nanoseconds

Example of LTIME:

```
LTIME1 := LTIME#1000d15h23m12s34ms2us44ns
```

Compare to TIME size 32 bit and resolution milliseconds.

For further information, refer to the following descriptions:
- Data Types
- TIME constants
- DATE constants
- DATE_AND_TIME constants
- TIME_OF_DAY constants

NOTE: You can use implicit checks to validate the conversion of variable types (refer to the chapter POUs for Implicit Checks).

ANY / ANY_<type>

When you implement a function, and one of the function inputs (VAR_INPUT) has a generic IEC data type (ANY or ANY_<type>), then the data type of the call parameter is not defined as unique. Variables of different data types can be passed to this function. The value passed and its type can be requested within the function via a predefined structure.

Generic IEC data types that allow the use of elementary data types for function inputs:

Hierarchy of generic data types			Elementary data types
ANY	ANY_BIT	–	BOOL, BYTE, WORD, DWORD, LWORD
	ANY_DATE	–	DATE_AND_TIME, DATE, TIME_OF_DAY

	ANY_NUM	ANY_REAL	REAL, LREAL
		ANY_INT	USINT, UINT, UDINT, ULINT SINT, INT, DINT, LINT
	ANY_STRING	–	STRING, WSTRING

Example:

```
FUNCTION ANYBIT_TO_BCD : DWORD
    VAR_INPUT
        value : ANY_BIT;
    END_VAR
```

If the function ANYBIT_TO_BCD is called, then a variable of data type BOOL, BYTE, WORD, DWORD, or LWORD can be passed to the function as a parameter.

Predefined structure:

When compiling the code, an ANY data type is replaced internally with the following structure:

```
TYPE AnyType :
STRUCT
          // the type of the actual parameter
     typeclass : __SYSTEM.TYPE_CLASS ;
          // the pointer to the actual parameter
     pvalue : POINTER TO BYTE;
          // the size of the data, to which the pointer points
     diSize : DINT;
END_STRUCT
END_TYPE
```

The actual call parameter assigns the structure elements at runtime.

Example:

This code example compares whether the two passed variables have the same type and the same value.

```
FUNCTION Generic_Compare : BOOL
VAR_INPUT
     any1 : ANY;
     any2 : ANY;
END_VAR
VAR
     icount: DINT;
```

```
END_VAR

Generic_Compare := FALSE;
IF any1.typeclass <> any2.typeclass THEN
     RETURN;
END_IF
IF any1.diSize <> any2.diSize THEN
     RETURN;
END_IF
// Byte comparison
FOR icount := 0 TO any1.diSize-1 DO
     IF any1.pvalue[iCount] <> any2.pvalue[iCount] THEN
          RETURN;
     END_IF
END_FOR
Generic_Compare := TRUE;
RETURN;
// END_FUNCTION
```

Also refer to the description of the `__VARINFO` operator.

Extensions to IEC Standard

Overview

This chapter lists the data types that are supported by EcoStruxure Machine Expert in addition to the standard IEC 61131-3.

What Is in This Section?

This section contains the following topics:

- UNION
- BIT
- References
- Pointers

UNION

Overview

As an extension to the IEC 61131-3 standard, you can declare unions in user-defined types.

The components of a union have the same offset. This means that they occupy the same storage location. Thus, assuming a union definition as shown in the following example, an assignment to `name.a` also manipulates `name.b`.

Example

```
TYPE name: UNION
a : LREAL;
b : LINT;
END_UNION
END_TYPE
```

BIT

Overview

You can use the BIT data type only for particular variables within Structures. The possible values are TRUE (1) and FALSE (0).

A BIT element consumes 1 bit of memory space and allows you to address single bits of a structure by name (for further information, refer to the paragraph Bit Access in Structures). Bit elements which are declared one after another will be combined in bytes. In contrast to BOOL types, where 8 bits are reserved in any case, the use of memory space can get optimized. On the other hand, the access to bits takes definitely more time. For that reason, use the BIT data type if you want to store several boolean pieces of information in a compact format.

References

Overview

This data type is available in extension to the IEC 61131-3 standard.

A reference stores the address of an object (a variable) that is located elsewhere in memory; in this respect, the behavior is identical to a pointer.

In contrast to a pointer, a variable behaves like an object when the syntax is concerned. Additionally, variables declared as REFERENCE provide the following advantages compared to POINTERS:

- A reference does not have to be dereferenced explicitly (with `^`) to access the contents of the referenced object.
- When passing values to input parameters of functions/function blocks/methods, the following applies: If an input is declared as `REFERENCE TO <data type>`, a variable of the corresponding `<data type>` can be passed (`refInput :=` variable instead of `ptrInput :=ADR`(variable)).
- The compiler verifies that references of the same data type are assigned to each other.

For further information, refer to the *Assignment operator* `REF` description.

Syntax

<identifier> : REFERENCE TO <data type>

Example Declaration

```
A : REFERENCE TO DUT;
B : DUT;
C : DUT;
A REF= B; // corresponds to A := ADR(B);
A := C; // corresponds to A^ := C;
```

NOTE: It is not possible to declare references like REFERENCE TO REFERENCE or ARRAY OF REFERENCE or POINTER TO REFERENCE.

Check for Valid References

You can use the operator `__ISVALIDREF` to check whether a reference points to a valid value that is a value unequal to 0.

Syntax

```
<boolean variable> := __ISVALIDREF(identifier, declared with type <REFERENCE TO <datatype>);
<boolean variable> will be TRUE, if the reference points to a valid value, FALSE if not.
```

Example

Declaration

```
ivar : INT;
ref_int : REFERENCE TO INT;
```

```
ref_int0: REFERENCE TO INT;
testref: BOOL := FALSE;
```

Implementation

```
ivar := ivar +1;
ref_int REF= ivar;
ref_int0 REF= 0;
testref := __ISVALIDREF(ref_int);    (* will be TRUE, because ref_int points to ivar, which is unequa
testref := __ISVALIDREF(ref_int0); (* will be FALSE, because ref_int is set to 0 *)
```

Pointers

Overview

As an extension to the IEC 61131-3 standard, you can use pointers.

Pointers save addresses while an application program is running. A pointer can point to a variable with any data type. The possibility of using an implicit pointer monitoring function is described further below in the paragraph CheckPointer function.

Syntax of a Pointer Declaration

```
<identifier>: POINTER TO <data type>;
```

Dereferencing a pointer means to obtain the value currently stored at the address to which it is pointing. You can dereference a pointer by adding the content operator ^ (ASCII caret or circumflex symbol) after the pointer identifier. See `pt^` in the example below.

You can use the `ADR` address operator to assign the address of a variable to a pointer.

Example

```
VAR
   pt:POINTER TO INT;   (* of pointer pt *)
var_int1:INT := 5;   (* declaration of variables var_int1 and var_int2 *)
   var_int2:INT;
END_VAR
pt := ADR(var_int1); (* address of var_int1 is assigned to pointer pt *)
var_int2:= pt^;        (* value 5 of var_int1 gets assigned to var_int2 via dereferencing of pointer
```

Function Pointers

EcoStruxure Machine Expert also supports function pointers. These pointers can be passed to external libraries, but it is not possible to call a function pointer within an application in the programming system. The runtime function for registration of callback functions (system library function) expects the function pointer, and, depending on the callback for which the registration was requested, the respective function will be called implicitly by the runtime system (for example, at STOP). In order to enable such a system call (runtime system), set the respective properties (by default under **View > Properties... > Build**) for the function object.

You can use the `ADR` operator on function names, program names, function block names, and method names. Since functions can move after online change, the result is not the address of the function, but the address of a pointer to the function. This address is valid as long as the function exists on the target.

Executing the **Online Change** command can change the contents of addresses.

CAUTION

INVALID POINTER

Verify the validity of the pointers when using pointers on addresses and executing the Online Change command.

Failure to follow these instructions can result in injury or equipment damage.

Index Access to Pointers

As an extension to the IEC 61131-3 standard, index access `[]` to variables of type POINTER, STRING and WSTRING is allowed.

- `pint[i]` returns the base data type.
- Index access on pointers is arithmetic:
  If the index access is used on a variable of type pointer, the offset `pint[i]` equates to `(pint + i * SIZEOF(base type))^`. The index access also performs an implicit dereferencing on the pointer. The result type is the base type of the pointer.
  Consider that `pint[7] does not equate to (pint + 7)^`.
- If the index access is used on a variable of type STRING, the result is the character at offset `index-expr`. The result is of type BYTE. `str[i]` will return the i-th character of the string as a SINT (ASCII).
- If the index access is used on a variable of type WSTRING, the result is the character at offset `index-expr`. The result is of type WORD. `wstr[i]` will return the i-th character of the string as INT (Unicode).

NOTE: You can also use References. In contrast to a pointer, references directly affect a value.

`CheckPointer` Function

To monitor pointer access during runtime, you can use the implicit monitoring function `CheckPointer`. You can adapt it, if required. To achieve this, add the **POUs for implicit checks** object to the application. Activate the check box related to the category **Pointer Checks**.

NOTE: You have to implement the `CheckPointer` function during machine commissioning to verify whether the passed pointer refers to a valid memory address and whether the alignment of the referenced memory area fits to the data type of the variable that the pointer points to.

`CheckPointer` monitors variables of type REFERENCE TO in a similar way.

NOTE: There is no implicit call of the check function for the `THIS` pointer.

Template:

Declaration part:

```
// Implicitly generated code : DO NOT EDIT
FUNCTION CheckPointer : POINTER TO BYTE
VAR_INPUT
ptToTest : POINTER TO BYTE;
iSize : DINT;
iGran : DINT;
bWrite: BOOL;
END_VAR
```

When called, the following input parameters are provided to the function:

- `ptToTest`: Target address of the pointer
- `iSize`: Size of referenced variable; the data type of `iSize` has to be integer-compatible and has to cover the maximum potential data size stored at the pointer address.
- `iGran`: Granularity of the access that is the largest non-structured data type used in the referenced variable; the data type of `iGran` has to be integer-compatible
- `bWrite`: Type of access (TRUE= write access, FALSE= read access); the data type of `bWrite` has to be BOOL.

Implementation of the `CheckPointer` Function for PacDrive Controllers

The implementation of the `CheckPointer` function for PacDrive controllers (PacDrive LMC Eco / PacDrive LMC Pro/Pro2) is to generate a system runtime exception and to write a call stack to the message logger when the memory address or the alignment is invalid.

Implementation part:

```
CheckPointer := ptToTest;
IF ptToTest = 0 THEN
```

```
            FC_DiagMsgWrite(4, 'CP = 0');
            FC_SysUserCallStack(0);
ELSE
            CheckPointer := ptToTest;
END_IF
```

Implementation of the `CheckPointer` Function for Optimized Controllers

The implementation of the `CheckPointer` function for Optimized Controllers (for example, Modicon M241 Logic Controller) is to return the passed pointer.

Implementation part (incomplete):

```
// No standard way of implementation. Fill your own code here
CheckPointer := ptToTest;
```

In case of a positive result of the check, the unmodified input pointer will be returned (`ptToTest`).

User-Defined Data Types

What Is in This Section?

This section contains the following topics:

- Defined Data Types
- Arrays
- Structures
- Enumerations
- Subrange Types

Defined Data Types

Overview

Additionally, to the standard data types, you can define special data types within a project.

You can define them via creating DUT (Data Unit Type) objects in the **POUs tree** or **Devices tree** or within the declaration part of a POU.

See the recommendations on the naming of objects in order to make it as unique as possible.

See the following user-defined data types:

- arrays
- structures
- enumerations
- subrange types
- references
- pointers

Arrays

Overview

One, two and three-dimensional fields (arrays) are supported as elementary data types. You can define arrays both in the declaration part of a POU and in the global variable lists. You can also use implicit boundary checks. You can declare arrays with defined length and with variable length.

The data type ARRAY with variable length can only be declared for `VAR_IN_OUT` variables of function blocks, methods, and functions. Use the operators `LOWER_BOUND(<array name>,<dim>)` and `UPPER_BOUND(<array name>,<dim>)` to get the lower and upper limits of this array.

Syntax for the Declaration of an Array with Defined Length

<Array_Name> : ARRAY [<ll1>..<ul1>,<ll2>..<ul2>,<ll3>..<ul3>] OF <elem. type>

`ll1`, `ll2`, `ll3` identify the lower limit of the field range.

`ul1`, `ul2`, and `ul3` identify the upper limit of the field range.

The range values have to be of type integer.

Example for the Declaration of an Array with Defined Length

```
Card_game: ARRAY [1..13, 1..4] OF INT;
```

Syntax for the Declaration of an Array with Variable Length

<Array name> : ARRAY [*|, *|, *] OF <data type>;

Example for the Declaration of an Array with Variable Length

```
FUNCTION SUM: INT; // Onedimensional
arrays of variable lengths can be
passed to this addition function.
VAR_IN_OUT
A: ARRAY [*] OF INT;
END_VAR
VAR
   i, sum2 : DINT;
END_VAR
sum2:= 0;
FOR i:= LOWER_BOUND(A,1) TO UPPER_BOUND(A,
1) // The length of the respective array is
determined.
   sum2:= sum2 + A[i];
END_FOR;
SUM:= sum2;
```

Initializing Arrays

Example for complete initialization of an array

```
arr1 : ARRAY [1..5] OF INT := [1,2,3,4,5];
arr2 : ARRAY [1..2,3..4] OF INT := [1,3(7)]; (* short for 1,7,7,7 *)
arr3 : ARRAY [1..2,2..3,3..4] OF INT := [2(0),4(4),2,3];
        (* short for 0,0,4,4,4,4,2,3 *)
```

Example of the initialization of an array of a structure

Structure definition

```
TYPE STRUCT1
STRUCT
 p1:int;
 p2:int;
 p3:dword;
END_STRUCT
END_TYPE
```

Array initialization

```
ARRAY[1..3] OF STRUCT1:= [(p1:=1,p2:=10,p3:=4723),(p1:=2,p2:=0,p3:=299),(p1:=14,p2:=5,p3:=112)];
```

Example of the partial initialization of an array

```
arr1 : ARRAY [1..10] OF INT := [1,2];
```

Elements where no value is pre-assigned are initialized with the default initial value of the basic type. In the previous example, the elements `arr1[3]...arr1[10]` are therefore initialized with 0.

Example of the Initialization of an Array of Function Blocks with Additional Parameters in `FB_Init`

Example of a function block `FB` and method `FB_Init` with two parameters:

```
FUNCTION_BLOCK FB
VAR
     _nId : INT;
     _lrIn : LREAL;
END_VAR
METHOD FB_Init : BOOL
VAR_INPUT
     bInitRetains : BOOL;
     bInCopyCode : BOOL;
     nId : INT;
     lrIn : LREAL;
END_VAR
_nId := nId;
_lrIn := lrIN;
```

Example of array declaration with initialization:

```
PROGRAM PLC_PRG
VAR
   inst : FB(nId := 11, lrIn := 33.44);
     ainst : ARRAY [0..1, 0..1] OF FB[(nId :=
12, lrIn := 11.22), (nId := 13, lrIn :=
22.33), (nId := 14, lrIn := 33.55),(nId := 15,
lrIn := 11.22)];
END_VAR
```

Accessing Array Elements

In a two-dimensional array, access the elements as follows:

<Array name>[Index1,Index2]

Example:

```
Card_game [9,2]
```

Check Functions on Array Bounds

In order to access an array element properly during runtime, the function `CheckBounds` has to be available to the application. For information on inserting the function, refer to the description of the **POUs for implicit checks** function.

This check function has to treat boundary violations by an appropriate method (for example, by setting a detected error flag or adjusting the index). The function is called implicitly as soon as a variable of type ARRAY is assigned.

Example for the Use of Function `CheckBounds`

The default implementation of the check function is the following:

Declaration part:

```
// Implicitly generated code : DO NOT EDIT
FUNCTION CheckBounds : DINT
VAR_INPUT
index, lower, upper: DINT;
END_VAR
```

Implementation part:

```
// Implicitly generated code : Only an Implementation suggestion
IF  index < lower THEN
CheckBounds := lower;
ELSIF  index > upper THEN
CheckBounds := upper;
ELSE
CheckBounds := index;
END_IF
```

When called, the function gets the following input parameters:

- `index`: field element index
- `lower`: the lower limit of the field range
- `upper`: the upper limit of the field range

As long as the index is within the range, the return value is the index itself. Otherwise, in correspondence to the range violation either the upper or the lower limit of the field range will be returned.

Exceeding the Upper Limit of the Array `a`

The upper limit of the array `a` is exceeded in the following example:

```
PROGRAM PLC_PRG
VAR
 a: ARRAY[0..7] OF BOOL;
     b: INT:=10;
END_VAR
a[b]:=TRUE;
```

In this example, the implicit call to the `CheckBounds` function preceding the assignment affects that the value of the index is changed from `10` into the upper limit `7`. Therefore, the value TRUE is assigned to the element `a[7]` of the array. This is how you can correct attempted access outside the array range via the function `CheckBounds`.

Structures

Overview

Create structures in a project as DUT (Data Type Unit) objects via the **Add Object** dialog box.

They begin with the keywords `TYPE` and `STRUCT` and end with `END_STRUCT` and `END_TYPE`.

Syntax

```
TYPE <structurename>:
STRUCT
     <declaration of variables 1>
     ...
     <declaration of variables n>
END_STRUCT
END_TYPE
```

<structurename> is a type that is recognized throughout the project and can be used like a standard data type.

Nested structures are allowed. The only restriction is that variables may not be assigned to addresses (the `AT` declaration is not allowed).

Example

Example for a structure definition named `Polygonline`:

```
TYPE Polygonline:
STRUCT
     Start:ARRAY [1..2] OF INT;
     Point1:ARRAY [1..2] OF INT;
     Point2:ARRAY [1..2] OF INT;
     Point3:ARRAY [1..2] OF INT;
     Point4:ARRAY [1..2] OF INT;
     End:ARRAY [1..2] OF INT;
END_STRUCT
END_TYPE
```

Initialization of Structures

Example:

```
Poly_1:polygonline := ( Start:=[3,3], Point1:=[5,2], Point2:=[7,3], Point3:=[8,5], Point4:=
[5,7], End:= [3,5]);
```

Initializations with variables are not possible. For an example of the initialization of an array of a structure, refer to Arrays.

Access on Structure Components

You can gain access to structure components using the following syntax:

<structurename>.<componentname>

For the previous example of the structure `Polygonline`, you can access the component `Start` by `Poly_1.Start`.

Bit Access in Structures

The data type BIT is a special data type which can only be defined in structures. It consumes memory space of 1 bit and allows you to address single bits of a structure by name.

```
TYPE <structurename>:
STRUCT
     <bitname bit1> : BIT;
     <bitname bit2> : BIT;
     <bitname bit3> : BIT;
     ...
```

```
      <bitname bitn> : BIT;
END_STRUCT
END_TYPE
```

You can gain access to the structure component `BIT` by using the following syntax:

<structurename>.<bitname>

NOTE: The usage of references and pointer on `BIT` variables is not possible. Furthermore, `BIT` variables are not allowed in arrays.

Comparing Structures

For `EQ` operators, it is possible to compare operands of type STRUCT (structure) if the target system supports structured data types.

Enumerations

Overview

An enumeration is a user-defined type that is made up of a number of comma-delimited string constants. These constants are referred to as enumeration values. Enumeration values are identifiers for global constants in the project.

Enumeration values are recognized globally in all areas of the project even if they are declared within a POU.

An enumeration is created in a project as a DUT object via the **Add Object** dialog box.

NOTE: Local enumeration declaration is only possible within TYPE.

Syntax for Declaring Enumerations

```
TYPE <enum identifier>: (<enum_0>|:=<value>,<enum_1>|:=<value>, ...,<enum _n>|:=<value>)|
<base data type>|:=<default value>;
END_TYPE
```

The following elements can additionally be used:
- Initialization of single enumeration values
- Definition of a `<base data type>` (refer to the paragraph Second Extension to the IEC 61131-3 Standard)
- Definitions of a default value for initializing all components

Syntax for Declaring Variables with the Enumeration Type

```
<variable identifier> : <enum identifier>| := <initialization value>
```

A variable of type `<enum identifier>` can take the enumeration values `<enum_..>`.

Initialization

If...	Then ...
o If you have not defined a `<default value>` in the declaration of the enumeration (see following example), and o If you have neither defined an explicit initialization value for the declaration of variables of the type `<enum identifier>`	The variable is initialized with the value of the first enumeration component. This applies unless a component is initialized with the value 0 in the enumeration declaration. In this case, the variable is also initialized with the value 0.

Example of an Enumeration with an Explicit Initialization Value for a Component

Declaration of the Enumeration

```
TYPE TRAFFIC_SIGNAL: (red, yellow, green:=10);
     (* The initial value for each of the colors is red 0, yellow 1, green 10 *)
END_TYPE
```

In this declaration, the first two components get default initialization values:

`red = 0, yellow = 1`

The initialization value of the third component is explicitly defined:

`green = 10`

Use in the Implementation

```
TRAFFIC_SIGNAL1 : TRAFFIC_SIGNAL;
TRAFFIC_SIGNAL1:=10; (* The value of the TRAFFIC_SIGNAL1 is "green" *)
FOR i:= red TO green DO
     i := i + 1;
END_FOR;
```

Example of an Enumeration with a Defined Default Value

Declaration of the Enumeration

```
TYPE COLOR :
(
```

```
      red,
      yellow,
      green
)  := green;
END_TYPE
```

Use in the Implementation

```
c1  :  COLOR;
c2  :  COLOR  :=  yellow;
```

In this case, the variable `c1` is initialized with the value `green`. The initialization value `yellow` is defined for `c2`.

First Extension to the IEC 61131-3 Standard

You can use the type name of enumerations (as a scope operator) to disambiguate the access to an enumeration constant. Therefore, it is possible to use the same constant in different enumerations.

Example

Definition of two enumerations with same-named components

```
TYPE  COLORS_1:  (red,  blue);
END_TYPE
TYPE  COLORS_2:  (green,  blue,  yellow);
END_TYPE
```

Use of same-named components from different enumerations in one block

```
colorvar1  :  COLORS_1;
colorvar2  :  COLORS_2;
```

```
(*  valid:  *)
colorvar1  :=  COLORS_1.blue;
colorvar2  :=  COLORS_2.blue;
```

```
(*  invalid:  *)
colorvar1  :=  blue;
colorvar2  :=  blue;
```

Second Extension to the IEC 61131-3 Standard

You can specify explicitly the base data type of the enumeration, which by default is INT.

Example of an Explicit Other Base Data Type for an Enumeration

```
TYPE COLORS_2 : (yellow, blue, green:=16#8000)DINT;
END_TYPE
```

NOTE: The `strict` attribute is automatically assigned to each enumeration that you add to a project in the line above the `TYPE` declaration. This leads to errors that are detected during the compile process in the following cases:

- Arithmetic operation with variables of the enumeration type
- Assignment of a constant value to a variable of the enumeration type, in which the constant does not correspond to an enumeration value
- Assignment of a non-constant value to a variable of the enumeration type, in which the non-constant has another data type than the enumeration type

You can explicitly add or remove the attribute.

Syntax: `{attribute 'strict'}`

Subrange Types

Overview

A subrange type is a user-defined type whose range of values is only a subset of that of the basic data type. You can also use implicit range boundary checks.

You can do the declaration in a DUT object but you can also declare a variable directly with a subrange type.

Syntax

Syntax for the declaration as a DUT object:

TYPE <name>: <Inttype> (<ug>..<og>) END_TYPE;

`<name>`	a valid IEC identifier
`<inttype>`	one of the data types SINT, USINT, INT, UINT, DINT, UDINT, BYTE, WORD, DWORD (LINT, ULINT, LWORD)
`<ug>`	a constant compatible with the basic type, setting the lower boundary of the range types The lower boundary itself is included in this range.

<og>	a constant compatible with the basic type, setting the upper boundary of the range types. The upper boundary itself is included in this basic type.

Example

```
TYPE
  SubInt  :  INT  (-4095..4095);
END_TYPE
```

Direct Declaration of a Variable with a Subrange Type

```
VAR
  i  :  INT  (-4095..4095);
  ui  :  UINT  (0..10000);
END_VAR
```

If a value is assigned to a subrange type (in the declaration or in the implementation) but does not match this range (for example, `i:=5000` in the upper shown declaration example), a message will be issued.

Check Functions for Range Bounds

In order to observe the range bounds of subrange types during runtime, the functions `CheckRangeSigned`, `CheckLRangeSigned` or `CheckRangeUnsigned`, `CheckLRangeUnsigned` have to be added to the application. For information on inserting the function, refer to the description of the **POUs for implicit checks** function.

The purpose of this check function is the proper treatment of violations of the subrange (for example, by setting an error flag or changing the value). The function is called implicitly as soon as a variable of subrange type is assigned.

WARNING

UNINTENDED EQUIPMENT OPERATION

Do not change the declaration part of an implicit check function.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

Example

The assignment of a variable belonging to a signed subrange type entails an implicit call to `CheckRangeSigned`. The default implementation of that function trimming a value to the permissible range is provided as follows:

Declaration part:

```
// Implicitly generated code : DO NOT EDIT
FUNCTION CheckRangeSigned : DINT
VAR_INPUT
value, lower, upper: DINT;
END_VAR
```

Implementation part:

```
// Implicitly generated code : Only an Implementation suggestion
IF (value < lower) THEN
CheckRangeSigned := lower;
ELSIF(value > upper) THEN
CheckRangeSigned := upper;
ELSE
CheckRangeSigned := value;
END_IF
```

When called, the function gets the following input parameters:

- `value`: the value to be assigned to the range type
- `lower`: the lower boundary of the range
- `upper`: the upper boundary of the range

As long as the assigned value is within the valid range, it will be used as return value of the function. Otherwise, in correspondence to the range violation, either the upper or the lower boundary of the range will be returned.

The assignment `i:=10*y` will now be replaced implicitly by

```
i := CheckRangeSigned(10*y, -4095, 4095);
```

If `y`, for example, has the value `1000`, the variable `i` will not be assigned to 10*1000=10000 (as provided by the original implementation), but to the upper boundary of the range that is 4095.

The same applies to function `CheckRangeUnsigned`.

NOTE: If neither of the functions is available, no type checking of subrange types occurs during runtime. In this case, you can assign any DINT/UDINT value to a variable of subrange type DINT/UDINT. You can assign any LINT/ULINT value to a variable of a subrange type LINT/ULINT.

Programming Guidelines

What Is in This Chapter?

This chapter contains the following sections:

Naming Conventions

General Information

Creating Designator Names

Choose a relevant, short, description in English for each designator: the basis name. The basis name should be self-explanatory. Capitalize the first letter of each word in the basis name. Write the rest in lower case letters (example: `FileSize`). This basis name receives prefixes to indicate scope and properties.

Whenever possible, the designator should not contain more than 20 characters. This value is a guideline. You can adjust the number upward or downward if necessary.

If abbreviations of standard terms (TP, JK-FlipFlop, ...) are used, the name should not contain more than 3 capital letters in sequence.

Case-Sensitivity

Consider case-sensitivity, especially for prefixes, to improve readability when using designators in the IEC program.

NOTE: The compiler is not case-sensitive.

Valid Characters

Use only the following letters, numbers and special characters in designators:

0...9, A...Z, a...z,

In order to be able to display the prefixes clearly, an underline is used as the separator. The syntax is explained in the respective prefix section.

Do not use underscores in the basis name.

Examples

Recommended Designator	Not Recommended Designators
`diState`	`diSTATE`
`xInit`	`x_Init`
`diCycleCounter`	`diCyclecounter`
`lrRefVelocity`	`lrRef_Velocity`
`c_lrMaxPosition`	`clrMaxPosition`
`FC_PidController`	`FC_PIDController`

Prefixes

What Is in This Section?

This section contains the following topics:

Prefix Parts

Overview

Prefixes are used to assign names by function.

The following parts of prefixes are available:

Prefix part	Use	Syntax	Example
scope_prefix	scope of variables and constants	[scope]_[designator]	`G_diFirstUserFault`
data_type_prefix	identifying the data type of variables and constants	[type][designator]	`xEnable`
property_prefix	identifying the properties of variables and constants	[property]_[designator]	`c_iNumberOfAxes`
POU_prefix	identifying if POU was implemented as a function, function block, or program	[POU]_[designator]	`FB_VisuController`
namespace_prefix	for POUs, data types, variables, and constants declared within a library	[namespace].[identifier]	`TPL.G_dwErrorCode`

Order of Prefixes

Overview

Designators contain the scope prefix and the type prefix. Use the property prefix according to the property of the variables (for example, for constants). An additional namespace prefix is used for libraries.

Obligatory Order

The following order is obligatory:

scope][property][_][type][identifier]

Scope prefixes and property prefixes are separated from type prefixes by an underscore (_).

Example

```
Gc_dwErrorCode     :  DWORD;
diCycleCounter     :  DINT;
```

The additional namespace prefix is used for libraries:

[namespace].[scope][property][_][type][identifier]

Example

```
ExampleLibrary.Gc_dwErrorCode
```

Independent Program Organization Units (POUs)

Insert an underscore to separate program organization units (functions, function blocks, and programs) prefixes from identifiers:

[POU][_][identifier]

Example

```
FB_MotionCorrection
```

Use the additional namespace prefix for libraries:

[namespace].[POU][_][identifier]

Namespace prefixes are separated from POU prefixes by a dot (.).

Example

```
ExampleLibrary.FC_SetError()
```

Dependent Program Organization Units (POUs)

Methods, actions, and properties are considered to be dependent POUs. These are used on a level below an independent POU.

Methods and actions do not have any prefixes.

Properties receive the type prefix of their return value.

Example

```
PROPERTY lrVelocity : LREAL
```

Scope Prefix

Overview

The scope prefix indicates the scope of variables and constants. It indicates whether it is a local or global variable, or a constant.

Global variables are indicated by a capital `G_` and a property prefix `c` is added to global constants (followed by an underscore in each case).

NOTE: Additionally identify the global variables and constants of libraries with the namespace of the library.

Scope Prefix	Type	Use	Example
no prefix	`VAR`	local variable	`xEnable`
`G_`	`VAR_GLOBAL`	global variable	`G_diFirstUserFault`
`Gc_`	`VAR_GLOBAL CONSTANT`	global constant	`Gc_dwErrorCode`

Example

```
VAR_GLOBAL CONSTANT
    Gc_dwExample : DWORD := 16#0000001A;
END_VAR
```

Access to the global variable of a library with the namespace `INF`:

```
INF.G_dwExample := 16#0000001A;
```

Data Type Prefix

Standard Data Types

The data type prefix identifies the data type of variables and constants.

NOTE: The data type prefix can also be composite, for example, for pointers, references and arrays. The pointer or array is listed first, followed by the prefix of the pointer type or array type.

The IEC 61131-3 standard data type prefixes as well as the prefixes for the extensions to the standard are listed in the table.

Data type prefix	Type	Use (memory location)	Example
x	BOOL	boolean (8 bit)	xName
by	BYTE	bit sequence (8 bit)	byName
w	WORD	bit sequence (16 bit)	wName
dw	DWORD	bit sequence (32 bit)	dwName
lw	LWORD	bit sequence (64 bit)	lwName
si	SINT	short integer (8 bit)	siName
i	INT	integer (16 bit)	iName
di	DINT	doubled integer (32 bit)	diName
li	LINT	long integer (64 bit)	liName
uli	ULINT	long integer (64 bit)	uliName
usi	USINT	short integer (8 bit)	usiName
ui	UINT	integer (16 bit)	uiName
udi	UDINT	doubled integer (32 bit)	udiName
r	REAL	floating-point number (32 bit)	rName
lr	LREAL	doubled floating-point number (64 bit)	lrName
dat	DATE	date (32 bit)	datName
t	TOD	time (32 bit)	tName
dt	DT	date and time (32 bit)	dtName
tim	TIME	duration (32 bit)	timName
ltim	LTIME	duration (64 bit)	ltimName
s	STRING	character string ASCII	sName

ws	WSTRING	character string unicode	wsName
p	pointers	pointer	pxName
r	reference	reference	rxName
a	array	field	axName
e	enumeration	list type	eName
st	struct	structure	stName
if	interface	interface	ifMotion
ut	union	union	uName
fb	function block	function block	fbName

Examples

```
piCounter: POINTER TO INT;
aiCounters: ARRAY [1..22] OF INT;
paiRefCounter: POINTER TO ARRAY [1..22] OF INT;
apstTest    : ARRAY[1..2] OF POINTER TO ST_MotionStructure;
rdiCounter : REFERENCE TO DINT;
ifMotion    : IF_Motion;
```

Property Prefix

Overview

The property prefix identifies the properties of variables and constants.

Prefix Type	Use	Syntax	Example
c_	VAR CONSTANT	local constant	c_xName

r_	VAR RETAIN	remanent variable type retain	r_xName
p_	VAR PERSISTENT	remanent variable type persistent	p_xName
rp_	VAR PERSISTENT	remanent variable of type retain persistent	rp_xName
i_	VAR_INPUT	input parameter of a POU	i_xName
q_	VAR_OUTPUT	output parameter of a POU	q_xName
iq_	VAR_IN_OUT	in-/output parameter of a POU	iq_xName
ati_	AT %IX x.y AT %IB z AT %IW k	input variable that should write on the IEC input area	ati_x0_0MasterEncoderInitOK
atq_	AT %QX x.y AT %QB z AT %QW k	output variable that should write on the IEC input area	atq_w18AxisNotDone
atm_	AT %MX x.y AT %MB z AT %MW k	marker variable that should write on the IEC marker area	atm_w19ModuleNotReady

NOTE:

- Do not declare constants as `RETAIN` or `PERSISTENT`.
- Do not declare any `RETAIN` variables within POUs. This administers the complete POU in the retain memory area.

Example of `AT`-Declared Variables

The name of the `AT`-declared variable also contains the type of the target variable. It is used like the type prefix.

```
ati_xEncoderInit AT %IX0.0 : BOOL;
atq_wAxisNotDone AT %QW18 : WORD;
atm_wModuleNotReady AT %MW19 : WORD;
```

NOTE: A variable can also be allocated to an address in the mapping dialog of a device in the controller configuration (device editor). Whether a device offers this dialog is described in its documentation.

POU Prefix

Overview

The following program organization units (POU) are defined in IEC 61131-3:

- function
- function block
- program
- data structure
- list type
- union
- interface

EcoStruxure Machine Expert offers additional POUs for the creation and management of test cases:

- test case
- test series
- test resources

The designator is composed of a POU prefix and as short a name as possible (for example, `FB_GetResult`). Just like a variable, capitalize the first letter of each word in the basis name. Write the rest in lower case letters. Form a composite POU name from a verb and a noun.

The prefix is written with an underscore before the name and identifies the type of POU based on the table:

POU prefix	Type	Use	Example
SR_	PROGRAM	program	SR_FlowPackerMachine
FB_	FUNCTION_BLOCK	function blocks	FB_VisuController
FC_	FUNCTION	functions	FC_SetUserFault
ST_	STRUCT	data structure	ST_StandardModuleInterface
ET_	Enumeration	list type	ST_StandardModuleInterface
UT_	UNION	union	UT_Values
IF_	INTERFACE	interface	IF_CamProfile
TC_	Testcase*	test case	TC_MultiCam
TS_	TestSeries*	test series	TS_Robotic
TR_	Test resources*	test resources	TR_Axes
*IEC identifier enhancements introduced by Schneider Electric ETest plugin			

Namespace Prefix

Overview

You can view the namespace of a library in the **Library Manager**. Use a short acronym (PacDriveLib -> `PDL`) as namespace. Do not change the default namespace of a library.

To reserve an unambiguous namespace for your own, self-developed libraries, contact your Schneider Electric responsible.

Example

A function `FC_DoSomething()` is located within the library TestlibraryA (namespace `TLA`) as well as in TestlibraryB (namespace `TLB`). The respective function is accessed by prefixing the namespace.

If both libraries are located within a project, the following call-up results in an error detected during compilation:

```
FC_DoSomething();
```

In this case, it is necessary to define clearly which POU is to be called up.

```
TLA.FC_DoSomething();
TLB.FC_DoSomething();
```

Operators

Overview

EcoStruxure Machine Expert supports all IEC operators. In contrast to the standard functions, these operators are recognized implicitly throughout the project.

Besides the IEC operators, the following operators are supported which are not prescribed by the standard:

- ANDN
- ORN
- XORN
- SIZEOF (refer to arithmetic operators)
- ADR
- BITADR
- content operator (refer to address operators
- some scope operators

What Is in This Chapter?

This chapter contains the following sections:

- Numeric Functions
- IEC Extending Operators
- Initialization Operator

Arithmetic Operators

Overview

The following operators, prescribed by the IEC1131-3 standard, are available:

- ADD
- MUL
- SUB
- DIV
- MOD
- MOVE

Additionally, there is the following standard-extending operator:

- SIZEOF

Consider possible overflows of arithmetic operations in case the resulting value exceeds the range of the data type used for the result variable. This may result in low values being written to the machine instead of high values or vice versa.

WARNING

UNINTENDED EQUIPMENT OPERATION

Always verify the operands and results used in mathematical operations to avoid arithmetic overflow.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

What Is in This Section?

This section contains the following topics:

- ADD
- MUL

- SUB
- DIV
- MOD
- MOVE
- SIZEOF

ADD

Overview

IEC operator for the addition of variables

Allowed types

- BYTE
- WORD
- DWORD
- LWORD
- SINT
- USINT
- INT
- UINT
- DINT
- UDINT
- LINT
- ULINT
- REAL
- LREAL
- TIME
- TIME_OF_DAY(TOD)
- DATE
- DATE_AND_TIME(DT)

For time data types, the following combinations are possible:

- TIME+TIME=TIME

- TOD+TIME=TOD
- DT+TIME=DT

In the FBD/LD editor, the `ADD` operator is an extensible box. This means, instead of a series of concatenated `ADD` boxes, you can use 1 box with multiple inputs. Use the command **Insert Input** for adding further inputs. The number is unlimited.

Example in IL

```
LD       7
ADD      2
ADD      4
ADD      7
ST       iVar
```

Example in ST

```
var1 := 7+2+4+7;
```

Examples in FBD

1. series of `ADD` boxes
2. extended `ADD` box
3. `ADD` box with `EN/ENO` parameters

MUL

Overview

IEC operator for the multiplication of variables

Allowed types

- BYTE
- WORD
- DWORD
- LWORD
- SINT
- USINT

- INT
- UINT
- DINT
- UDINT
- LINT
- ULINT
- REAL
- LREAL
- TIME

TIME variables can be multiplied with integer variables.

In the FBD/LD editor, the `MUL` operator is an extensible box. This means, instead of a series of concatenated `MUL` boxes, you can use 1 box with multiple inputs. Use the command **Insert Input** for adding further inputs. The number is unlimited.

Example in IL

```
LD       7
MUL      2       ,
          4       ,
          7
ST       Var1
```

Example in ST

```
var1 := 7*2*4*7;
```

Examples in FBD

1. series of `MUL` boxes
2. extended `MUL` box
3. `MUL` box with `EN/ENO` parameters

SUB

Overview

IEC operator for the subtraction of one variable from another one.

Allowed types:

- BYTE
- WORD
- DWORD
- LWORD
- SINT
- USINT
- INT
- UINT
- DINT
- UDINT
- LINT
- ULINT
- REAL
- LREAL
- TIME
- TIME_OF_DAY(TOD)
- DATE
- DATE_AND_TIME(DT)

For time data types, the following combinations are possible:

- TIME–TIME=TIME
- DATE–DATE=TIME
- TOD–TIME=TOD
- TOD–TOD=TIME
- DT-TIME=DT
- DT-DT=TIME

Consider that negative TIME values are undefined.

Example in IL

```
LD      7
SUB     2
ST      Var1
```

Example in ST

```
var1 := 7-2;
```

Example in FBD

DIV

Overview

IEC operator for the division of one variable by another one:

Allowed types:

- BYTE
- WORD
- DWORD
- LWORD
- SINT
- USINT
- INT
- UINT
- DINT
- UDINT
- LINT
- ULINT
- REAL
- LREAL
- TIME

TIME variables can be divided by integer variables.

Example in IL

(Result in `Var1` is 4.)

```
LD        8
DIV       2
ST        Var1
```

Example in ST

```
var1  :=  8/2;
```

Examples in FBD

1. series of `DIV` boxes
2. single `DIV` box
3. `DIV` box with `EN/ENO` parameters

Different target systems may behave differently concerning a division by zero error. It can lead to a controller HALT, or may go undetected.

WARNING

UNINTENDED EQUIPMENT OPERATION

Use the check functions described in this document, or write your own checks to avoid division by zero in the programming code.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

NOTE: For more information about the implicit check functions, refer to the chapter POUs for Implicit Checks.

Check Functions

You can use the following check functions to verify the value of the divisor in order to avoid a division by 0 and adapt them, if necessary:

- `CheckDivDInt`
- `CheckDivLint`
- `CheckDivReal`
- `CheckDivLReal`

For information on inserting the function, refer to the description of the **POUs for implicit checks** function.

The check functions are called automatically before each division found in the application code.

See the following example for an implementation of the function `CheckDivReal`.

Default Implementation of the Function `CheckDivReal`

Declaration part

```
// Implicitly generated code : DO NOT EDIT
FUNCTION CheckDivReal : REAL
VAR_INPUT
 divisor:REAL;
END_VAR
```

Implementation part:

```
// Implicitly generated code : only an suggestion for implementation
IF divisor = 0 THEN
 CheckDivReal:=1;
ELSE
 CheckDivReal:=divisor;
END_IF;
```

The operator `DIV` uses the output of function `CheckDivReal` as a divisor. In the following example, a division by 0 is prohibited as with the 0 initialized value of the divisor `d` is changed to 1 by `CheckDivReal` before the division is executed. Therefore, the result of the division is 799.

```
PROGRAM PLC_PRG
VAR
 erg:REAL;
 v1:REAL:=799;
 d:REAL;
END_VAR
erg:= v1 / d;
```

MOD

Overview

IEC operator for the modulo division of one variable by another one.

Allowed types:

- BYTE
- WORD
- DWORD
- LWORD
- SINT
- USINT
- INT
- UINT
- DINT
- UDINT
- LINT
- ULINT

The result of this function is the integer remainder of the division.

Different target systems may behave differently concerning a division by zero error. It can lead to a controller HALT, or may go undetected.

WARNING

UNINTENDED EQUIPMENT OPERATION

Use the check functions described in this document, or write your own checks to avoid division by zero in the programming code.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

NOTE: For more information about the implicit check functions, refer to the chapter POUs for Implicit Checks.

Example in IL

Result in `Var1` is 1.

```
LD      9
MOD     2
ST      Var1
```

Example in ST

```
var1 := 9 MOD 2;
```

Examples in FBD

MOVE

Overview

IEC operator for the assignment of a variable to another variable of an appropriate data type.

The `MOVE` operator is possible for all data types.

As `MOVE` is available as a box in the graphic editors FBD, LD, CFC, there the (unlocking) `EN/ENO` functionality can also be applied on a variable assignment.

Example in CFC in Conjunction with the EN/ENO Function

Only if `en_i` is TRUE, `var1` will be assigned to `var2`.

Example in IL

Result: `var2` gets value of `var1`

```
LD        var1
MOVE
ST        var2
```

You get the same result with

```
LD        var1
ST        var2
```

Example in ST

```
ivar2  :=  MOVE(ivar1);
```

You get the same result with

```
ivar2  :=  ivar1;
```

SIZEOF

Overview

This arithmetic operator is not specified by the standard IEC 61131-3.

You can use it to determine the number of bytes required by the given variable `x`.

The `SIZEOF` operator returns an unsigned value. The type of the return value will be adapted to the found size of variable `x`.

Return Value of SIZEOF(x)	Data Type of the Constant Implicitly Used for the Found Size
0 ≤ size of x < 256	USINT
256 ≤ size of x < 65,536	UINT
65,536 ≤ size of x < 4,294,967,296	UDINT
4,294,967,296 ≤ size of x	ULINT

Example in ST

```
var1 := SIZEOF(arr1);   (* d.h.: var1:=USINT#10; *)
```

Example in IL

Result is 10

```
arr1:ARRAY[0..4] OF INT;
Var1:INT;
```

```
LD      arr1
SIZEOF
ST      Var1
```

Bitstring Operators

Overview

The following bitstring operators are available, matching the IEC1131-3 standard:

- AND
- OR
- XOR
- NOT

The following operators are not specified by the standard and are not available:

- ANDN
- ORN
- XORN

Bitstring operators compare the corresponding bits of 2 or several operands.

What Is in This Section?

This section contains the following topics:

AND

Overview

IEC bitstring operator for bitwise AND of bit operands.

If the input bits each are 1, then the resulting bit will be 1, otherwise 0.

Allowed types:

- BOOL
- BYTE
- WORD
- DWORD
- LWORD

Example in IL

Result in `Var1` is 2#1000_0010.

```
Var1:BYTE;
```

```
LD       2#1001_0011
AND      2#1000_1010
ST       var1
```

Example in ST

```
var1 := 2#1001_0011 AND 2#1000_1010
```

Example in FBD

OR

Overview

IEC bitstring operator for bitwise OR of bit operands.

If at least 1 of the input bits is 1, the resulting bit will be 1, otherwise 0.

Allowed types:

- BOOL
- BYTE
- WORD
- DWORD
- LWORD

Example in IL

Result in `var1` is 2#1001_1011.

```
var1:BYTE;
```

```
LD       2#1001_0011
OR       2#1000_1010
```

```
ST        Var1
```

Example in ST

```
Var1 := 2#1001_0011 OR 2#1000_1010
```

Example in FBD

XOR

Overview

IEC bitstring operator for bitwise XOR of bit operands.

If only 1 of the input bits is 1, then the resulting bit will be 1; if both or none are 1, the resulting bit will be 0.

Allowed types:

- BOOL
- BYTE
- WORD
- DWORD
- LWORD

NOTE: `XOR` allows adding additional inputs. If more than 2 inputs are available, then an XOR operation is performed on the first 2 inputs. The result, in turn, will be XOR combined with input 3, and so on. This has the effect that an odd number of inputs will lead to a resulting bit = 1.

Example in IL

Result is 2#0001_1001.

```
Var1:BYTE;
```

```
LD        2#1001_0011
XOR      2#1000_1010
ST        var1
```

Example in ST

```
Var1 := 2#1001_0011 XOR 2#1000_1010
```

Example in FBD

NOT

Overview

IEC bitstring operator for bitwise NOT operation of a bit operand.

The resulting bit will be 1 if the corresponding input bit is 0 and vice versa.

Allowed types

- BOOL
- BYTE
- WORD
- DWORD
- LWORD

Example in IL

Result in `Var1` is 2#0110_1100.

```
Var1:BYTE;
```

```
LD      2#1001_0011
NOT
ST      var1
```

Example in ST

```
Var1 := NOT 2#1001_0011
```

Example in FBD

Bit-Shift Operators

What Is in This Section?

This section contains the following topics:

SHL

Overview

IEC operator for bitwise left-shift of an operand.

```
erg:= SHL (in, n)
```

`in`: operand to be shifted to the left

`n`: number of bits, by which `in` gets shifted to the left

NOTE: If `n` exceeds the data type width, it depends on the target system how BYTE, WORD, DWORD and LWORD operands will be filled. Some cause filling with zeros (`0`), others with `n MOD <register width>`.

NOTE: The amount of bits which is considered for the arithmetic operation depends on the data type of the input variable. If the input variable is a constant, the smallest possible data type is considered. The data type of the output variable has no effect at all on the arithmetic operation.

Examples

See in the following example in hexadecimal notation the different results for `erg_byte` and `erg_word`. The result depends on the data type of the input variable (BYTE or WORD), although the values of the input variables `in_byte` and `in_word` are the same.

Example in ST

```
PROGRAM shl_st
VAR
  in_byte : BYTE:=16#45; (* 2#01000101 )
  in_word : WORD:=16#0045; (* 2#0000000001000101 )
  erg_byte : BYTE;
  erg_word : WORD;
  n: BYTE :=2;
END_VAR
erg_byte:=SHL(in_byte,n); (* Result is 16#14, 2#00010100 *)
erg_word:=SHL(in_word,n); (* Result is 16#0114, 2#0000000100010100 *)
```

Example in FBD

Example in IL

```
LD       in_byte
SHL      2
ST       erg_byte
```

SHR

Overview

IEC operator for bitwise right-shift of an operand.

```
erg:= SHR (in, n)
```

`in`: operand to be shifted to the right

`n`: number of bits, by which `in` gets shifted to the right

NOTE: If `n` exceeds the data type width, it depends on the target system how BYTE, WORD, DWORD and LWORD operands will be filled. Some cause filling with zeros (`0`), others with `n MOD <register width>`.

Examples

The following example in hexadecimal notation shows the results of the arithmetic operation depending on the type of the input variable (BYTE or WORD).

Example in ST

```
PROGRAM shr_st
VAR
  in_byte : BYTE:=16#45; (* 2#01000101 )
  in_word : WORD:=16#0045; (* 2#0000000001000101 )
  erg_byte : BYTE;
  erg_word : WORD;
  n: BYTE :=2;
END_VAR
erg_byte:=SHR(in_byte,n); (* Result is 16#11, 2#00010001 *)
erg_word:=SHR(in_word,n); (* Result is 16#0011, 2#0000000000010001 *)
```

Example in FBD

Example in IL

```
LD       in_byte
SHR      2
ST       erg_byte
```

ROL

Overview

IEC operator for bitwise rotation of an operand to the left.

```
erg:= ROL (in, n)
```

Allowed data types
- BYTE
- WORD
- DWORD

- LWORD

`in` will be shifted 1 bit position to the left `n` times while the bit that is furthest to the left will be reinserted from the right

NOTE: The amount of bits which is considered for the arithmetic operation depends on the data type of the input variable. If the input variable is a constant, the smallest possible data type is considered. The data type of the output variable has no effect at all on the arithmetic operation.

Examples

See in the following example in hexadecimal notation the different results for `erg_byte` and `erg_word`. The result depends on the data type of the input variable (BYTE or WORD), although the values of the input variables `in_byte` and `in_word` are the same.

Example in ST

```
PROGRAM rol_st
VAR
in_byte : BYTE:=16#45;
in_word : WORD:=16#45;
erg_byte : BYTE;
erg_word : WORD;
n: BYTE :=2;
END_VAR
erg_byte:=ROL(in_byte,n); (* Result is 16#15 *)
erg_word:=ROL(in_word,n); (* Result is 16#0114 *)
```

Example in FBD

Example in IL

```
LD      in_byte
ROL     n
ST      erg_byte
```

ROR

Overview

IEC operator for bitwise rotation of an operand to the right.

```
erg:= ROR (in, n)
```

Allowed data types
- BYTE
- WORD
- DWORD
- LWORD

`in` will be shifted 1 bit position to the right `n` times while the bit that is furthest to the left will be reinserted from the left.

NOTE: The amount of bits which is noticed for the arithmetic operation depends on the data type of the input variable. If the input variable is a constant, the smallest possible data type is noticed. The data type of the output variable has no effect at all on the arithmetic operation.

Examples

See in the following example in hexadecimal notation the different results for `erg_byte` and `erg_word`. The result depends on the data type of the input variable (BYTE or WORD), although the values of the input variables `in_byte` and `in_word` are the same.

Example in ST

```
PROGRAM ror_st
VAR
in_byte : BYTE:=16#45;
in_word : WORD:=16#45;
erg_byte : BYTE;
erg_word : WORD;
n: BYTE :=2;
END_VAR
erg_byte:=ROR(in_byte,n); (* Result is 16#51 *)
erg_word:=ROR(in_word,n); (* Result is 16#4011 *)
```

Example in FBD

Example in IL

```
LD        in_byte
ROR       n
ST        erg_byte
```

Selection Operators

Overview

Selection operations can also be performed with variables.

For purposes of clarity the examples provided in this document are limited to the following which use constants as operators:

- SEL
- MAX
- MIN
- LIMIT
- MUX

What Is in This Section?

This section contains the following topics:

SEL

Overview

IEC selection operator for binary selection.

`G` determines whether IN0 or IN1 is assigned to `OUT`.

`OUT := SEL(G, IN0, IN1)` means:

`OUT := IN0;` if `G`=FALSE

`OUT := IN1;` if `G`=TRUE

Allowed data types:

`IN0, ...,INn` and `OUT` can be any identical data type. Make sure that variables of the identical data type are used at these positions, especially when using user-defined data types. The compiler verifies the identity of the types and returns compiler errors. Assigning function block instances to interface variables is not supported.

`G`: BOOL

Example in IL

```
LD   TRUE
SEL  3,4     (* IN0 = 3, IN1 =4 *)
ST   Var1    (* result is 4 *)
LD   FALSE
SEL  3,4
ST   Var1    (* result is 3 *)
```

Example in ST

```
Var1:=SEL(TRUE,3,4);  (* result is 4 *)
```

Example in FBD

Note

NOTE: An expression occurring ahead of `IN0` will not be processed if `G` is TRUE. An expression occurring ahead of `IN1` will not be processed if `G` is FALSE.

MAX

Overview

IEC selection operator performing a maximum function.

The MAX operator returns the greater of the 2 values.

```
OUT := MAX(IN0, IN1)
```

IN0, IN1 and OUT can be any type of variable.

Example in IL

Result is 90

```
LD      90
MAX     30
MAX     40
MAX     77
ST      Var1
```

Example in ST

```
Var1:=MAX(30,40); (* Result is 40 *)
Var1:=MAX(40,MAX(90,30)); (* Result is 90 *)
```

Example in FBD

MIN

Overview

IEC selection operator performing a minimum function.

The MIN operator returns the lesser of the 2 values.

```
OUT := MIN(IN0, IN1)
```

IN0, IN1 and OUT can be any type of variable.

Example in IL

Result is 30

```
LD       90
MIN      30
MIN      40
MIN      77
ST       Var1
```

Example in ST

```
Var1:=MIN(90,30);  (* Result is 30 *);
Var1:=MIN(MIN(90,30),40);  (* Result is 30 *);
```

Example in FBD

LIMIT

Overview

IEC selection operator performing a limiting function.

```
OUT := LIMIT(Min, IN, Max) means:
OUT := MIN (MAX (IN, Min), Max)
```

`Max` is the upper and `Min` the lower limit for the result. Should the value `IN` exceed the upper limit `Max`, `LIMIT` will return `Max`. Should `IN` fall below `Min`, the result will be `Min`.

`IN` and `OUT` can be any type of variable.

Example in IL

Result is 80

```
LD       90
LIMIT   30       ,
          80
ST       Var1
```

Example in ST

```
Var1:=LIMIT(30,90,80);  (*  Result  is  80  *);
```

MUX

Overview

IEC selection operator for multiplexing operation.

`OUT := MUX(K, IN0,...,INn))` means:

```
OUT := INk
```

`IN0, ...,INn` and `OUT` can be any identical data type. Make sure that variables of the identical data type are used at these positions, especially when using user-defined data types. The compiler verifies the identity of the types and returns compiler errors. Assigning function block instances to interface variables is not supported.

`K` has to be BYTE, WORD, DWORD, LWORD, SINT, USINT, INT, UINT, DINT, LINT, ULINT or UDINT.

`MUX` selects the K^{th} value from among a group of values.

Example in IL

Result is 30

```
LD        0
MUX      30        ,
          40        ,
          50        ,
          60        ,
          70        ,
          80
ST        Var1
```

Example in ST

```
Var1:=MUX(0,30,40,50,60,70,80);  (*  Result  is  30  *);
```

NOTE: An expression occurring ahead of an input other than `INk` will not be processed to save run time. Only in simulation mode will all expressions be executed.

Comparison Operators

Overview

The following operators matching the IEC1131-3 standard are available:

- GT
- LT
- LE
- GE
- EQ
- NE

What Is in This Section?

This section contains the following topics:

GT

Overview

Comparison operator performing a Greater Than function.

The GT operator is a boolean operator which returns the value TRUE when the value of the first operand is greater than that of the second.

The operands can be of any basic data type.

Example in IL

Result is FALSE

```
LD        20
GT        30
ST        Var1
```

Example in ST

```
VAR1  :=  20  >  30;
```

Example in FBD

LT

Overview

Comparison operator performing a Less Than function.

The LT operator is a boolean operator which returns the value TRUE when the value of the first operand is less than that of the second.

The operands can be of any basic data type.

Example in IL

Result is TRUE

```
LD        20
LT        30
ST        Var1
```

Example in ST

```
VAR1 := 20 < 30;
```

Example in FBD

LE

Overview

Comparison operator performing a Less Than Or Equal To function.

The LE operator is a boolean operator which returns the value TRUE when the value of the first operand is less than or equal to that of the second.

The operands can be of any basic data type.

Example in IL

Result is TRUE

```
LD      20
LE      30
ST      Var1
```

Example in ST

```
VAR1 := 20 <= 30;
```

Example in FBD

GE

Overview

Comparison operator performing a Greater Than Or Equal To function.

The GE operator is a boolean operator which returns the value TRUE when the value of the first operand is greater than or equal to that of the second.

The operands can be of any basic data type.

Example in IL

Result is TRUE

```
LD        60
GE        40
ST        Var1
```

Example in ST

```
VAR1  :=  60  >=  40;
```

Example in FBD

EQ

Overview

Comparison operator performing an Equal To function.

The EQ operator is a boolean operator which returns the value TRUE when the operands are equal.

The operands can be of any basic data type.

Example in IL

Result is TRUE

```
LD        40
EQ        40
ST        Var1
```

Example in ST

```
VAR1 := 40 = 40;
```

Example in FBD

NE

Overview

Comparison operator performing a Not Equal To function.

The NE operator is a boolean operator which returns the value TRUE when the operands are not equal.

The operands can be of any basic data type.

Example in IL

```
LD      40
NE      40
ST      Var1
```

Example in ST

```
VAR1 := 40 <> 40;
```

Example in FBD

Address Operators

What Is in This Section?

This section contains the following topics:

- ADR
- Content_Operator
- BITADR

ADR

Overview

This address operator is not specified by the standard IEC 61131-3.

`ADR` returns the address of its argument in a DWORD. This address can be assigned to a pointer within the project.

NOTE: EcoStruxure Machine Expert allows you to use the `ADR` operator with function names, program names, function block names, and method names.

Refer to the chapter Pointers and consider that function pointers can be passed to external libraries. Nevertheless, there is no possibility to call a function pointer within EcoStruxure Machine Expert. In order to enable a system call (runtime system), set the respective object property (in the menu **View > Properties... > Build**) for the function object.

Example in ST

```
dwVar:=ADR(bVAR);
```

Example in IL

```
LD      bVar
ADR
ST      dwVar
```

Considerations for Online Changes

Executing the command **Online Change** can move variables to another place in the memory. There is an indication during online change if copying is necessary.

The shift of variables may have the effect that `POINTER` variables point to invalid memory. So, ensure that a pointer is not kept between cycles, but is reassigned in each cycle.

WARNING

UNINTENDED EQUIPMENT OPERATION

Assign the value of any `POINTER TO` type variable(s) prior to the first use of it within a POU, and at every subsequent cycle.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

NOTE: `POINTER TO` variables of functions and methods should not be returned to the caller of this function or passed to global variables.

Content Operator

Overview

This address operator is not specified by the standard IEC 61131-3. You can dereference a pointer by adding the content operator ^ (ASCII caret or circumflex symbol) after the pointer identifier.

Example in ST

```
pt:POINTER TO INT;
var_int1:INT;
var_int2:INT;
pt := ADR(var_int1);
var_int2:=pt^;
```

Considerations for Online Changes

Executing the **Online Change** command can change the contents of addresses.

CAUTION

INVALID POINTER

Verify the validity of the pointers when using pointers on addresses and executing the Online Change command.

Failure to follow these instructions can result in injury or equipment damage.

BITADR

Overview

This address operator is not specified by the standard IEC 61131-3.

`BITADR` returns the bit offset within the segment in a DWORD. The offset value depends on whether the option **Byte addressing** in the target settings is activated or not.

The highest nibble in that DWORD indicates the memory area:

Memory: 16x40000000

Input: 16x80000000

Output: 16xC0000000

Example in ST

```
VAR
           var1 AT %IX2.3:BOOL;
           bitoffset: DWORD;
END_VAR
bitoffset:=BITADR(var1); (* Result if byte addressing=TRUE: 16x80000013, if byte addressing=FALSE: 16x8
```

Example in IL

```
LD       Var1
BITADR
ST       bitoffset
```

Considerations for Online Changes

Executing the **Online Change** command can change the contents of addresses.

CAUTION

INVALID POINTER

Verify the validity of the pointers when using pointers on addresses and executing the Online Change command.

Failure to follow these instructions can result in injury or equipment damage.

Calling Operator

CAL

Overview

IEC operator for calling a function block or a program.

Use `CAL` in IL to call up a function block instance. Place the variables that will serve as the input variables in parentheses right after the name of the function block instance.

Example

Calling up the instance `Inst` of a function block where input variables `Par1` and `Par2` are 0 and TRUE, respectively.

```
CAL INST(PAR1 := 0, PAR2 := TRUE)
```

Type Conversion Operators

What Is in This Section?

This section contains the following topics:

- Type Conversion Functions
- BOOL_TO Conversions
- TO_BOOL Conversions
- Conversion Between Integral Number Types

- REAL_TO_/_LREAL_TO_Conversions
- TIME_TO/TIME_OF_DAY_Conversions
- DATE_TO/DT_TO_Conversions
- STRING_TO_Conversions
- TRUNC
- TRUNC_INT
- ANY_..._TO_Conversions
- TO_<xxx>_Conversions

Type Conversion Functions

Overview

It is not allowed to convert implicitly from a larger type to a smaller type (for example, from INT to BYTE or from DINT to WORD). To achieve this, you have to perform special type conversions. You can basically convert from any elementary type to any other elementary type.

Syntax

Typed conversion: `<elem.type1>_TO_<elem.type2>`

Overloaded conversion: `TO_<elem.type2>`

NOTE: At ...TO_STRING conversions the string is generated as left-justified. If it is defined too short, it will be cut from the right side.

The following type conversions are supported:

- BOOL_TO_conversions
- TO_BOOL_conversions
- conversion_between_integral_number_types
- REAL_TO-/_LREAL_TO_conversions
- TIME_TO/TIME_OF_DAY_conversions
- DATE_TO/DT_TO_conversions
- STRING_TO_conversions

- TRUNC (conversion to DINT)
- TRUNC_INT
- ANY_NUM_TO_<numeric data type>

BOOL_TO Conversions

Definition

IEC operator for conversions from type BOOL to any other type.

Syntax

BOOL_TO_<data type>

Conversion Results

The conversion results for number types and for string types depend on the state of the operand:

Operand State	Result for Number Types	Result for String Types
TRUE	1	TRUE
FALSE	0	FALSE

Examples in ST

Examples in ST with conversion results:

Example	Result
`i:=BOOL_TO_INT(TRUE);`	1
`str:=BOOL_TO_STRING(TRUE);`	'TRUE'
`t:=BOOL_TO_TIME(TRUE);`	T#1ms
`tof:=BOOL_TO_TOD(TRUE);`	TOD#00:00:00.001
`dat:=BOOL_TO_DATE(FALSE);`	D#1970

`dandt:=BOOL_TO_DT(TRUE);`	`DT#1970-01-01-00:00:01`

Examples in IL

Examples in IL with conversion results:

Example	Result

`LD TRUE`
`BOOL_TO_INT`
`ST i`
	`1`

`LD TRUE`
`BOOL_TO_STRI...`
`ST str`
	`'TRUE'`

`LD TRUE`
`BOOL_TO_TIME`
`ST t`
	`T#1ms`

`LD TRUE`
`BOOL_TO_TOD`
`ST tof`
	`TOD#00:00:00.001`

`LD FALSE`
`BOOL_TO_DATE`
`ST dandt`
	`D#1970-01-01`

`LD TRUE`
`BOOL_TO_DT`
`ST dandt`
	`DT#1970-01-01-00:00:01`

Examples in FBD

Examples in FBD with conversion results:

Example	Result
	`1`

	'TRUE'
	T#1ms
	TOD#00:00:00.001
	D#1970-01-01
	DT#1970-01-01-00:00:01

TO_BOOL Conversions

Definition

IEC operator for conversions from another variable type to BOOL.

Syntax

<data type>_TO_BOOL

Conversion Results

The result is TRUE when the operand is not equal to 0. The result is FALSE when the operand is equal to 0. The result is TRUE for STRING type variables when the operand is TRUE. Otherwise the result is FALSE.

Examples in ST

Examples in ST with conversion results:

Example	Result
`b := BYTE_TO_BOOL(2#11010101);`	TRUE
`b := INT_TO_BOOL(0);`	FALSE
`b := TIME_TO_BOOL(T#5ms);`	TRUE
`b := STRING_TO_BOOL('TRUE');`	TRUE

Examples in IL

Examples in IL with conversion results:

Example	Result

`LD 213`
`BYTE_TO_BOOL`
`ST b`
	TRUE

`LD 0`
`INT_TO_BOOL`
`ST b`
	FALSE

`LD T#5ms`
`TIME_TO_BOOL`
`ST b`
	TRUE

`LD 'TRUE'`
`STRING_TO_BOOL`
`ST b`
	TRUE

Examples in FBD

Examples in FBD with conversion results:

Example	Result
	TRUE
	FALSE
	TRUE
	TRUE

Conversion Between Integral Number Types

Definition

Conversion from an integral number type to another number type.

Syntax

<INT data type>_TO_<INT data type>

For information on the integer data type, refer to the chapter Standard Data Types.

Conversion Results

If the number you are converting exceeds the range limit, the first bytes for the number will be ignored.

Example in ST

```
si := INT_TO_SINT(4223); (* Result is 127 *)
```

If you save the integer 4223 (16#107f represented hexadecimally) as a SINT variable, it will appear as 127 (16#7f represented hexadecimally).

Example in IL

```
LD                      4223
INT_TO_SINT
ST                      si
```

Example in FBD

REAL_TO / LREAL_TO Conversions

Definition

IEC operator for conversions from the variable type REAL or LREAL to a different type.

The value will be rounded up or down to the nearest whole number and converted into the new variable type.

Exceptions to this are the following variable types:
- STRING
- BOOL
- REAL
- LREAL

Conversion Results

If a REAL or LREAL is converted to SINT, USINT, INT, UINT, DINT, UDINT, LINT or ULINT and the value of the real number is out of the value range of that integer, the result will be undefined, and may lead to a controller exception.

NOTE: Validate any range overflows by your application and verify that the value of the REAL or LREAL is within the bounds of the target integer before performing the conversion.

When converting to type STRING, consider that the total number of digits is limited to 16. If the (L)REAL number has more digits, then the sixteenth will be rounded. If the length of the STRING is defined too short, it will be cut from the right end.

Example in ST

Examples in ST with conversion results:

Example	Result
`i := REAL_TO_INT(1.5);`	2
`j := REAL_TO_INT(1.4);`	1
`i := REAL_TO_INT(-1.5);`	-2
`j := REAL_TO_INT(-1.4);`	-1

Example in IL

```
LD                          2.75
REAL_TO_INT
ST                          i
```

Example in FBD

TIME_TO/TIME_OF_DAY Conversions

Definition

IEC operator for conversions from the variable type TIME or TIME_OF_DAY to a different type.

Syntax

TIME_TO_<data type>

TOD_TO_<data type>

Conversion Results

The time will be stored internally in a DWORD in milliseconds (beginning with 12:00 A.M. for the TIME_OF_DAY variable). This value will then be converted.

In case of type STRING the result is a time constant.

Examples in ST

Examples in ST with conversion results:

Example	Result
`str := TIME_TO_STRING(T#12ms);`	`'T#12ms'`
`dw := TIME_TO_DWORD(T#5m);`	`300000`
`si := TOD_TO_SINT(TOD#00:00:00.012);`	`12`

Examples in IL

Examples in IL with conversion results:

Example	Result

`LD T#12ms`
`TIME_TO_STRI...`
`ST str`
	`'T#12ms'`

`LD T#300000ms`
`TIME_TO_DWORD`
`ST dw`
	`300000`

`LD TOD#00:00:00.012`
`TIME_TO_SINT`
`ST si`
	`12`

Examples in FBD

DATE_TO/DT_TO Conversions

Definition

IEC operator for conversions from the variable type DATE or DATE_AND_TIME to a different type.

Syntax

DATE_TO_<data type>

DT_TO_<data type>

Conversion Results

The date will be stored internally in a DWORD in seconds since Jan. 1, 1970. This value will then be converted. For STRING type variables, the result is the date constant.

Examples in ST

Examples in ST with conversion results:

Example	Result
`b := DATE_TO_BOOL(D#1970-01-01);`	`FALSE`
`i := DATE_TO_INT(D#1970-01-15);`	`29952`
`byt := DT_TO_BYTE(DT#1970-01-15-05:05:05);`	`129`
`str := DT_TO_STRING(DT#1998-02-13-14:20);`	`'DT#1998-02-13-14:20'`

Examples in IL

Examples in IL with conversion results:

Example	Result

`LD D#1970-01-01`
`DATE_TO_BOOL`
`ST b`
	`FALSE`

`LD D#1970-01-01`
`DATE_TO_INT`
`ST i`
	`29952`

`LD D#1970-01-15-05:05:`
`DATE_TO_BYTE`
`ST byt`
	`129`

`LD D#1998-02-13-14:20`
`DATE_TO_STRI...`
`ST str`
	`'DT#1998-02-13-14:20'`

Examples in FBD

STRING_TO Conversions

Definition

IEC operator for conversions from the variable type STRING to a different type.

Syntax

STRING_TO_<data type>

Specifying Values

Specify the operand of type STRING matching the IEC61131-3 standard. The value must correspond to a valid constant (literal) of the target type. This applies to the specification of exponential values, infinite values, prefixes, grouping character ("_") and comma. Additional characters after the digits of a number are allowed, as for example, `23xy`. Characters preceding a number are not allowed.

The operand must represent a valid value of the target data type.

NOTE: If the data type of the operand does not match the target type, or if the value exceeds the range of the target data type, then the result depends on the processor type and is therefore undefined.
Conversions from larger types to smaller types may result in loss of information.

CAUTION

LOSS OF DATA

When converting mismatched data types or when the value being converted is larger than the target data type, be sure that the result is validated within your application.

Failure to follow these instructions can result in injury or equipment damage.

Example in IL

Example	Conversion result

`LD 'TRUE'`
`STRING_TO_BOOL`
`ST b`
	`TRUE`

Examples in ST

Example	Conversion result
`b := STRING_TO_BOOL('TRUE');`	`TRUE`

`w := STRING_TO_WORD('abc34');`	0
`w := STRING_TO_WORD('34abc');`	34
`t := STRING_TO_TIME('T#127ms');`	T#127ms
`r := STRING_TO_REAL('1.234');`	1.234
`bv := STRING_TO_BYTE('500');`	244

Example in FBD

Example	Conversion result
	TRUE

TRUNC

Definition

IEC operator for conversions from REAL to DINT. The whole number portion of the value will be used.

The result of these functions is not defined if the input value cannot be represented with a DINT or INT. The behavior of such input values is platform-dependent.

Examples in ST

Examples in ST with conversion results:

Example	Result
`diVar:=TRUNC(1.9);`	1
`diVar:=TRUNC(-1.4);`	-1

Example in IL

```
LD                          1.9
TRUNC
ST                          diVar
```

TRUNC_INT

Definition

IEC operator for conversions from REAL to INT. The whole number portion of the value will be used.

The result of these functions is not defined if the input value cannot be represented with a DINT or INT. The behavior of such input values is platform-dependent.

Examples in ST

Examples in ST with conversion results:

Example	Result
`iVar:=TRUNC_INT(1.9);`	1
`iVar:=TRUNC_INT(-1.4);`	-1

Example in IL

```
LD                          1.9
TRUNC_INT
ST                          iVar
```

ANY_..._TO Conversions

Definition

Conversion from any data type, or more specifically from any numeric data type to another data type. As with any type of conversion, the size of the operands must be taken into account in order to have a successful conversion.

Syntax

ANY_NUM_TO_<numeric data type>

ANY_TO_<any data type>

Example

Conversion from a variable of data type REAL to INT:

```
re : REAL := 1.234;
i : INT := ANY_TO_INT(re)
```

TO_<xxx> Conversions

Definition

Conversion of variables from one type to another type. The input type must not be specified explicitly (overloaded conversion). Also refer to the description of type conversion functions.

Syntax

```
TO_<data type>
```

Example in ST

Conversion from a variable of data type REAL to INT:

```
VAR
    iVar: INT;
    bVar: BOOL;
    strVar: STRING;
                rVar: REAL;
END_VAR
```

```
wVar:=TO_WORD('123')       (* result is 123 *)
bVar:=TO_BOOL(1);           (* result is TRUE *)
strVar:=TO_STRING(342);   (* result is '342' *)
iVar_=TO_INT(4.22);         (* result is 4 *)
```

Numeric Functions

Overview

This chapter describes the available numeric IEC operators specified by the standard IEC 61131-3.

What Is in This Section?

This section contains the following topics:

ABS

Definition

Numeric IEC operator for returning the absolute value of a number.

In- and output can be of any numeric basic data type.

Example in IL

The result in `i` is 2.

```
LD                          -2
ABS
ST                          i
```

Example in ST

```
i:=ABS(-2);
```

Example in FBD

SQRT

Definition

Numeric IEC operator for returning the square root of a number.

The input variable can be of any numeric basic data type, the output variable has to be type REAL or LREAL.

Example in IL

The result in `q` is 4.

```
LD                          16
SQRT
ST                          q
```

Example in ST

```
q:=SQRT(16);
```

Example in FBD

LN

Definition

Numeric IEC operator for returning the natural logarithm of a number.

The input variable can be of any numeric basic data type, the output variable has to be type REAL or LREAL.

Example in IL

The result in `q` is 3.80666.

```
LD                      45
LN
ST                      q
```

Example in ST

```
q:=LN(45);
```

Example in FBD

LOG

Definition

Numeric IEC operator for returning the logarithm of a number in base 10.

The input variable can be of any numeric basic data type, the output variable has to be type REAL or LREAL.

Example in IL

The result in `q` is 2.49762.

```
LD                          314.5
LOG
ST                          q
```

Example in ST

```
q:=LOG(314.5);
```

Example in FBD

EXP

Definition

Numeric IEC operator for returning the exponential function.

The input variable can be of any numeric basic data type, the output variable has to be type REAL or LREAL.

Example in IL

The result in `q` is 7.389056099.

```
LD                          2
EXP
ST                          q
```

Example in ST

```
q:=EXP(2);
```

Example in FBD

SIN

Definition

Numeric IEC operator for returning the sine of an angle.

The input defining the angle in radians can be of any numeric basic data type, the output variable has to be of type REAL or LREAL.

Example in IL

The result in `q` is 0.479426.

```
LD                      0.5
SIN
ST                      q
```

Example in ST

```
q:=SIN(0.5);
```

Example in FBD

COS

Definition

Numeric IEC operator for returning the cosine of an angle.

The input defining the angle in arch minutes can be of any numeric basic data type where the output variable has to be of type REAL or LREAL.

Example in IL

The result in `q` is 0.877583.

```
LD                          0.5
COS
ST                          q
```

Example in ST

```
q:=COS(0.5);
```

Example in FBD

TAN

Definition

Numeric IEC operator for returning the tangent of a number. The value is calculated in arch minutes.

The input variable can be of any numeric basic data type where the output variable has to be type REAL or LREAL.

Example in IL

The result in `q` is 0.546302.

```
LD                          0.5
TAN
ST                          q
```

Example in ST

```
q:=TAN(0.5);
```

Example in FBD

ASIN

Definition

Numeric IEC operator for returning the arc sine (inverse function of sine) of a number. The value is calculated in arch minutes.

The input variable can be of any numeric basic data type where the output variable has to be type REAL or LREAL.

Example in IL

The result in `q` is 0.523599.

```
LD                      0.5
ASIN
ST                      q
```

Example in ST

```
q:=ASIN(0.5);
```

Example in FBD

ACOS

Definition

Numeric IEC operator for returning the arc cosine (inverse function of cosine) of a number. The value is calculated in arch minutes.

The input variable can be of any numeric basic data type where the output variable has to be type REAL or LREAL.

Example in IL

The result in `q` is 1.0472.

```
LD                          0.5
ACOS
ST                          q
```

Example in ST

```
q:=ACOS(0.5);
```

Example in FBD

ATAN

Definition

Numeric IEC operator for returning the arc tangent (inverse function of tangent) of a number. The value is calculated in arch minutes.

The input variable can be of any numeric basic data type where the output variable has to be type REAL or LREAL.

Example in IL

The result in `q` is 0.463648.

```
LD                          0.5
ATAN
ST                          q
```

Example in ST

```
q:=ATAN(0.5);
```

Example in FBD

EXPT

Definition

Numeric IEC operator for exponentiation of a variable with another variable:

`OUT = IN1` to the `IN2`

The input variable can be of numeric basic data types (SINT, USINT, INT, UINT, DINT, UDINT, LINT, ULINT, REAL, LREAL, BYTE, WORD, DWORD, and LWORD) where the output variable has to be type REAL or LREAL.

The result of this function is not defined if the following applies:
- The base is negative.
- The base is zero and the exponent is ≤ 0.

The behavior for such input values is platform-dependent.

Example in IL

The result is 49.

```
LD                          7
EXPT                       2
ST                          Var1
```

Example in ST

```
var1:=EXPT(7,2);
```

Example in FBD

IEC Extending Operators

What Is in This Section?

This section contains the following topics:

IEC Extending Operators

Overview

In addition to the IEC operators, EcoStruxure Machine Expert supports following IEC extending operators:

__DELETE

Definition

This operator is not specified by the IEC 61131-3 standard.

For compatibility information, refer to the EcoStruxure Machine Expert/CoDeSys compiler version mapping table in the *Compatibility and Migration User Guide*.

The __DELETE operator deallocates the memory for objects allocated before via the __NEW operator.

__DELETE has no return value and its operand will be set to 0 after the operation.

Activate the option **Use dynamic memory allocation** in the **Application build options** *view* (**View > Properties... > Application build options**). Consult the *Programming Guide* specific to your controller for whether the options are available on your controller.

Syntax

__DELETE (<pointer>)

If pointer is a pointer to a function block, the dedicated method FB_Exit will be called before the pointer is set to NULL.

NOTE: Use the exact data type of the derived function block and not that of the base function block. Do not use a variable of type POINTER TO BaseFB. This is necessary because if the base function block implements no FB_Exit function, then at the later usage of __DELETE(pBaseFB), no FB_Exit is called.

Example

In the following example, the function block FBDynamic is allocated dynamically via _NEW from the POU PLC_PRG. By doing so, FB_Init method will be called, in which a type DUT is allocated. When __DELETE is called on, the function block pointer from PLC_PRG, FB_Exit will be called, which in turn frees the allocated internal type.

```
FUNCTION_BLOCK FBDynamic
VAR_INPUT
in1, in2 : INT;
END_VAR
VAR_OUTPUT
out : INT;
END_VAR
VAR
test1 : INT := 1234;
_inc : INT := 0;
_dut : POINTER TO DUT;
END_VAR
out := in1 + in2;
```

```
METHOD FB_Exit : BOOL
VAR_INPUT
bInCopyCode : BOOL;
END_VAR
__Delete(_dut);
```

```
METHOD FB_Init : BOOL
VAR_INPUT
bInitRetains : BOOL;
bInCopyCode : BOOL;
END_VAR
_dut := __NEW(DUT);
```

```
METHOD INC : INT
VAR_INPUT
END_VAR
_inc := _inc + 1;
INC := _inc;
```

```
PLC_PRG(PRG)
VAR
pFB : POINTER TO FBDynamic;
bInit: BOOL := TRUE;
bDelete: BOOL;
loc : INT;
END_VAR
IF (bInit) THEN
pFB := __NEW(FBDynamic);
bInit := FALSE;
END_IF
IF (pFB <> 0) THEN
pFB^(in1 := 1, in2 := loc, out => loc);
pFB^.INC();
END_IF
```

```
IF (bDelete) THEN
__DELETE(pFB);
END_IF
```

__ISVALIDREF

Definition

This operator is not specified by the IEC 61131-3 standard.

It allows you to check whether a reference points to a valid value.

For how to use and an example, see the description of the reference data type.

__NEW

Definition

This operator is not prescribed by the IEC 61131-3 standard.

The operator `__NEW` allocates memory for function block instances or arrays of standard data types. The operator returns a suitably typed pointer to the object. If the operator is not used within an assignment, a message is displayed.

Activate the option **Use dynamic memory allocation** in the **Application build options** *view* (**View > Properties... > Application build options**) to use the `__NEW` operator. Consult the *Programming Guide* specific to your controller for whether the options are available on your controller.

If no memory could be allocated, by `__NEW` will return 0.

For deallocating, use `__DELETE`.

Syntax

__NEW (<type>, [<size>]

The operator creates a new object of the specified type `<type>` and returns a pointer to that `<type>`. The initialization of the object is called after creation. If 0 is returned, the operation has not been completed successfully.

NOTE: Use the exact data type of the derived function block and not that of the base function block. Do not use a variable of type `POINTER TO BaseFB`. This is necessary because if the base function block implements no `FB_Exit` function, then at the later usage of `__DELETE(pBaseFB)`, no `FB_Exit` is called.

If `<type>` is scalar, the optional operand `<length>` has to be set additionally and the operator creates an array of scalar types with the size length.

Example

```
pScalarType := __New(ScalarType, length);
```

NOTE: A function block created with `__NEW` has a fixed memory area. The data layout cannot be modified by an online change. Therefore, new variables cannot be added or deleted, and types cannot be modified.

Therefore, only function blocks out of libraries (because they cannot change) and function blocks with attribute `enable_dynamic_creation` are allowed for the `__NEW` operator. If a function block changes with this flag so that copy code will be necessary, a message is produced.

NOTE: The code for memory allocation needs to be non-re-entrant.

A semaphore (`SysSemEnter`) is used to avoid 2 tasks try to allocate memory at the same time. Thus, extensive usage of `__New` may produce higher jitter.

Example with a scalar type:

```
TYPE DUT :
     STRUCT
          a,b,c,d,e,f : INT;
     END_STRUCT
END_TYPE
PROGRAM PLC_PRG
VAR
     pDut : POINTER TO DUT;
     bInit: BOOL := TRUE;
     bDelete: BOOL;
END_VAR
IF (bInit) THEN
     pDut := __NEW(DUT);
     bInit := FALSE;
END_IF
IF (bDelete) THEN
```

```
    __DELETE(pDut);
END_IF
```

Example with a function block:

```
{attribute 'enable_dynamic_creation'}
FUNCTION_BLOCK FBDynamic
VAR_INPUT
    in1, in2 : INT;
END_VAR
VAR_OUTPUT
    out : INT;
END_VAR
VAR
    test1 : INT := 1234;
    _inc : INT := 0;
    _dut : POINTER TO DUT;
END_VAR
out := in1 + in2;
```

```
PROGRAM_PLC_PRG
VAR
    pFB : POINTER TO FBDynamic;
    loc : INT;
    bInit: BOOL := TRUE;
    bDelete: BOOL;
END_VAR
```

```
IF (bInit) THEN
    pFB := __NEW(FBDynamic);
    bInit := FALSE;
END_IF
IF (pFB <> 0) THEN
    pFB^(in1 := 1, in2 := loc, out => loc);
    pFB^.INC();
END_IF
IF (bDelete) THEN
    __DELETE(pFB);
END_IF
```

Example with an array:

```
PLC_PRG(PRG)
VAR
      bInit: BOOL := TRUE;
      bDelete: BOOL;
      pArrayBytes : POINTER TO BYTE;
      test: INT;
      parr : POINTER TO BYTE;
END_VAR
IF (bInit) THEN
      pArrayBytes := __NEW(BYTE, 25);
      bInit := FALSE;
END_IF
IF (pArrayBytes <> 0) THEN
      pArrayBytes[24] := 125;
      test := pArrayBytes[24];
END_IF
IF (bDelete) THEN
      __DELETE(pArrayBytes);
END_IF
```

__QUERYINTERFACE

Definition

This operator is not prescribed by the IEC 61131-3 standard.

At runtime, `__QUERYINTERFACE` is enabling a type conversion of an interface reference to another. The operator returns a result with type BOOL. TRUE implies, that the conversion is successfully executed.

Syntax

__QUERYINTERFACE(<ITF_Source>, < ITF_Dest>

The operator needs as the first operand an interface reference or a function block instance of the intended type and as second operand an interface reference. After execution of `__QUERYINTERFACE`, the `ITF_Dest` holds a reference to the intended interface if the object referenced from `ITF` source implements the interface. In this case, the conversion is successful and the result of the operator returns TRUE. In all other cases, the operator returns FALSE.

A precondition for an explicit conversion is that not only the `ITF_Source` but also `ITF_Dest` is an extension of the interface `__System.IQueryInterface`. This interface is provided implicitly and needs no library.

Example

Example in ST:

```
INTERFACE ItfBase EXTENDS __System.IQueryInterface
METHOD mbase : BOOL
END_METHOD
INTERFACE ItfDerived1 EXTENDS ItfBase
METHOD mderived1 : BOOL
END_METHOD
INTERFACE ItfDerived2 EXTENDS ItfBase
METHOD mderived2 : BOOL
END_METHOD
FUNCTION_BLOCK FB1 IMPLEMENTS ItfDerived1
METHOD mbase : BOOL
     mbase := TRUE;
END_METHOD
METHOD mderived1 : BOOL
     mderived1 := TRUE;
END_METHOD
END_FUNCTION_BLOCK
FUNCTION_BLOCK FB2 IMPLEMENTS ItfDerived2
METHOD mbase : BOOL
     mbase := FALSE;
END_METHOD
METHOD mderived2 : BOOL
     mderived2 := TRUE;
END_METHOD
END_FUNCTION_BLOCK
PROGRAM POU
VAR
     inst1 : FB1;
     inst2 : FB2;
     itfbase1 : ItfBase := inst1;
     itfbase2 : ItfBase := inst2;
     itfderived1 : ItfDerived1 := 0;
```

```
     itfderived2 : ItfDerived2 := 0;
     bTest1, bTest2, xResult1, xResult2: BOOL;
END_VAR
xResult1 := __QUERYINTERFACE(itfbase1, itfderived1); // xResult = TRUE, itfderived1 <> 0
                                                     // references the instance inst1
xResult2 := __QUERYINTERFACE(itfbase1, itfderived2); // xResult = FALSE, itfderived2 = 0
xResult3 := __QUERYINTERFACE(itfbase2, itfderived1); // xResult = FALSE, itfderived1 = 0
xResult4 := __QUERYINTERFACE(itfbase2, itfderived2); // xResult = TRUE, itfderived2 <> 0
                                                     // references the instance inst2
```

__QUERYPOINTER

Definition

This operator is not specified by the IEC 61131-3 standard.

At runtime, `__QUERYPOINTER` is assigning an interface reference to an untyped pointer. The operator returns a result with type BOOL. TRUE implies, that the conversion has been successfully executed.

Syntax

__QUERYPOINTER (<ITF_Source>, < Pointer_Dest>

For the first operand, the operator requires an interface reference or a function block instance of the intended type and for the second operand an untyped pointer. After execution of `__QUERYPOINTER`, the `Pointer_Dest` holds the address of the reference to the intended interface. In this case, the conversion is successful and the result of the operator returns TRUE. In all other cases, the operator returns FALSE. `Pointer_Dest` is untyped and can be cast to any type. The programmer has to ensure the actual type. For example, the interface could provide a method returning a type code.

A precondition for an explicit conversion is that the `ITF_Source` is an extension of the interface `__System.IQueryInterface`. This interface is provided implicitly and needs no library.

Example

```
TYPE KindOfFB
     (FB1 := 1, FB2 := 2, UNKOWN := -1);
END_TYPE
INTERFACE Itf EXTENDS __System.IQueryInterface
METHOD KindOf : KindOfFB
```

```
END_METHOD
FUNCTION_BLOCK F_BLOCK_1 IMPLEMENTS ITF
METHOD KindOf : KindOfFB
     KindOf := KindOfFB.FB1;
END_METHOD
FUNCTION_BLOCK F_BLOCK_2 IMPLEMENTS ITF
METHOD KindOf : KindOfFB
     KindOf := KindOfFB.FB2;
END_METHOD
FUNCTION CAST_TO_ANY_FB : BOOL
VAR_INPUT
     itf_in : Itf;
END_VAR
VAR_OUTPUT
     pfb_1: POINTER TO F_BLOCK_1 := 0;
     pfb_2: POINTER TO F_BLOCK_2 := 0;
END_VAR
VAR
     xResult1, xResult2 : BOOL;
END_VAR
IF itf_in <> 0
     CASE itf_in.KindOf OF
          KindOfFB.FB1:
               xResult1 := __QUERYPOINTER(itf_in, pfb_1);
          KindOfFB.FB2 THEN
               xResult2 := __QUERYPOINTER(itf_in, pfb_2);
     END_CASE
END_IF
CAST_TO_ANY_FB := xResult1 OR xResult2;
```

AND_THEN

Definition

This operator is not specified by the IEC 61131-3 standard. It is only allowed for programming in structured text (ST).

The AND_THEN performs an AND-operation of operands of type BOOL and BIT with short-circuiting mode. This has the following effect:

If all operands are TRUE, then the result of the operation is TRUE, otherwise it is FALSE.

When one operand is FALSE, then the expressions at the other operands are not evaluated (lazy evaluation). In this regard, the AND_THEN operator differs from the AND operator as defined in the standard IEC-61131-3. AND always evaluates all expressions.

In contrast to this, when using the standard IEC-operator AND, always all operands are evaluated.

OR_ELSE

Definition

This operator is not specified by the IEC 61131-3 standard. It is only allowed for programming in structured text (ST).

The OR_ELSE performs an OR-operation of operands of type BOOL and BIT with short-circuiting mode. This has the following effect:

If at least one of the operands is TRUE, then the result of the operation is TRUE, otherwise it is FALSE.

When one operand is TRUE, then the expressions at the other operands are not evaluated (lazy evaluation). In this regard, the OR_ELSE operator differs from the OR operator as defined in the standard IEC-61131-3. OR always evaluates all expressions.

Example

```
 VAR
   bEver:  BOOL;
       bX1:  BOOL;
   dw:  DWORD  :=  16#000000FF;
END_VAR
bEver  :=  FALSE;
bX  :=  dw.8  OR_ELSE  dw.1  OR_ELSE  dw.1  OR_ELSE  (bEver  :=  TRUE);
```

dw.8 is FALSE and dw.1 is TRUE, thus the result of the operation (bX) is TRUE. However, the expression at the third input is not executed, bEver remains FALSE. If you used the standard OR operation instead, bEver were set to TRUE.

__TRY, __CATCH, __FINALLY, __ENDTRY

Definition

These operators are not specified by the IEC 61131-3 standard.

The __TRY, __CATCH, __FINALLY, __ENDTRY operators are used for specific handling of exceptions in IEC code. With these operators, you can execute specific statements in case an error is detected. Furthermore, the program is not stopped (as usual) when an exception is detected.

Syntax

```
__TRY
<statements_try>
__CATCH(exec)
<statements_catch>
__FINALLY
<statements_finally>
__ENDTRY
<statements_next>
```

Function

If an exception is detected while <statements_try> is executed (even in functions called from there), then <statements_catch> is executed. In that way, execution does not stop as it is the case when other exceptions are detected. However, there is a log message in the runtime system with information about the range offset and the type of exception that has been detected.

After executing <statements_catch>, <statements_finally> is automatically executed (if programmed), and then <statements_next> is executed.

The variable <exception> must be of type __SystemExceptionCode.

Example

If the statement in __TRY produces an exception, then program execution is not stopped, and then the statements in __CATCH are executed. This means that the function exc is executed. Then the statement in __ENDTRY is executed.

```
FUNCTION Tester : UDINT
VAR_INPUT
     count : UDINT;
END_VAR
VAR_OUTPUT
```

```
      strExceptionText : STRING;
END_VAR
VAR
      exc : __SYSTEM.ExceptionCode;
END_VAR
__TRY
Tester := tryFun(count := count, testcase := g_testcase);
//This statement is tested. If it produces an exception, then the statement in __CATCH is executed f
__CATCH(exc)
HandleException(exc, strExceptionText => strExceptionText);
__FINALLY
GVL.g_count := GVL.g_count + 2;
__ENDTRY
```

Use the **Stop execution on handled exceptions** *command* to stop execution at the error location despite the programmed exception handling.

Type __System.Exception

```
TYPE ExceptionCode:
( RTSEXCPT_UNKNOWN                                  := 16#FFFFFFFF,
RTSEXCPT_NOEXCEPTION                               := 16#00000000,
RTSEXCPT_WATCHDOG                                   := 16#00000010,
RTSEXCPT_HARDWAREWATCHDOG                         := 16#00000011,
RTSEXCPT_IO_CONFIG_ERROR                           := 16#00000012,
RTSEXCPT_PROGRAMCHECKSUM                           := 16#00000013,
RTSEXCPT_FIELDBUS_ERROR                             := 16#00000014,
RTSEXCPT_IOUPDATE_ERROR                             := 16#00000015,
RTSEXCPT_CYCLE_TIME_EXCEED                        := 16#00000016,
RTSEXCPT_ONLCHANGE_PROGRAM_EXCEEDED             := 16#00000017,
RTSEXCPT_UNRESOLVED_EXTREFS                        := 16#00000018,
RTSEXCPT_DOWNLOAD_REJECTED                         := 16#00000019,
RTSEXCPT_BOOTPROJECT_REJECTED_DUE_RETAIN_ERROR   := 16#0000001A,
RTSEXCPT_LOADBOOTPROJECT_FAILED                    := 16#0000001B,
RTSEXCPT_OUT_OF_MEMORY                              := 16#0000001C,
RTSEXCPT_RETAIN_MEMORY_ERROR                      := 16#0000001D,
RTSEXCPT_BOOTPROJECT_CRASH                         := 16#0000001E,
RTSEXCPT_BOOTPROJECTTARGETMISMATCH               := 16#00000021,
RTSEXCPT_SCHEDULEERROR                              := 16#00000022,
```

```
RTSEXCPT_FILE_CHECKSUM_ERR                    := 16#00000023,
RTSEXCPT_RETAIN_IDENTITY_MISMATCH             := 16#00000024,
RTSEXCPT_IEC_TASK_CONFIG_ERROR                := 16#00000025,
RTSEXCPT_APP_TARGET_MISMATCH                  := 16#00000026,
RTSEXCPT_ILLEGAL_INSTRUCTION                  := 16#00000050,
RTSEXCPT_ACCESS_VIOLATION                     := 16#00000051,
RTSEXCPT_PRIV_INSTRUCTION                     := 16#00000052,
RTSEXCPT_IN_PAGE_ERROR                        := 16#00000053,
RTSEXCPT_STACK_OVERFLOW                       := 16#00000054,
RTSEXCPT_INVALID_DISPOSITION                  := 16#00000055,
RTSEXCPT_INVALID_HANDLE                       := 16#00000056,
RTSEXCPT_GUARD_PAGE                           := 16#00000057,
RTSEXCPT_DOUBLE_FAULT                         := 16#00000058,
RTSEXCPT_INVALID_OPCODE                       := 16#00000059,
RTSEXCPT_MISALIGNMENT                         := 16#00000100,
RTSEXCPT_ARRAYBOUNDS                          := 16#00000101,
RTSEXCPT_DIVIDEBYZERO                         := 16#00000102,
RTSEXCPT_OVERFLOW                             := 16#00000103,
RTSEXCPT_NONCONTINUABLE                       := 16#00000104,
RTSEXCPT_PROCESSORLOAD_WATCHDOG               := 16#00000105,
RTSEXCPT_FPU_ERROR                            := 16#00000150,
RTSEXCPT_FPU_DENORMAL_OPERAND                 := 16#00000151,
RTSEXCPT_FPU_DIVIDEBYZERO                     := 16#00000152,
RTSEXCPT_FPU_INEXACT_RESULT                   := 16#00000153,
RTSEXCPT_FPU_INVALID_OPERATION                := 16#00000154,
RTSEXCPT_FPU_OVERFLOW                         := 16#00000155,
RTSEXCPT_FPU_STACK_CHECK                      := 16#00000156,
RTSEXCPT_FPU_UNDERFLOW                        := 16#00000157,
RTSEXCPT_VENDOR_EXCEPTION_BASE                := 16#00002000
RTSEXCPT_USER_EXCEPTION_BASE                  :=16#00010000 )
UDINT ; END_TYPE
```

__VARINFO

Definition

This operator is not specified by the IEC 61131-3 standard.

Function

The __VARINFO operator provides information about a variable of the project at runtime. The information is stored as a data structure in a variable of data type __SYSTEM.VAR_INFO.

Example

At runtime, the variable MyVarInfo contains the information about the variable MyVar.

```
//Declaration
VAR
    MyVarInfo:  __SYSTEM.VAR_INFO
    MyVAR:  INT;
END_VAR

//Program code
MyVarInfo:=  __VARINFO  (MyVar);
```

Type SYSTEM.VAR_INFO

A variable with data type __SYSTEM.VAR_INFO contains the following elements:

Element	Description
ByteAddress	Address of the variable.
ByteOffset	Offset (in bytes).
Area	Number of the memory area.
BitNr	Number of the bit in the byte. If it is not a bit type, the value is -1.
BitSize	Size of the variable (in bits).
BitAddress	Bit address of the variable.
TypeClass	Data type class of the variable.
TypeName	Data type of the variable.
NumElements	For arrays: Number of array elements.

BaseTypeClass	For arrays: Data type class of the base data type.
ElemBitSize	For arrays: Bit size of an array element.
MemoryArea	Information for the memory area: memory, input, output, retain, global, local.
Symbol	Variable name.
Comment	Comment.

Scope Operators

Definition

In extension to the IEC operators, there are several possibilities to disambiguate the access to variables or modules if the variables or module name is used multiple times within the scope of a project.

The following scope operators can be used:
- global scope operator
- global variable list name
- enumeration name
- library namespace
- global node operator

Global Scope Operator

An instance path starting with dot (.) opens a global scope (namespace). Therefore, if there is a local variable with the same name `<varname>` as a global variable, then `.<varname>` refers to the global variable.

Global Variable List Name

You can use the name of a global variable list as a namespace for the variables enclosed in this list. Thus, it is possible to declare variables with identical names in different global variable lists and, by preceding the variable name by `<global variable list name>.`, it is possible to access the desired one.

Syntax

<global variable list name>.<variable>

Example

The global variable lists `globlist1` and `globlist2` each contain a variable named `varx`. In the following line, `varx` out of `globlist2` is copied to `varx` in `globlist1`:

```
globlist1.varx := globlist2.varx;
```

If a variable name declared in more than one global variable lists is referenced without the global variable list name as a preceding operator, a message will be generated.

Library Namespace

You can add the library namespace to a POU as a prefix separated by a dot to make the access to the POU unique. By default, the `namespace` of a library is identical to the library name.

Example: `LIB_A.FB_A`

Syntax

```
<library namespace>.<library POU>
```

Example

If a library which is included in a project contains the POU `FB_A` and there is also a POU `FB_A` defined locally in the project, then you can assign the name `LIB_A.FB_A` to the function block of the library in order to differentiate from the POU.

```
var1 := FB_A(in := 12); // Call of the project function block FB_A
var2 := LIB_A.FB_A(in := 22); // Call of the library function block FB_A
```

You can define another name for the namespace either in the **Project Information** when creating a library project in the **Project Information** (by default in the **Project** menu), or later in the **Properties** dialog box of an included library in the **Library Manager**.

Enumeration Name

You can use the type name of an enumeration to disambiguate the access to an enumeration constant. Therefore, it is possible to use the same constant in different enumerations.

The enumeration name has to precede the constant name, separated by a dot (.).

Syntax

<enumeration name>.<constant name>

Example

The constant `Blue` is a component of enumeration `Colors` as well as of enumeration `Feelings`.

```
color := Colors.Blue; // Access to enum value Blue in type Colors
feeling := Feelings.Blue; // Access to enum value Blue in type Feelings
```

Global Node Operator

Variables declared in GVLs or POUs of the **Global** node of the **Applications tree** can be accessed by prefixing them with the operator "__POOL.".

Initialization Operator

INI Operator

Overview

NOTE: The INI operator is obsolete. The method FB_init replaces the INI operator. For further information about the FB_init method, refer to the chapter FB_init, FB_reinit Methods. However, the operator can still be used for keeping compatibility with projects imported from earlier EcoStruxure Machine Expert versions.

You can use the INI operator to initialize retain variables which are provided by a function block instance used in the POU.

Assign the operator to a boolean variable.

Syntax

<bool-variable> := INI(<FB-instance, TRUE|FALSE)

If the second parameter of the operator is set to TRUE, all retain variables defined in the function block FB will be initialized.

Example in ST

fbinst is the instance of function block fb, in which a retain variable retvar is defined.

Declaration in POU

```
fbinst:fb;
b:bool;
```

Implementation part

```
b := INI(fbinst, TRUE);
ivar:=fbinst.retvar (* => retvar gets initialized *)
```

Example of Operator Call in FBD

Operands

What Is in This Chapter?

This chapter contains the following sections:

- Constants
- Variables
- Addresses
- Functions

Constants

What Is in This Section?

This section contains the following topics:

- BOOL Constants
- TIME Constants
- DATE Constants
- DATE_AND_TIME Constants
- TIME_OF_DAY Constants
- Number Constants

- REAL/LREAL Constants
- String Constants
- Typed Constants / Typed Literals

BOOL Constants

Overview

BOOL constants are the logical values TRUE and FALSE.

Refer to the description of the data type BOOL.

TIME Constants

Overview

TIME constants are used to operate the standard timer modules. The time constant TIME is of size 32 bit and matches the IEC 61131-3 standard. Additionally, LTIME is supported as an extension to the standard as time base for high-resolution timers. LTIME is of size 64 bit and resolution nanoseconds.

The library standard64.lib provides functions for WSTRING strings.

Syntax for TIME Constant

t#<time declaration>

Instead of `t#`, you can also use the following:

- `T#`
- `time`
- `TIME`

The time declaration can include the following time units. They have to be used in the following sequence, but it is not required to use all of them.

- d: days
- h: hours
- m: minutes

- s: seconds
- ms: milliseconds

Examples of correct TIME constants in an ST assignment

Example	Description
`TIME1 := T#14ms;`	–
`TIME1 := T#100S12ms;`	(* The highest component may be allowed to exceed its limit *)
`TIME1 := t#12h34m15s;`	–

Examples of incorrect usage

Example	Description
`TIME1 := t#5m68s;`	(* limit exceeded in a lower component *)
`TIME1 := 15ms;`	(* T# is missing *)
`TIME1 := t#4ms13d;`	(* incorrect order of entries *)

Syntax for LTIME Constant

LTIME#<time declaration>

The time declaration can include the time units as used with the TIME constant and additionally:

- us: microseconds
- ns: nanoseconds

Examples of correct LTIME constants in an ST assignment:

```
LTIME1 := LTIME#1000d15h23m12s34ms2us44ns
LTIME1 := LTIME#3445343m3424732874823ns
```

For further information, refer to the description of the TIME data types.

DATE Constants

Overview

Use these constants to enter dates.

Syntax

d#<date declaration>

Instead of `d#` you can also use the following:

- `D#`
- `date`
- `DATE`

Enter the date declaration in format <year-month-day>.

DATE values are internally handled as DWORD values, containing the time span in seconds since 01.01.1970, 00:00 clock.

Examples

```
DATE#1996-05-06
d#1972-03-29
```

For further information, refer to the description of the TIME data types.

DATE_AND_TIME Constants

Overview

DATE constants and TIME_OF_DAY constants can also be combined to form so-called DATE_AND_TIME constants.

Syntax

`dt#`<date and time declaration>

Instead of `dt#` you can use the following:

- `DT#`
- `date_and_time`
- `DATE_AND_TIME`

Enter the date and time declaration in format <year-month-day-hour:minute:second>.

You can enter seconds as real numbers. This allows you to specify fractions of a second.

DATE_AND_TIME values are internally handled as DWORD values, containing the time span in seconds since 01.01.1970, 00:00 clock.

Examples

```
DATE_AND_TIME#1996-05-06-15:36:30
dt#1972-03-29-00:00:00
```

For further information, refer to the description of the TIME data types.

TIME_OF_DAY Constants

Overview

Use this type of constant to store times of the day.

Syntax

tod#<time declaration>

Instead of `tod#` you can also use the following:

- `TOD#`
- `time_of_day#`
- `TIME_OF_DAY#`

Enter the time declaration in format <hour:minute:second>.

You can enter seconds as real numbers. This allows you to specify fractions of a second.

TIME_OF_DAY values are internally handled as DWORD values, containing the time span in milliseconds since 00:00 clock.

Examples

```
TIME_OF_DAY#15:36:30.123
tod#00:00:00
```

For further information, refer to the description of the TIME data types.

Number Constants

Overview

Number values can appear as binary numbers, octal numbers, decimal numbers, and hexadecimal numbers. Integer values that are not decimal numbers are represented by the base followed by the number sign (#) in front of the integer constant. The values for the numbers 10...15 in hexadecimal numbers are represented by the letters A...F.

You can include the underscore character within the number.

Examples

`14`	(decimal number)
`2#1001_0011`	(dual number)
`8#67`	(octal number)
`16#A`	(hexadecimal number)

These number values can be of type:

- BYTE
- WORD
- DWORD
- SINT
- USINT
- INT
- UINT
- DINT
- UDINT
- REAL
- LREAL

Implicit conversions from larger to smaller variable types are not permitted. This means that a DINT variable cannot simply be used as an INT variable. Use the type conversion functions.

REAL/LREAL Constants

Overview

REAL and LREAL constants can be given as decimal fractions and represented exponentially. Use the standard American format with the decimal point to do this.

Examples

`7.4`	instead of `7,4`
`1.64e+009`	instead of `1,64e+009`

String Constants

Overview

A string constant is an arbitrary sequence of characters. STRING constants are preceded and followed by single quotation marks. WSTRING constants are preceded and followed by double quotation marks. The characters are coded according to the character set specified in ISO/IEC 8859-1. You may also enter blank spaces and special characters (special characters for different languages, like accents or umlauts).

In strings, the combination of the dollar sign ($) followed by 2 hexadecimal numbers is interpreted as a hexadecimal code according to the coding in ISO/IEC 8859-1. The code corresponds to ASCII code. In addition, note the special cases presented in the table.

Hexadecimal Code

Combinations of characters starting with a dollar sign which are interpreted as hexadecimal code:

String with $ code	Interpretation
`'$<8-bit code>'`	8-bit code: Two-digit hexadecimal number that is interpreted according to ISO/IEC 8859-1.
`'$41'`	A
`'$9A'`	©
`'$40'`	@
`'$0D'`	Control character: Line break (corresponds to `'$R'`)
`'$0A'`	Control character: New line (corresponds to `'$L'` and `'$N'`)

Special Cases of a STRING

Combinations of characters starting with a dollar sign which have a specific meaning:

String with $ code	Interpretation
'$L', '$l'	Control character: Line feed (corresponds to '$0A')
'$N', '$n'	Control character: New line (corresponds to '$0A')
'$P' '$p'	Control character: Form feed
'$R', '$r'	Control character: Line break (corresponds to '$0D')
'$T', '$t'	Control character: Tab
'$$'	Dollar sign $
'$''	Single straight quotation mark: '

Examples

Constant declaration of a STRING:

```
VAR CONSTANT
     constA : STRING := 'Hello world';
     constB : STRING := 'Hello world $21'; // Hello world!
END_VAR
```

Examples of WSTRING declarations:

```
wstr:WSTRING:="This is a WString";
```

```
wstr10 : WSTRING(10) := "1234567890";
```

Typed Constants / Typed Literals

Overview

Basically, in using IEC constants, the smallest possible data type will be used. An exception is REAL/LREAL constants where LREAL is always used. If another data type has to be used, use typed literals (typed constants) without the necessity of explicitly declaring the constants. For this purpose, the constant will be provided with a prefix which determines the type.

Syntax

`<Type>#<Literal>`

`<Type>` is the desired data type. Possible entries are:

- BOOL
- SINT
- USINT
- BYTE
- INT
- UINT
- WORD
- DINT
- UDINT
- DWORD
- REAL
- LREAL

Write the type in uppercase letters.

`<Literal>` specifies the constant. The data entered has to fit within the data type specified in `<Type>`.

Example

```
var1:=DINT#34;
```

If the constant cannot be converted to the target type without data loss, a message will be generated.

You can use typed literals wherever normal constants can be used.

Variables

What Is in This Section?

This section contains the following topics:

- Variables
- Addressing Bits in Variables

Variables

Overview

You can declare variables either locally in the declaration part of a POU or in a global variable list (GVL) or in a persistent variables list or in the I/O mapping of devices.

Refer to the chapter Variables Declaration for information on the declaration of a variable, including the rules concerning the variable identifier and multiple use.

It depends on the data type where a variable can be used.

You can access available variables through the **Input Assistant**.

Accessing Variables for Arrays, Structures, and POUs

The table lists the respective syntax for accessing arrays, structures, and POUs:

Syntax	Access to
<array name>[Index1, Index2]	2-dimensional array components
<structure name>.<variable name>	structure variables
<function block name>.<variable name>	function block and program variables

Addressing Bits in Variables

Overview

In integer variables, individual bits can be accessed. For this purpose, append the index of the bit to be addressed to the variable and separate it by a dot. You can give any constant to the bit index. Indexing is 0-based.

Syntax

<variablename>.<bitindex>

Example

```
a  :  INT;
b  :  BOOL;
...
a.2  :=  b;
```

The third bit of the variable `a` will be set to the value of the variable `b`, this means that variable `a` will equal 3.

If the index is greater than the bit width of the variable, the following message will be generated:

'Index '<n>' outside the valid range for variable '<var>'!'

Bit addressing is possible with variables of the following data types:

- SINT
- INT
- DINT
- USINT
- UINT
- UDINT
- BYTE
- WORD
- DWORD

If the data type does not allow bit accessing, the following message will be generated:

'Invalid data type '<type>' for direct indexing'.

Do not assign bit access to a `VAR_IN_OUT` variable.

Bit Access Via a Global Constant

If you have declared a global constant defining the bit index, you can use this constant for a bit access.

Example for a bit access via a global constant and on a variable:

1. Declaration of the global constant in a global variable list

The variable `enable` defines the bit that is accessed:

```
VAR_GLOBAL  CONSTANT
      enable:int:=2;
END_VAR
```

2. Bit access on an integer variable

Declaration in POU:

```
VAR
      xxx:int;
END_VAR
```

Bit access:

```
xxx.enable := true; (* -> the third bit in variable xxx will be set TRUE *)
```

Bit Access on BIT Data Types

The BIT data type is a special data type which is only allowed in structures. For further information, refer to Bit Access in Structures.

Example: Bit access on BIT data types

Declaration of structure

```
TYPE ControllerData :
STRUCT
      Status_OperationEnabled : BIT;
      Status_SwitchOnActive : BIT;
      Status_EnableOperation : BIT;
      Status_Error : BIT;
      Status_VoltageEnabled : BIT;
      Status_QuickStop : BIT;
      Status_SwitchOnLocked : BIT;
      Status_Warning : BIT;
END_STRUCT
END_TYPE
```

Declaration in POU

```
VAR
      ControllerDrive1:ControllerData;
END_VAR
```

Bit access

```
ControllerDrive1.Status_OperationEnabled := TRUE;
```

Addresses

Direct Addresses

Overview

A direct address specified in EcoStruxure Machine Expert contains the following information:

- Information on the memory location.
- Memory format (size)
- Offset of the memory location. The offset is specified by an integer number, which in case of a bit address is followed by a dot and a number for the position of the bit.

Syntax

%<memory area prefix><size prefix><number|.number|.number....>

The following memory area prefixes are supported:

I	input (physical inputs via input driver, sensors)
Q	output (physical outputs via output driver, actors)
M	memory location

The following size prefixes are supported:

X	single bit
None	single bit
B	byte (8 bits)
W	word (16 bits)
D	double word (32 bits)

Examples

Example address	Description
%QX7.5	output bit 7.5
%Q7.5	
%IW215	input word 215

%QB7	output byte 7
%MD48	double word in memory position 48 in the memory location
ivar AT %IW0: WORD;	example of a variable declaration including an address assignment For further information, refer to the AT Declaration chapter.

NOTE: The memory size for input, output, and memory data (declarations with AT %I, %Q and %M) is predefined by the target device and can be overwritten in the *properties of an application object* for PacDrive controllers (PacDrive LMC Eco, PacDrive LMC Pro/Pro2).

Byte Addressing Mode and Word Addressing Mode

Devices either use byte addressing mode or word addressing mode.

Examples

Mode	Example
Byte addressing	`ADR(%IW1) = ADR(%IB1)`
Word addressing	`ADR(%IW1) = ADR(%IB2)`

The range for the second element of the bit address that is the number following the dot, is as follows:
- byte addressing mode: 0...7
- word addressing mode: 0...15

Also for the handling of bit addresses, you can configure the devices differently. They are then interpreted correspondingly by the EcoStruxure Machine Expert compiler.

Example: In a byte-addressing device, the bit address %IX2.5 addresses byte 2 (IB2). In a word-addressing device, however, it addresses word 2, which refers to a different location within the memory.

NOTE: Boolean values are allocated bytewise if no explicit single-bit address is specified. Example: A change in the value of varbool1 AT %QB7 affects the range from QX0.0 to QX0.7.

Functions

Functions

Overview

In ST, a function call can be used as an operand.

Example

```
Result := Fct(7) + 3;
```

For a general description of functions and their declarations, refer to the Function description in the Program Organization Unit (POU) section of this Programming Guide.

TIME() Function

This function returns the time (based on milliseconds) which has been passed since the system was started.

The data type is TIME.

Example in IL

```
TIME
ST systime   (* Result for example: T#35m11s342ms *)
```

Example in ST

```
systime:=TIME();
```